Reproducibility Engineering - Module 1 Exam Guide

The terminology below follows **ACM Artifact Review and Badging v1.1**, which is the convention used by the course. Terminology has not been uniform: after NISO guidance, ACM itself swapped *reproducibility* and *replicability* in 2020. In an exam answer, use the current v1.1 mapping, not older ACM material, and state the convention consistently.

1. The module in one page

The three R's

Concept	Team	Experimental setup/artifacts	Computational interpretation
Repeatability	Same team	Same setup	The original researcher can rerun the computation reliably.
Reproducibility	Different team	Same setup; author-supplied artifacts	An independent group gets an agreeing result using the author's artifacts.
Replicability	Different team	Different, independently created setup/artifacts	An independent group rebuilds the experiment and still gets an agreeing result.

Fast decision rule:

1. **Same team and same setup?** -> repeatability.
2. **Different team?** Ask whether it reuses the original setup/artifacts.
   - Reuses them -> reproducibility.
   - Rebuilds independently -> replicability.
3. **Same team but different setup?** -> not separately classified by this three-term ACM scheme; explain the facts rather than forcing a label.

Mnemonic: **1S, 2S, 2D** = same team/same setup, different team/same setup, different team/different setup.

Nature survey numbers

Of 1,576 surveyed researchers:

- **52%**: yes, a significant crisis
- **38%**: yes, a slight crisis
- **3%**: no crisis
- **7%**: do not know

Therefore, **90% perceived at least some crisis**, but the survey measured researchers' perceptions, not the true fraction of invalid scientific results.

Docker in one sentence

A **Dockerfile** describes the environment; `docker build` reads it to create a read-only **image**; `docker run` creates a **container**, an isolated instance with its own file system that shares the host's operating-system kernel.

Exact identity versus similarity

- **SHA-256** answers: "Are these files byte-for-byte the same?"
- **Pearson correlation** answers: "How strongly do these aligned pixel values vary together?"
- A correlation close to `1` does **not** prove that two files or images are identical.

2. Why reproducibility matters

Science depends on claims surviving checks beyond the original analysis. Repeating, reproducing, or replicating an experiment can:

- confirm and verify a reported result;
- reveal hidden assumptions, undocumented steps, or environment dependencies;
- distinguish a robust effect from a chance result or false lead;
- make errors easier to detect and correct;
- increase confidence and allow later researchers to build on the work.

Important distinction: a failure to reproduce is **evidence that needs investigation**, not automatic proof that the original claim is false. Differences can arise from procedures, data, materials, software, hardware, statistical power, or tacit knowledge.

What the Nature survey found

The assigned Nature material reports that:

- more than **70%** had tried and failed to reproduce another scientist's experiment;
- more than **half** had failed to reproduce one of their own experiments;
- fewer than **31%** thought a failed reproduction means the original result is probably wrong;
- **73%** still believed that at least half of the papers in their field could be trusted;
- only a minority had tried to publish a reproduction attempt;
- successful and failed reproduction attempts both faced weak publication incentives.

This apparent tension is exam-worthy: many scientists perceived a crisis and had experienced failures, yet most still trusted much of the literature. Reproducibility is not a binary synonym for truth.

Lower-priority article numbers to recognize:

- failed attempts to reproduce someone else's work were reported by about **87% in chemistry, 77% in biology, 69% in physics/engineering, 67% in medicine, 64% in earth/environment, and 62% in other fields**;
- empirical estimates cited in the article were roughly **40% reproducibility in psychology** and **10% in cancer biology**;
- **24%** reported publishing a successful reproduction and **13%** an unsuccessful one; **12%** and **10%**, respectively, reported being unable to publish those attempts;
- regarding lab procedures, **26%** said procedures existed when they joined, **33%** introduced them in the previous five years, **7%** had introduced them earlier, and **34%** reported none.

Do not confuse any of these percentages with the 52/38/3/7 crisis-perception chart.

Vocabulary warning: the 2016 Nature article uses *reproduce* and *replication* informally and notes that no consensus existed. Its "failed to reproduce" percentages do not by themselves specify current-ACM reproducibility (different team using author artifacts).

Main causes identified

Group the causes into three levels:

Level	Examples
Research design and analysis	Poor experimental design, low statistical power, weak analysis, selective reporting, insufficient repetition in the original lab
Artifacts and communication	Missing methods or code, unavailable raw data, incomplete documentation, specialized techniques, variable materials/reagents
Research culture	Pressure to publish, insufficient mentoring or oversight, insufficient peer review, competition for grants and jobs

More than 60% of respondents said that **pressure to publish** and **selective reporting** often or always contributed. More than half also pointed to insufficient replication, poor oversight, or low statistical power.

Proposed improvements

The strongest survey support was for:

- more robust experimental design;
- better statistics;
- better mentorship;
- redoing important work within the lab;
- clearer, standardized methods and documentation;
- making code, data, and workflows available;
- preregistering hypotheses and analysis plans when appropriate;
- stronger action and incentives from journals, funders, and institutions.

Nearly 90% endorsed robust design, better statistics, and better mentorship. Around 80% thought funders and publishers should do more.

Survey caveat

The questionnaire was advertised to Nature readers and through related websites/social media as a survey about reproducibility. This can create **self-selection bias**: people already interested in the issue may be more likely to respond. Treat the results as evidence about the experiences and views of respondents, not a precise estimate for all scientists or all published work.

3. Repeatability, reproducibility, and replicability

How to justify a classification

A complete classification identifies **who performed the new run**, **whether the setup/artifacts were reused or rebuilt**, and **whether the result agreed within the stated precision**.

- Repeatability example: you rerun your own computation with the same code, inputs, environment, parameters, and procedure.
- Reproducibility example: an independent evaluator uses the paper's code, data, container, and instructions to regenerate its main tables.
- Replicability example: another group independently implements the published method and reaches a result supporting the same main claim.

"Same result" does not mean identical digits

ACM requires agreement within a tolerance appropriate for the experiment. Exact equality may be impossible because of randomness, floating-point arithmetic, hardware, sampling, or measurement noise. The decisive question is whether differences invalidate or materially change the paper's main claims.

4. Research artifacts and ACM badges

What is an artifact?

In ACM's badging policy, an artifact is a digital object created for a study or generated by it. Examples include:

- source code and executable software;
- experiment, build, and analysis scripts;
- input datasets, collected raw data, and processed data;
- configuration files, parameters, and random seeds;
- notebooks, queries, schemas, and workload definitions;
- environment descriptions, lockfiles, Dockerfiles, and container recipes;
- logs, intermediate results, tables, plots, and scripts that generate them;
- documentation, licenses, checksums, and provenance metadata.

The course exercises sometimes use *artifact* more broadly, including physical equipment. For an ACM badge, the formal focus is the associated digital objects.

The three independent badge families

The badges are independent: a paper may earn one, several, or all of them.

1. Artifacts Evaluated

The artifacts passed an independent audit. Public availability is not required, but reviewers must receive them.

- **Functional**: documented, consistent with the paper, complete as far as possible, exercisable, and supported by verification/validation evidence.
- **Reusable**: satisfies Functional and is additionally well structured, carefully documented, and prepared for reuse according to community standards.

Functional and Reusable are alternative levels within this badge family. Reusable subsumes Functional, so only one of the two is applied in an instance.

2. Artifacts Available

Relevant author-created artifacts are in a **publicly accessible archival repository**, with a DOI or link and a unique identifier.

Key consequences:

- use a repository with a plan for long-term access;
- an institutional repository, publisher repository, Zenodo, Figshare, or Dryad can qualify;
- a personal web page is not sufficient;
- availability does not imply that the artifacts were evaluated, complete, or reusable.

3. Results Validated

Another person or team successfully obtained the paper's main results.

- **Results Reproduced**: the independent team used at least some author-supplied artifacts.

- **Results Replicated**: the independent team did not use author-supplied artifacts.

These result badges may be awarded after publication; ACM requires a peer-reviewed reproduction or replication report as evidence.

5. Docker mental model

Core objects

Object	Meaning
Dockerfile	Text instructions that describe how to build an image.
Image	Read-only template containing the application environment; a blueprint.
Container	A created/running instance of an image with isolation and a writable container layer.
Host	The machine on which the Docker engine and containers run.

Layered architecture

From top to bottom:

1. application inside the container's isolated file system;
2. Docker/container engine managing the container;
3. host file system;
4. physical or remote host machine.

Containers share the host's **operating-system kernel**, even though their file systems and processes are isolated. This is why containers are more lightweight than virtual machines that normally include a separate guest operating system/kernel.

Local versus course server:

- Local Docker: your own PC is the **host**.
- Remote Docker server: the server is the **host**; your PC is a client connected through SSH.

Why Docker helps reproducibility

An image can package the application with its libraries, tools, and configuration, reducing "works on my machine" variation. Docker does not guarantee reproducibility by itself. Results can still depend on unpinned packages or base images, input data, random seeds, CPU/GPU architecture, host kernel behavior, network services, clock/time, or undocumented commands.

For stronger reproducibility, pin versions or image digests, archive inputs, record seeds and hardware, automate the full workflow, and document expected outputs.

6. Docker commands you should recognize

Important flags

Flag	Meaning
`docker run -i`	Keep standard input open.
`docker run -t`	Allocate a pseudo-terminal.
`docker run -it`	Interactive terminal (`-i` plus `-t`).
`docker run -d`	Run detached/in the background.
`--name NAME`	Give the container a stable human-readable name.
`-v HOST:CONTAINER`	Bind-mount a host path at a container path.

Trap: `-t` means **tag** for `docker build`, but **pseudo-terminal** for `docker run`.

7. Comparing the two images

Method 1: visual inspection and metadata

Visual appearance, file size, and timestamps are quick clues, but none proves content identity.

- Same size does not imply same bytes.
- Different size does prove the files are not byte-identical.
- Timestamps describe metadata/history, not content.
- A hidden or subtle pixel change can be invisible to a person.

Method 2: SHA-256

- Different hashes -> the byte sequences are definitely different.
- Equal SHA-256 hashes -> treat the files as byte-identical in normal engineering practice; a deliberate hash collision is theoretically possible but negligible here.

Checksums are appropriate for integrity and exact artifact verification.

Method 3: Pearson correlation of pixels

For paired values `(x_i, y_i)`, Pearson's `r` measures linear association:

- `r = 1`: perfect positive linear relationship;
- `r = 0`: no linear correlation (a nonlinear relationship can still exist);
- `r = -1`: perfect negative linear relationship.

A value near `1` means the overall aligned grayscale patterns are extremely similar. It does **not** mean the images or files are identical: a small embedded message can alter bytes and pixels while barely changing the global correlation. Converting to grayscale can also discard color differences.

Exam comparison:

Question	Best method
Are the files byte-for-byte identical?	SHA-256/checksum
Do the images look similar overall?	Visual/perceptual or statistical comparison
Are aligned pixel intensities linearly similar?	Pearson correlation

9. Common exam traps

1. **Different person does not automatically mean replication.** If that person uses the original artifacts/setup, it is reproduction.
2. **Different physical location does not automatically mean a different setup.** Ask whether the measuring system/artifacts were reused or independently rebuilt.
3. **Reproducible does not mean correct.** A flawed deterministic analysis can be perfectly reproducible.
4. **A failed reproduction does not automatically falsify a claim.** Investigate procedures, conditions, data, and tolerances.
5. **Artifact Available is not Artifact Evaluated.** Public presence says nothing by itself about functionality or reusability.
6. **A Git repository is not automatically permanent archival storage.** Prefer an immutable release with a DOI or unique persistent identifier.
7. **An image is not a container.** The image is the template; the container is an instance.
8. **The host and container have different file systems.** Use `docker cp` or a bind mount deliberately.
9. **`docker run`, `start`, and `exec` are not synonyms.** Create, restart, and execute-inside are different operations.
10. **Same size or high correlation is not byte identity.** Use a checksum for exact comparison.
11. **The container shares the host kernel.** It does not contain a complete independent guest kernel like a typical VM.
12. **Survey percentages are perceptions.** They do not directly measure what percentage of all science is false.

10. Rapid-recall checklist

Before the exam, you should be able to say each answer without notes:

- 52 significant, 38 slight, 3 no, 7 do not know; 1,576 respondents.
- More than 70% failed to reproduce someone else's experiment; more than half failed to reproduce their own.
- Same team/same setup = repeatability.
- Different team/same author artifacts = reproducibility.
- Different team/independent artifacts = replicability.
- Artifact Available = public archival repository plus DOI/link and unique identifier.
- Functional = documented, consistent, complete, exercisable, and verified/validated.
- Results Reproduced uses author artifacts; Results Replicated does not.
- Dockerfile `--docker build`-> image `--docker run`-> container.
- Containers share the host kernel but have isolated file systems.
- `docker exec CONTAINER COMMAND` runs a command inside a running container.
- `docker cp CONTAINER:path hostpath` copies out; reverse the arguments to copy in.
- `-v HOST:CONTAINER` creates a bind mount.
- SHA-256 tests byte identity; correlation tests linear similarity.
- High correlation does not prove equality.

Reproducibility Engineering - Module 2 Exam Guide

The whole module in three lines

- **DPC: D**epth = what is shared; **P**ortability = where it runs; **C**overage = how much can be rerun.
- **PRE: P**rospective provenance = planned recipe; **R**etrospective provenance = actual run; workflow **E**volution = version history.
- **BSG: B**ronze shares artifacts; **S**ilver adds setup, details, and determinism; **G**old adds one-command full automation.

1. What is a reproducible computational experiment?

An experiment developed at time `t`, on system `s`, with data `d` is reproducible if it can later be executed at time `t'`, on system `s'`, with the same or sufficiently similar data `d'`, and produce **consistent results**. Reproducing an experiment therefore needs more than its final plot.

Minimum anatomy

1. **Input data description**
   - **By extension:** provide the actual data.
   - **By intention:** provide a precise procedure/script that obtains or derives the data.
2. **System description:** hardware, operating system, software, libraries, versions, and relevant configuration.
3. **Executable specification:** the ordered computational steps that derive the result.

The key idea is:

```
inputs + environment + executable process -> result
```

A paper that supplies only prose, tables, or figures shows the result but usually does not preserve the full derivation.

Three dimensions: Depth, Portability, Coverage

These dimensions are **independent criteria**, not three stages of one ladder.

Dimension	Exam definition	Scale	Diagnostic question
Depth	How much of the experiment is made available	Figures only -> scripts and data -> raw/intermediate data -> full setup/protocol -> software and build environment	**What** was shared?
Portability	The range of environments in which it can be rerun	Original environment -> similar environment -> different environment	**Where** can it run?
Coverage	The fraction of the experiment that can be reproduced	Partial -> full	**How much** can be rerun?

Depth in increasing order

1. Figures in the manuscript.
2. Figure-generating script or spreadsheet plus the appropriate data.
3. Raw data and intermediate results.
4. Full experimental setup: initialization, scripts, workload, configuration, and measurement protocol.
5. The software as a **white box** (source, configuration, build environment) or **black box** (executable).

Portability and terminology

The assigned article defines portability operationally:

- reruns in the **original environment**;
- reruns in a **similar environment**, such as the same OS on another machine (reproducibility);
- reruns in a **different environment**, such as another OS or machine.

Coverage example: special hardware

Suppose special hardware generated the raw data and other researchers cannot access it. Sharing the hardware output plus the entire downstream analysis still gives **partial reproducibility**. Data generation is not covered,

but the analysis and plot generation are. This is better than making no part reproducible.

2. Workflows and provenance

Workflows and dataflows

A workflow represents computational steps as **modules** connected by dependencies. Dependencies can be control-driven or data-driven. When they are data-driven, the workflow is a **dataflow**, naturally represented as a directed acyclic graph (**DAG**): nodes are modules and edges carry data between them.

A **workflow instance** combines this structure with one concrete configuration: its input data and parameter values.

Why workflows help reproducibility:

- The experiment has an explicit, executable structure.
- Repetitive tasks are automated.
- The system can capture provenance transparently.
- A visual interface can make composition understandable to non-programmers.
- Groups of modules can be abstracted to hide irrelevant detail.
- The same workflow can run with different inputs/parameters for parameter sweeps, sensitivity analysis, and side-by-side comparison.

Important limitation: a workflow is **not automatically portable**. Hard-coded paths, absent libraries, OS incompatibilities, changing dependencies, external services, or hardware differences can still prevent execution.

The three provenance types

Provenance is information about where a result came from and how it was produced. Workflow systems can capture three types.

Type	Meaning	Typical contents	Best mnemonic
Prospective provenance	The intended experiment specification	Modules, connections, inputs, parameters, workflow structure	**Recipe**
Retrospective provenance	What actually happened during one execution	Start/end times, machine, executed modules, intermediate dependencies, success/failure, errors	**Run diary**
Workflow evolution provenance	How the workflow changed over time	All versions, branches, edits, parameter/module changes, relation between versions	**Version tree**

Exam traps:

- A workflow definition is **prospective**, even before it is executed.
- An execution log is **retrospective**, even if it contains an error.
- Evolution provenance records changes among workflow versions, not merely a list of output files.

VisTrails: why it matters

VisTrails is a provenance-aware scientific workflow system built for exploratory work, where changing the workflow is normal. It manages computations, data, and provenance together.

High-yield features:

- A **version tree**: each node is a workflow version; each edge is an action or sequence of actions that transforms parent into child.
- Old versions are retained rather than overwritten, so users can branch, undo without losing results, compare versions, and reproduce earlier outputs.
- A workflow editor plus a visual spreadsheet for side-by-side results and parameter sweeps.
- Tags, annotations, authors, timestamps, notes, and searchable metadata.
- Provenance for data products, workflow specifications, and actual executions.
- Execution records with timings, errors, success/failure, and the system used.
- Workflow analogies and completion suggestions that reuse patterns from provenance.

A **vistrail** is the complete bundle of related workflows, the changes that distinguish their versions, and the provenance of their executions. A workflow DAG and a version tree must not be confused: DAG nodes/edges are modules/data dependencies; version-tree nodes/edges are workflow versions/edit actions.

Workflow upgrades

Software evolves and old modules become stale. VisTrails compares a workflow's module version with the available version and attempts an upgrade. A compatible interface may be upgraded automatically; complex changes need a developer-provided upgrade path or user intervention. Crucially, the upgrade itself is recorded as evolution provenance and the original workflow is retained. One can therefore compare the old version in a compatible virtual machine with the upgraded version.

Data versions and databases

- A filename is not enough because the file may be changed, moved, or deleted. VisTrails records a **content hash** to verify identity and can keep input, output, and intermediate files in a versioned repository. A hash only verifies content; it does **not preserve the file**. The repository supplies preservation and recovery.
- Output data can be strongly linked to the computation that generated it through matching hashes and provenance traces.
- Workflows are normally stateless and deterministic, while databases are **stateful** and change over time. VisTrails addresses this mismatch with temporal database states recorded in provenance, allowing an earlier state to be restored for reproduction.

Linking publications to computations

VisTrails can bind a figure to the exact workflow that generated it. Its LaTeX mechanism can recalculate a result and embed a link to the workflow, identified by a tag or unique ID. Readers can inspect, rerun, and modify the computation instead of seeing only a static picture. Similar mechanisms were developed for wikis, web pages, Word, PowerPoint, and the crowdLabs site.

An **interactive visualization alone is not enough** if its underlying computation cannot be inspected or executed. Download/execute/modify access is preferable. crowdLabs reduces local setup through server-side execution, lets mashup users change parameters without local VisTrails installation, and supports comments. Retrospective dependency traces can also flag every result derived from a faulty input, such as a malfunctioning sensor.

Limits and related tools - lower priority

End-to-end, long-term reproducibility can still fail because of specialized hardware, proprietary data, changing hardware/software, the work needed to wrap an existing experiment as a workflow, or interactive tools that cannot be wrapped cleanly. The transferable lesson is the provenance design, not dependence on one permanent tool.

Know the related-tool names only at recognition level:

- **Madagascar:** reproducible geophysics documents using SCons and LaTeX.
- **Sweave:** embeds R code in LaTeX to create dynamic reports.
- **ReproZip:** captures an existing experiment and packages what is needed to rerun it elsewhere; it also derives a workflow specification.
- **SOLE / Collage / SHARE:** respectively link scientific objects to papers, create executable papers, and share citable remote virtual machines.
- **VCR/VRI:** a Verifiable Result Identifier links a result to a repository containing both the result and its computational process.

3. Bronze, Silver, and Gold reproducibility

Heil et al. define **computational reproducibility** as a third party obtaining the same results with the authors' published **data, trained models, and code**. The purpose is trust: outsiders can check accuracy, inspect implementation, and find biases or artifacts the original authors did not know about.

Reproducibility is a continuum measured by the time/effort needed to reproduce:

```
"forever" / effectively impossible ---- Bronze ---- Silver ---- Gold ----
approximately zero effort
```

The table to memorize exactly

The standards are **cumulative**: Gold includes Silver and Bronze; Silver includes Bronze.

Requirement	Bronze	Silver	Gold
Data published and downloadable	✓	✓	✓
Trained models published and downloadable	✓	✓	✓
Source code published and downloadable	✓	✓	✓
Dependencies set up with one command		✓	✓
Key analysis details recorded		✓	✓
Random components made deterministic		✓	✓
Entire analysis reproducible with one command			✓

Fast memory formula:

```
Bronze = 3 artifacts
Silver = Bronze + 3 controls
Gold   = Silver + 1 complete command
```

Why reporting is not enough

Reporting hyperparameters, architecture, data splitting, and known limitations resembles a **nutrition label**: it summarizes what the authors know, but it does not expose the complete executable process. Without data, model, and code, outsiders cannot rerun the analysis, reconstruct omitted implementation choices, or discover unknown bias after publication.

Bronze = minimum artifact availability

1. Data

- Publish the **raw form** of every dataset when the work first appears as a preprint or publication.
- Prefer a specialist repository, such as GEO for gene expression or BioImage Archive for microscopy.
- If no specialist repository exists, the article's stated rule is **Zenodo up to and including 50 GB; Dryad above 50 GB**.
- For an existing external dataset, provide the information and code needed to download and preprocess it.

2. Trained models

- Publish the trained weights actually used to generate the reported results.
- Publishing every additional set of trained weights from a hyperparameter sweep is unnecessary if the reported result can be reproduced without it; those extra models may still have participated in tuning or model selection.
- Prefer a specialist model zoo; otherwise use an archival repository such as Zenodo.
- Models matter even when code exists: retraining costs time/compute, may be nondeterministic, and does not necessarily recover the model whose fairness, generalization, or learned artifacts must be examined.

3. Source code

Code is often a more exact methods description than prose. It must cover data processing, model training, tuning, testing, figure generation, and final-result generation. A computational paper without code deserves skepticism similar to a paper without a methods section.

Why GitHub plus Zenodo? GitHub supports active development, collaboration, reuse, and Issues; Zenodo supplies a persistent, citable, archived snapshot. GitHub alone is mutable and is not the archival record required by the article.

Silver = Bronze plus reproducible setup and control

Silver adds three requirements.

1. **One-command dependency setup.** Record dependency versions with tools such as Conda/Packrat or a suitable environment file. Guessing old versions is the paper's "package Battleship" problem.
2. **Key details documented.** Record execution order and instructions, OS version, wall-clock and CPU time, CPU/GPU models and counts, and CPU/GPU RAM. Order may be expressed in a README, numbered filenames, or an orchestration script. The paper requires OS, runtime, and resource details in **both the manuscript body and README**.

3. **Random components deterministic.** Seed pseudorandom generators and use deterministic framework modes where available.

Do not confuse the commands:

- **Silver:** dependencies are installed with one command.
- **Gold:** the entire analysis is reproduced with one command.

Containers help, but do not prove identity

Containers capture more of the software environment than a package list, but they do not fully isolate the computation from hardware. Different GPUs or drivers may yield different results, and some parallel operations remain nondeterministic. Containers can also preserve brittle old code. The article recommends that containerized code still work on a current version of at least one OS distribution.

Gold = Silver plus full automation

One command must automate the **whole path**: data download, preprocessing, training, tables, figure generation, figure annotation, and final outputs. Merely running one selected script is not Gold.

Snakemake and Nextflow are examples of workflow managers. A shell script may start the pipeline, but workflow systems also help restart after errors, parallelize tasks, and show progress.

Important caveats

Privacy-restricted data

- Put sensitive data in a journal-approved controlled-access repository; do not hide behind informal "available upon request."
- Models can leak training data, so researchers with privacy-constrained data should routinely use privacy-preserving training such as differential privacy; **Opacus** is the article's example library.
- If data cannot be public, the authors say the trained model **must be shared** to retain any hope of computational reproducibility. If neither data nor model is available, the code has little with which to operate.
- A model-only release helps but is **not full reproducibility**.

Compute-intensive analyses

If a full rerun is infeasible, publish intermediate outputs so reviewers can verify downstream stages. A workflow manager can track them. A lightweight demo - for example, a small web app or Colab notebook using a pretrained model - can support inspection. This increases **partial coverage** but is not a Gold-level full rerun.

Reusable software packages

Bronze/Silver/Gold primarily describe analyses. Reusable libraries need additional quality practices: unit tests, style consistency, clear documentation, and compatibility across major operating systems.

Incentives

- **Journals:** make Bronze the minimum publication standard, optionally require Silver/Gold for all or analysis-focused papers, and verify compliance through reviewers or a dedicated reproducibility reviewer.
- **Badging:** an independent body verifies the achieved standard and awards a visible badge for papers, CVs, or biosketches. The career/reputation signal encourages authors to invest in reproducibility. Exam nuance: the badge verifies a reproducibility level; do not infer unrelated properties without evidence.
- **Reproducibility collaborator:** a person outside the primary authors' group reproduces the work using only the published artifacts and documentation. This is the CRediT **validation** role, and the collaborator should not have designed or implemented the original analysis.

The incentive argument is temporal: irreproducible practices may save the original authors effort now but create more work later for both them and future researchers. Rewarding reproducibility prevents that cost shifting. Pure computation should also be easier to repeat than wet-lab work, which adds variability from reagents, cell lines, and environmental conditions.

4. Randomness, automation, and Docker

Unix stream mechanics

- `stdin`: the program's input stream.
- `stdout`: the program's normal output stream.
- `|`: sends one command's stdout to the next command's stdin.
- `< file`: takes stdin from a file.
- `> file`: writes stdout to a file, replacing its contents.
- `./script`: runs an executable file from the current directory.

Making randomness repeatable

A pseudorandom generator is deterministic once its initial **seed** is fixed. To make the experiment repeatable:

1. Set or accept a known seed once, before the first random draw; do not reseed for each sentence.
2. Record the seed.
3. Keep the input, code, generator/algorithm, dependency versions, and order/number of random calls fixed.
4. Run the generator and analysis through one script and compare every run with the first baseline.

Conceptual Python pattern:

```
rng = random.Random(seed)
if rng.random() < 0.5:
    # append ", please" before the punctuation
```

Reproducibility package checklist

Instruction meanings:

Instruction	Meaning
`FROM`	Select the base image
`LABEL`	Add image metadata
`RUN`	Execute a command while building an image layer
`WORKDIR`	Set the working directory for later instructions/runtime
`COPY`	Copy files from the build context into the image
`CMD`	Supply the default command and arguments when a container starts

`chmod +x` makes the runner executable. `sed -i 's/\r$//'` removes Windows carriage returns (CRLF line endings) so the shell script runs correctly on Linux.

Checklist to memorize:

- First one or two lines state the package's purpose.
- State copyright ownership.
- State the license with an SPDX identifier.
- Use a recent Ubuntu **LTS**, not `ubuntu:latest`.
- Use `LABEL org.opencontainers.image.authors=<email>` for the maintainer.
- Sort installed package names alphabetically.
- Remove dead code, including commented-out Docker instructions.
- Optional but stronger: pin exact dependency versions; for still stronger image identity, pin the base image by digest and preserve lock files.
- `docker run --rm image` removes the stopped container after execution; it does not remove the image.

Even a corrected Dockerfile does **not guarantee the exact same rebuilt image years later**: mutable tags, changing package repositories, unpinned versions, and disappearing artifacts can change the build. Separately, even the same image may not guarantee the exact same **runtime result** when CPU/GPU/driver differences or nondeterministic operations matter.

6. Exam traps and scenario classification

Frequent traps

- **Depth is not coverage:** sharing detailed information about only the analysis can be high depth for that part but still partial coverage overall.
- **Workflow is not provenance:** the planned workflow is prospective provenance; actual executions and changes require other provenance types.
- **Reporting is not reproducibility:** a perfect description without executable artifacts is insufficient.
- **The standards are cumulative:** missing a Bronze artifact means the work cannot be Silver or Gold.
- **One command has two meanings:** dependency installation is Silver; full analysis execution is Gold.
- **GitHub is not the archive:** pair active development with an archival snapshot.
- **A seed is necessary but not universally sufficient:** hardware/framework nondeterminism can remain.
- **A container is not a time machine:** it improves environment capture but does not guarantee bitwise identity or long-term availability.
- **Partial reproduction is still valuable:** special hardware or huge compute may make full coverage unrealistic.
- **A badge verifies the achieved reproducibility level; do not automatically treat it as evidence for unrelated scientific properties.**

7. Final 2-minute recall sheet

```
REPRODUCIBLE EXPERIMENT
  data (actual or derivation) + system + executable steps -> consistent result

DPC
  Depth       = how much material is available
  Portability = original / similar / different environment
  Coverage    = partial / full experiment

PRE PROVENANCE
  Prospective = intended recipe
  Retrospective = actual execution
  Evolution   = workflow versions and changes

BSG
  Bronze = data + trained model + code
  Silver = Bronze + one-command deps + key details + deterministic components
  Gold   = Silver + entire analysis in one command

PRACTICAL
  seed + record it + freeze inputs/code/environment/call order
  archive artifacts + manage versions + automate pipeline
  container helps, but hardware/drivers and mutable dependencies still matter

NUMBERS
  3 reproducibility dimensions
  3 provenance types
  7 Bronze/Silver/Gold requirements
  Zenodo <= 50 GB; Dryad > 50 GB
```

Reproducibility Engineering - Module 3 Exam Guide

1. The whole module in 90 seconds

1.1 A good hypothesis: PSUL + B-MEDS

PSUL describes its quality:

```
P  Precise
S  Specific
U  Unambiguous
L  Limited in scope, with limitations made clear
```

B-MEDS describes what an experimental hypothesis should identify:

```
B  Baseline or comparator
M  Metric
E  Expected direction or effect size
D  Data, workload, or population
S  Scope and operating conditions
```

```
Under [conditions], [method A] will [increase/decrease] [metric]
by [magnitude or threshold] relative to [baseline B] on [data/workload].
```

Example:

```
Under high contention on TPC-C, system A will reduce mean query
latency by at least 20% relative to baseline B.
```

Avoid undefined or unqualified uses of words such as `better`, `improves performance`, `most`, `realistic`, `easy`, and `fast`.

1.2 The research logic

```
topic/problem
  -> informal model or tentative hypothesis
  <-> specific research question
```

```
  -> observable prediction
  -> evidence/measurement
  -> argument linking evidence to the hypothesis
  -> support, refinement, or rejection
```

Evidence does not interpret itself. A result earns its meaning from the **connecting argument**.

1.3 Four forms of evidence: PMSE

Form	Core idea	Main risk
Proof	Formally establishes a proposition; a disproof/counterexample establishes falsity	A proof can contain an error; real-world claims may resist formalization
Model	Mathematical description of the hypothesis/system	Simplifying assumptions may make it unrealistic
Simulation	Simplified/partial implementation, often with artificial data	Controlled and flexible, but may not transfer to reality
Experiment	Full test using an implementation and real or highly realistic data	Strong practical evidence, but directly covers only tested cases

Do not call a program that merely evaluates a mathematical model an experiment on the real algorithm.

1.4 Occam's razor

If two hypotheses explain the observations **equally well**, prefer the simpler one. Simplicity is a tie-breaker, not a reason to ignore stronger evidence for a more complex hypothesis.

1.5 Experiment-section classifier

Part	Diagnostic question	Typical content
Setup	What was tested, against what, and how?	Data, workload, baselines, metrics, hardware, procedure
Result	What was directly observed?	Numbers, comparisons, table/figure findings
Discussion	What might it mean and why?	Interpretation, possible causes, implications, limitations

Signal words: `we evaluate` and `we measure` usually indicate **setup**; `X%` and `Figure shows` usually indicate **result**; `suggests` and `possibly because` usually indicate **discussion**.

1.6 Four comparison levels

Object	Level	Definition
Data/output	**Bitwise identity**	Same representation, byte for byte
Data/output	**Structural equivalence**	Same logical content despite order/format differences
Programs	**Functional equivalence**	Same outputs for the same inputs
Programs	**Behavioral equivalence**	Same observable behavior under a stated observation model, possibly including I/O, timing, and side effects

Same output does not imply same timing or side effects. Tests over a finite input set do not prove equivalence over every possible input.

2. Hypotheses, questions, predictions, and evidence

2.1 What a hypothesis does

A hypothesis is a clear, testable statement about how something behaves, interacts, or works. It converts a broad topic into a claim that determines:

- what should be observed if the claim is right;
- what evidence should be collected;
- what could count against the claim;
- what measurements and conditions are required.

A **research question** asks what the study will determine. A **hypothesis** predicts or asserts an answer. A **prediction** states an observable consequence. **Evidence** is what is collected. An **argument** explains why that evidence supports or challenges the hypothesis.

Example chain:

```
Topic: CPU-cache-aware data structures
Question: Can an array-based structure improve a sorting method?
Hypothesis: Better locality outweighs the extra computation for large inputs.
Prediction: As input size grows, the tree version's cache-miss rate rises
faster.
Evidence: Cache misses and execution time across controlled input sizes.
Argument: Higher miss rates explain the measured latency difference.
```

Ideas may begin with subjective intuition. The final paper must make an objective case with explicit claims, measurements, and reasoning.

2.2 Properties of a strong hypothesis

Must memorize: PSUL. A strong hypothesis is precise, specific, unambiguous, and limited in scope.

It should also be:

- **testable:** an observation, derivation, proof, or experiment can evaluate the claim;
- **falsifiable:** at least one possible observation could show it is wrong;
- **operationalized:** concepts such as quality or performance are mapped to defined measures;
- **predictive:** it says more than "the method worked on the data already seen";
- **interesting:** it teaches something beyond a single black-box result;
- **explicit about assumptions and exclusions:** readers can tell what is and is not claimed.

Limitations do not automatically weaken a result. A truthful boundary often makes a claim stronger because it prevents overgeneralization.

Compare:

```
Too broad:
Q-lists are superior to P-lists.

Scoped and testable:
For large in-memory data sets with a skewed access pattern, Q-lists
use less space and answer searches faster than P-lists.
```

The second claim can remain valid even if P-lists win under a different access pattern.

2.3 Diagnose and repair a vague hypothesis

Use five questions:

1. **Compared with what?** Name the baseline.
2. **What changes?** Name the dependent metric.
3. **By how much or in which direction?** State an effect or threshold.
4. **On what?** Name the data, workload, or population.
5. **Under which conditions?** Bound the scope.

Claim	Diagnosis	Repair
`Our system improves database performance.`	`improves` and `performance` are undefined; no baseline, metric, amount, workload, or conditions	Name latency/throughput, comparator, threshold, workload, and operating condition
`Our system reduces average query latency by at least 20% on TPC-C workloads under high contention.`	Intended good example: metric, direction, threshold, workload, and condition are explicit	explicitly add the baseline
`Our system improves performance in most realistic scenarios.`	`performance`, `most`, `realistic`, and `scenarios` are undefined	Define the population of scenarios and every success criterion

TPC-C is a standard benchmark for online transaction processing (**OLTP**) systems.

2.4 Falsifiability, prediction, and fair testing

A claim is scientifically useful only if some result could count against it. Repeated positive evidence can strengthen confidence, but it does not permanently prove a general empirical theory.

Important distinction:

```
Observation after looking at the data:
  "The algorithm worked on our data."

Predictive test:
  "We predicted it would work for data of class C, then tested that
   prediction on fresh data from C."
```

The second is stronger. If the hypothesis and experiment are repeatedly tuned on the same data, that data is development evidence, not an independent confirmation. Prefer a fresh or blind test.

Zobel notes that hypotheses may need refinement after initial testing. As stronger confirmatory practice, disclose the refinement and test the revised claim on fresh evidence. Do not present hindsight as an advance prediction.

2.5 Occam's razor and the equal-fit condition

Correct application:

```
H1 and H2 fit the observations equally well.
H1 is clearly simpler.
Therefore prefer H1.
```

Incorrect application:

```
H1 is simpler, but H2 explains substantially more evidence.
Choose H1 anyway.
```

Occam's razor does not override evidence.

2.6 Defending a hypothesis

A paper needs three distinct elements:

```
hypothesis + evidence + connecting argument
```

Evidence without a connecting argument is just an observation. The argument must explain why the measured outcome bears on the claim and why plausible alternatives are less convincing.

Defend the claim as a skeptical reader would:

- What phenomenon or property is being investigated, and why is it interesting?
- What is the research aim, and are the aim, hypothesis, and question convincingly connected?
- Do the stated claims accurately reflect the actual innovation?
- What result would disprove it?
- Which assumptions does it rely on, and are they plausible?
- Could another mechanism explain the same observation?
- Does the claim have improbable consequences?
- Does it cover the observations explained by the current theory?
- Is the contribution really new, or merely renamed existing work?
- Were counterexamples and extreme conditions actively sought?
- Are the data representative and sufficient?
- As a cross-module reproducibility extension: is the code/artifact quality good enough for inspection?
- Which objections cannot be rebutted and must instead be conceded as limitations?

The stronger your personal attachment to a hypothesis, the more aggressively you should try to break it. Do not twist results to preserve a favored idea.

2.7 Black-box and renaming traps

A black-box system beating one baseline on one dataset may be an observation without a useful general explanation. Ask whether the study reveals anything about:

- why the method works;
- when it stops working;
- whether the behavior predicts performance on new data;
- which properties of the data or method cause the result.

Changing terminology does not create novelty. Calling an ordinary cache a fashionable new kind of agent does not change its behavior. Inflated labels such as `intelligent`, `aware`, or `semantic` require operational definitions rather than rhetorical force.

---

3. Evidence and measurement

3.1 The four forms of evidence in depth

Zobel distinguishes proof, model, simulation, and experiment.

Proof

A proof formally establishes a proposition; a disproof or counterexample establishes that a proposition is false.

- Strong for mathematical claims.
- Can generalize beyond individual test cases.
- May still contain a mistake.
- May not capture human behavior, complex systems, hardware effects, or other real-world factors.
- Asymptotic analysis alone may omit constants, locality, or actual workloads.

Model
A model is a mathematical description of a system, hypothesis, or component.

- Makes assumptions explicit and supports reasoning or prediction.
- Can explore behavior beyond a few measured cases.
- Needs an argument that the model corresponds to the real object.
- Too many estimated parameters or simplifying assumptions can destroy realism.

Simulation
A simulation implements a simplified or partial version of the hypothesis, often with artificial data.

- Parameters can be controlled smoothly across a wide range.
- Extreme and failure conditions can be explored safely.
- Artificial data may make causal factors easier to isolate than real data.
- The result may be an artifact of unrealistic simplifications.
- Practical claims eventually need comparison with reality.

Experiment
An experiment tests an implemented proposal on real or highly realistic data.

- Offers strong practical evidence.
- Should be driven by predictions rather than merely used to find a favorable story.
- Should include severe tests and conditions likely to reveal failure.
- Directly demonstrates behavior only for the tested data and context.
- Models or simulations may help explain or generalize the observation.

3.2 Combine evidence, but do not confuse it

Different evidence types can support one another. For example, an experiment may confirm a trend predicted by a model, while a simulation explores boundary cases that are difficult to create physically.

However:

```
Implementing a mathematical performance model and printing its predicted
values tests the implementation/model calculations. It is not automatically
an experiment on the real algorithm or system.
```

Choose evidence for how convincingly it addresses the claim, not merely because it is easiest to produce.

3.3 Measurement: phenomenon versus proxy

The phenomenon is the underlying property of interest. A measurement is a context-dependent observation used as evidence for it.

Research aims are often qualitative:

- improve an interface;
- accelerate an algorithm;
- improve translations;
- make a network service better.

Evaluation usually needs quantitative proxies:

- task completion time;
- execution latency;
- textual-overlap score;
- packet delay.

The critical question is whether the proxy is logically connected to the aim. Text overlap, for example, can be high even when a translation is incoherent. Lower average packet delay may ignore video smoothness or service to remote users.

Measurement checklist:

1. What underlying phenomenon matters?
2. What exactly will be measured?
3. How will it be measured?
4. Is the measure objective, appropriate, and reasonable?
5. What simplifications or biases does it introduce?
6. Is one metric enough, or are several complementary metrics needed?
7. Are the data/workloads representative?
8. Could optimizing the score make the real outcome worse?

Beware of tuning a method repeatedly to one static benchmark. Performance on the benchmark can become the goal even when it no longer represents the broader problem.

3.4 Confirmation, falsification, and experimental failure

Concept	Correct interpretation
Falsification	A counterexample can refute a universal claim; therefore search seriously for counterevidence
Confirmation	Evidence increases confidence; it does not mean final proof of an empirical theory
Failed experiment	May challenge the hypothesis, but may also expose a faulty auxiliary assumption, insensitive instrument, bad implementation, or poor measurement

Do not automatically protect a hypothesis after every failure, but do not automatically discard it before checking the experiment. Theory and evidence refine one another iteratively.

3.5 Warning signs of weak science

- Vague or inflated terminology substitutes for a definition.
- A proposal has no serious evaluation or measurable consequence.
- The problem, solution, and success measure are all invented together without external justification.
- Only favorable results are reported.
- Implausible claims are accepted without checking quantitative limits.
- The work ignores relevant prior results or merely renames an existing idea.
- Results are internally inconsistent or not predictive.
- Authors argue for belief instead of seeking critical evidence.
- The system is always almost ready for demonstration but never testable.

Good reporting includes failures, limitations, uncertainty, and unresolved objections.

---

4. Presenting experiments: setup, result, discussion

4.1 What belongs in each part

Setup

- research question/hypothesis;
- datasets and collection method;
- workload and input selection;
- baselines/comparators;
- independent and dependent variables;
- metrics;
- hardware/software/environment;
- procedure, repetitions, seeds, and controls.

Results

- observed measurements;
- direct comparisons;
- tables and figures;
- uncertainty/variability;
- positive, null, and negative outcomes.

Discussion

- interpretation and possible mechanisms;
- implications for the hypothesis;
- alternative explanations;
- limitations and threats to validity;
- why the result matters;
- what should be tested next.

Do not hide an inference inside a results sentence as if it were an observation. Also do not leave a figure unexplained: state what pattern it shows and why that pattern matters.

5. Levels of equivalence

5.1 Definitions and boundaries

Level	Applies mainly to	Required sameness	What may differ
Bitwise identity	Files/data/output	Exact byte representation	Nothing at the byte level
Structural equivalence	Files/data/output	Logical content/structure	Ordering or formatting, where those are semantically irrelevant
Functional equivalence	Programs/functions	Output for each input in the stated domain	Language, implementation, speed, I/O pattern, side effects
Behavioral equivalence	Programs/systems	All behavior visible under the chosen observation model	Only behavior excluded by that model

Behavioral equivalence is meaningful only after the observation model is stated. If the model observes outputs, timing, I/O, and side effects, it demands more than functional equivalence.

6. Runtime evidence and fair comparison

6.3 Better runtime experiment

1. Define the target statistic and hypothesis before running the test.
2. Warm up runtimes/caches/JITs where appropriate.
3. Use many controlled, paired repetitions on the same machine and inputs.
4. Randomize or interleave A/B order to reduce drift bias.
5. Record the environment and background load.
6. Report the distribution, mean/median, variability such as SD/IQR, and confidence intervals.
7. Investigate unusually high observations rather than cherry-picking them away.
8. Use a suitable paired statistical test if inferential evidence is required.

---

10. Common exam traps

- A hypothesis is not strong merely because it sounds confident.
- `Performance`, `better`, `most`, and `realistic` are not metrics.
- A percentage threshold still needs a named baseline.
- Limitations can strengthen a claim by defining its valid scope.
- Positive evidence confirms in the weak scientific sense; it does not prove a general empirical theory forever.
- A failed experiment may reveal a false hypothesis or a bad auxiliary assumption/instrument/measurement.
- Occam's razor applies only when evidential fit is equal.
- Evidence and an argument connecting it to the hypothesis are different things.
- A model implemented as code is not automatically an experiment on a real system.
- Optimizing one proxy can move research away from its qualitative goal.
- `Figure shows` normally reports a result; `suggests` normally begins interpretation.
- Reporting only positive outcomes is not acceptable.
- Same logical content does not require identical bytes.
- Same outputs do not prove identical timing, I/O, or side effects.
- Tests show equivalence only over the tested set unless a proof/general argument extends it.

---

11. Final two-minute recall sheet

```
GOOD HYPOTHESIS
  PSUL = precise, specific, unambiguous, limited in scope
  B–MEDS = baseline, metric, effect, data/workload, scope/conditions
  must be testable, falsifiable, predictive, and explicit about assumptions

RESEARCH LOGIC
  tentative hypothesis <-> question -> prediction -> evidence -> argument
  positive evidence strengthens; it does not permanently prove
  revised-after-data hypothesis needs a fresh test
  equal evidential fit + simpler explanation = Occam's razor

EVIDENCE
  PMSE = proof, model, simulation, experiment
  model: mathematical + assumptions
  simulation: controlled/artificial + realism risk
  experiment: real/highly realistic + only tested cases directly covered

PAPER STRUCTURE
  setup = what/how
  result = observed
  discussion = meaning/why/limitations

EQUIVALENCE
  bitwise = same bytes
  structural = same content, representation may differ
```

```
  functional = same outputs for same inputs
  behavioral = same observations under stated model
```

Reproducibility Engineering - Module 4 Exam Guide

1. The whole module in one page

Four memory blocks

```
GIT STATE
  working tree --git add--> index --git commit--> repository

GOOD CHANGE
  one logical purpose + clear diff + meaningful message
  + responsibility trailers where project policy requires them

INTEGRATION
  merge preserves topology; rebase replays commits and rewrites identities

XPATH E-D-I-G
  eq = one value; deep-equal = structure; is = node identity; = = any matching
pair
```

The highest-yield tables

Command	Comparison performed
`git diff`	Working tree versus index: unstaged tracked changes
`git diff --staged`	Index versus `HEAD`: what the next commit would contain
`git diff HEAD`	Working tree plus index versus `HEAD`: all tracked uncommitted changes
`git diff A B`	Snapshot/tree at `A` versus snapshot/tree at `B`
`git show C`	Commit `C`, its metadata, and normally its patch
`git log -p`	Commit history plus the patch introduced by each commit

XPath form	Operands	Meaning	Key trap
`A eq B`	Singleton atomic values	Exact value comparison	More than one item causes `XPTY0004`
`deep-equal(A,B)`	Any two sequences	Same ordered recursive structure/content	Same identity is unnecessary; whitespace can matter
`A is B`	Single nodes	Same node identity in the same XDM tree	Identical-looking nodes are still different nodes
`A = B`	Sequences of any length	True if any cross-pair has equal values	It is existential, not whole-sequence equality

The central lesson

Version control records the exact change, its base, its rationale, and a trail of responsibility. This turns development history into research provenance.

Output equality is always relative to a chosen criterion. Two PDFs may contain the same scientific result but have different bytes; two XML nodes may contain the same text but differ in attributes or identity. Always state the **equivalence relation appropriate to the claim**.

2. Git's mental model

Working tree, index, and repository

1. **Working tree:** files currently checked out and edited on disk.
2. **Index / staging area:** the exact candidate snapshot for the next commit.
3. **Repository:** committed snapshots and their history under `.git`.

```
edit files              select content                  record snapshot
working tree  -------------------------> index ---------------------------->
repository
                              git add                          git commit
```

The index is why one commit need not contain every edit in the working tree.

```
git status
git diff
git add -p
git diff --staged
git commit
```

`git add -p` interactively stages selected hunks, helping separate logical changes.

Commits, branches, and `HEAD`

A commit records:

- a tree representing the project snapshot;
- parent commit(s);
- author and committer identities/timestamps;
- a commit message.

A normal commit has one parent, the root has none, and a merge commit normally has two or more. Git derives a commit's patch by comparing its tree with its parent. A commit is fundamentally a snapshot plus history metadata, not merely a stored diff.

A commit ID changes if relevant commit content changes, including its tree, parent, metadata, or message. Rewriting one commit therefore gives it a new ID and normally changes all descendant IDs.

A **branch** is a movable reference to a commit. `HEAD` normally points to the checked-out branch.

```
A <- B <- C        main -> C
                   HEAD -> main
```

Remote operations

Command	Exam meaning
`git init`	Create a repository in the current directory
`git clone URL`	Copy a repository and its reachable history; configure a remote
`git fetch`	Download remote objects/refs without integrating them
`git pull`	Fetch and then integrate by configured merge/rebase/fast-forward behavior
`git push`	Update remote refs using local commits
`git remote -v`	Show remote names and URLs

Trap: `git pull` is not simply "download files"; it also integrates.

Essential command map

Goal	Command	Key effect/trap
Set identity	`git config --global user.name "Name"` and `git config --global user.email "mail"`	This is recorded as metadata; it is not authentication
Inspect	`git status`, `git log --oneline --graph --decorate --all`	Read state/history before changing it
Create/switch branch	`git switch -c feature`, `git switch main`	Older material may use `git checkout -b feature`
Mark a release	`git tag -a v1.0 -m "Release v1.0"`	Record/archive the tagged commit; a tag name alone can still be moved
Unstage	`git restore --staged FILE`	Keeps the working-tree edit
Discard unstaged edit	`git restore FILE`	Destructive to that uncommitted edit; inspect first
Correct last private commit	`git commit --amend`	Replaces it with a new commit ID
Undo a shared commit	`git revert COMMIT`	Adds a new inverse commit; preserves published history
Move branch/history	`git reset` variants	`--hard` can discard index/working-tree work; do not use casually

For exam answers, prefer `revert` for already shared history and reserve amend/rebase/reset for private or explicitly coordinated rewriting.

3. Reading `git diff` and `git log -p`

Which diff?

```
HEAD snapshot ---- staged changes ---- unstaged changes
                  index                working tree
```

- `git diff`: working tree versus index, so unstaged tracked changes.
- `git diff --cached` / `--staged`: index versus `HEAD`, so staged changes.
- `git diff HEAD`: working tree versus `HEAD`, so staged plus unstaged tracked changes.
- Untracked files do not appear as normal content changes until Git is told about them.

Revision comparisons:

```
git diff A B
git diff A..B
git diff A...B
```

- `A B` and `A..B` compare the endpoint trees.
- `A...B` compares the merge base of `A` and `B` with `B`; it asks what `B` introduced since divergence and is asymmetric.

Unified diff anatomy

```
diff --git a/sec/hash.c b/sec/hash.c
index 1234567..89abcde 100644
--- a/sec/hash.c
+++ b/sec/hash.c
@@ -1,7 +1,7 @@
 doSomething();
-hash = getHash(val);
+hash = getSaltedHash(val, salt());
```

Part	Meaning
`diff --git a/... b/...`	Old and new paths in Git diff notation
`index old..new 100644`	Abbreviated old/new blob IDs and file mode
`--- a/...`	Old file
`+++ b/...`	New file
`@@ -1,7 +1,7 @@`	Hunk location: old range then new range
leading space	Unchanged context
`-`	Line removed from old version
`+`	Line added in new version

`100644` is a regular non-executable file. For creation/deletion, one side may be `/dev/null`. The `+` in `+++ b/file` is a header marker, not a code addition.

History direction

Default `git log` is newest first. To reconstruct current content from `git log -p`, begin at the oldest relevant commit at the bottom and apply each patch forward. A direct check in a real repository is:

```
git show HEAD:path/to/file
```

4. A high-quality commit

Atomicity

An **atomic commit** represents one complete, coherent logical change. Ideally it:

- has one purpose;
- excludes unrelated work in progress;
- builds and passes relevant tests;
- is independently reviewable and, where reasonable, revertible;
- has a message that explains intent.

Atomic does not mean "one file" or "very few lines." A coherent change can require code, tests, docs, and configuration together.

Message structure

```
Use salted hashes

Function getHash() stores password hashes without a salt. Add the salting
step required by the referenced security design.

Signed-off-by: Jane Doe <jane@doe.com>
Reviewed-by: Jean Doe <jean@doe.com>
Tested-by: Judy Doe <judy@doe.com>
```

1. Concise imperative subject.
2. Blank line.
3. Body explaining motivation, context, design, consequences, or references. Explain **why**; the diff shows the technical **what**.
4. Structured responsibility trailers.

Author versus committer

- **Author:** originally created the change.
- **Committer:** integrated/recorded that change in this repository history.

They may differ when a maintainer applies a patch or a commit is rebased/cherry-picked.

5. DCO and responsibility trailers

Developer Certificate of Origin

The DCO is a contributor certification. In exam language, a signatory states that one route applies:

- they created the contribution and may submit it under the indicated open-source license;
- it derives from appropriately licensed work and they may submit the modification;
- another person supplied it after making the same certification and it was passed on unchanged;
- they understand the contribution and identifying sign-off are public, retained, and redistributable under the project/license.

```
git commit -s
```

adds:

```
Signed-off-by: Name <email>
```

What sign-off is not

- not automatically a cryptographic signature;
- not the same as `git commit -S`, which cryptographically signs;
- not merely a claim of authorship;
- not proof that code is correct, reviewed, or tested;
- not the same instrument as a Contributor License Agreement.

It is a legal/provenance certification about the contribution's allowed origin and route of submission. Multiple sign-offs can document a chain.

Trailer	Course-figure meaning
`Signed-off-by`	DCO/provenance certification
`Reviewed-by`	Named person reports review and an acceptable result under project policy
`Tested-by`	Named person reports testing

Git does not magically verify arbitrary trailers. Never add another person's trailer without the required action/permission.

7. Merge versus rebase

Assume branches diverged:

```
          F1 <- F2   feature
         /
A <- B <- C <- D     main
```

Merge

On `main`:

```
git merge feature
```

With divergence, Git performs a three-way merge using both tips and their common ancestor, then normally makes a two-parent commit:

```
          F1 <- F2
         /        \
A <- B <- C <- D ---- M   main
```

Properties:

- preserves existing commits and branch topology;
- records when lines of development joined;
- normally preserves existing commit IDs;
- may add a non-linear merge commit.

If the current tip is an ancestor of the other tip, Git can **fast-forward** by moving the branch reference. No two-parent merge commit is needed.

Rebase

On `feature`:

```
git rebase main
```

Git finds the common base and replays feature changes on top of `main`:

```
A <- B <- C <- D <- F1' <- F2'   feature
```

Properties:

- creates new commits with new IDs;
- gives a linear presentation;
- makes the work appear based on the new tip;
- does not preserve the original feature topology;
- may require conflict resolution for several replayed commits.

Conflicts

```
# resolve marked files, then:
git add <resolved-files>
git rebase --continue

# abandon:
git rebase --abort
```

For a merge, resolve, stage, and commit; use `git merge --abort` to abandon when available.

Golden rule

Do not casually rebase commits collaborators already use. Rebase abandons old identities and replaces them. Safe default:

- clean/rebase your own private branch before sharing;
- merge shared/public history;
- rewrite shared history only with explicit coordination.

8. Interactive rebase and presentation history

```
git rebase -i --root
# or only the latest five commits
git rebase -i HEAD~5
```

Action	Effect
`pick`	Keep commit
`reword`	Keep change, edit message
`edit`	Stop so content/metadata can be amended
`squash`	Combine into previous commit and combine/edit messages
`fixup`	Combine into previous commit and normally discard this message
`drop`	Remove commit
reorder lines	Replay commits in a new order

New fixups can be prepared with:

```
git commit --fixup=<target-commit>
git rebase -i --autosquash <base>
```

Development history can preserve experiments and dead ends, while presentation history optimizes review and reuse. A strong answer acknowledges the trade-off: keep an archival/raw reference if needed, publish a clean logical series, and never claim rewritten history is untouched chronology.

9. Creating and applying patches

Plain diff patch

Create:

```
git diff BASE..TIP > change.patch
```

Check and apply:

```
git apply --check change.patch
git apply change.patch
git diff
git add <files>
git commit
```

Key properties:

- `git apply` changes the working tree and normally creates no commit;
- a plain diff does not preserve original author/date/message as a new commit;
- `--check` tests applicability without changing files;
- `--index` applies to the index and working tree;
- `--3way` can help when blob information/objects are available;
- `-R` applies the patch in reverse;
- use `git diff --binary` when a raw patch must include suitable binary changes.

Commit-aware series

```
git format-patch BASE..TIP
git format-patch -1 COMMIT
git am --3way 0001-description.patch
```

If application conflicts:

```
# resolve and stage
git am --continue
# or
git am --abort
```

`format-patch` normally emits one mail-formatted patch per non-merge commit, including author identity/date, message, and technical diff. `git am` creates new commits that preserve those author/message fields and boundaries, but their parent, committer metadata/date, and commit IDs can differ.

```
git diff / git apply      -> file changes; receiver stages/commits
git format-patch / git am -> commit-aware patches become commits
```

For a reproducible patch stack, record exact upstream URL/base commit, patch order and hashes, tools/commands, submodules/assets, license/provenance, tests, and expected final tree/commit.

10. Snapshot versus clone plus patches versus fork

Strategy	Advantages	Risks/disadvantages
Final source snapshot	Simple, offline/self-contained if complete, easy to checksum/archive	Often loses Git history, exact upstream base, responsibility trail, and separation of changes; harder to review/update; license/notices still required
Exact upstream plus patch stack	Compact, separates upstream/research work, reviewable/replayable; `format-patch` can retain authors/messages	Must pin/preserve base and order; network/upstream can disappear; patches can conflict; raw diffs may lose metadata/binary detail
Hosted fork	Full reachable history, exact commits, natural branches/tags/PRs, easy collaboration/upstream syncing	Platform/account/network dependent; refs can move or be force-pushed; fork can disappear/diverge; not archival by itself

The word **latest** is a warning. Reproduction needs an exact immutable revision.

12. Structural equivalence and XPath

Equality is a design decision

Possible output relations include byte identity, scalar equality, normalized-value equality, recursive structural equality, object identity, or scientific agreement within tolerance. The right relation depends on what the experiment promises.

`eq`: singleton value comparison

`eq` atomizes nodes and compares one atomic value with one value.

- empty operand can yield an empty sequence;
- more than one atomized item causes dynamic error `XPTY0004`;
- untyped XML values are cast to strings for value comparison;
- spaces in `" 42 "` remain significant unless explicitly normalized.

Use it for one measurement against one expected scalar.

`deep-equal()`: recursive structure/content

For these examples, it requires sequences with the same length/order and selected nodes with matching kinds, names, attributes, text, and recursive children.

- node identity need not match;
- parents do not matter if only child nodes are selected;
- attribute order is irrelevant, but attribute names/values matter;
- child order and significant text/whitespace matter.

Use it to regression-test XML subtrees or ordered structured results.

`is`: node identity

`A is B` is true only when both expressions identify the exact same XDM node. Matching appearance is insufficient. Each operand must be a single node or empty; a longer sequence is an error.

Use it to test whether two paths/variables alias the same parsed node.

`=`: general/existential comparison

General comparison accepts sequences and is true when **at least one cross-pair** has equal values:

```
(42) = (40, 42)  -> true
```

It does not require equal lengths/order or all values equal. For sequences, `!=` is not always the logical inverse of `=`: one pair can be equal while another pair is unequal.

Use it for membership/overlap: "does any observed value match any allowed value?"

13. Freezing LaTeX dependencies

LaTeX is modular. A TeX Live bundle is convenient but large. TinyTeX is lighter, but downloading missing packages on demand makes the effective environment drift. Install the complete package set during the image build.

Pinning hierarchy

1. `ubuntu:latest` and unversioned packages: freely drift.
2. `ubuntu:24.04`: better release selection, but still a movable tag.
3. Exact package versions: better, but repositories can later remove them.
4. Image digest plus locks/hashes and repository snapshots: much stronger.
5. Preserve/archive the built image and sources: also protects against disappearance.

Pinning selection and guaranteeing future buildability are different problems.

14. Automated reporting and repeatability

End-to-end Gold pipeline

```
input + code + frozen environment
              |
              v
        run experiments
              |
              v
      result data + chart
              |
              v
       LaTeX paper -> PDF
```

This reaches Module 2's Gold idea because one dispatcher covers experiments, chart, and final paper.

Hashes and repeatability

- equal hashes: byte streams are identical for this test;
- unequal hashes: bytes differ;
- unequal hashes alone do not prove data, chart values, conclusions, or visible pages differ.

Non-scientific byte differences can come from timestamps, PDF IDs/metadata, generated-chart metadata, object ordering/compression, locale, timezone, paths, or tool versions. Scientific differences can come from randomness, changed inputs, floating-point/environment variation, or nondeterministic ordering.

MD5 is acceptable here as a quick equality test; SHA-256 is preferable for serious integrity/security use.

16. Common exam traps

1. `git diff` excludes staged changes; use `git diff --staged`.
2. A commit is a snapshot with parents/metadata; a patch is change relative to a base.
3. Default `git log` is newest first.
4. Context lines begin with a space; `---`/`+++` are headers.
5. Author created; committer integrated.
6. `Signed-off-by` is DCO certification, not GPG signing or testing.
7. Atomic means one logical purpose, not one file.
8. A hotfix must start from clean production, not a WIP feature snapshot.
9. Fast-forward moves a reference; it need not create a merge commit.
10. Merge preserves topology/IDs; rebase replaces replayed commit IDs.
11. Do not rewrite shared history without coordination.
12. `git apply` normally creates no commit; `git am` applies mail patches as commits.
13. A live fork or "latest" snapshot is not an immutable archive.
14. `eq` with multiple items is an error, not `false`.
15. `=` asks whether any pair matches, not whether sequences are identical.
16. `is` asks identity, not content equality.
17. `deep-equal(B/time,C/time)` ignores their unselected parent IDs.
18. XML text whitespace can be data.
19. TinyTeX auto-download allows dependency drift.
20. Run LaTeX twice when references must resolve.
21. Different PDF hashes prove different bytes, not automatically different science.

18. Final two-minute recall sheet

```
GIT STATES
  WT --add--> index --commit--> repository
  diff = WT:index
  diff --staged = index:HEAD
  diff HEAD = WT:HEAD (tracked)

DIFF
  --- old, +++ new
  @@ -old +new @@ hunk
  space=context, -=removed, +=added

COMMIT
  one coherent logical change
  author made it; committer integrated it
  subject + body + trailers
  -s DCO sign-off; -S cryptographic signature

INTEGRATION
  merge preserves graph/IDs; fast-forward may only move a ref
  rebase replays -> new IDs; do not rewrite shared history casually
  rebase -i: pick/reword/edit/squash/fixup/drop/reorder

PATCHES
  git diff -> git apply -> stage/commit yourself
  format-patch -> am -> preserve author/date/message + boundaries
                        but new committer/parents/IDs are possible

PACKAGE
  never say "latest"
  upstream URL + license + exact base + exact final revision
  archive repo/patches + tested reconstruction

XPATH E-D-I-G
  eq = singleton value; >1 item is XPTY0004
  deep-equal = ordered recursive structure/content
  is = exact same node
  = = any equal cross-pair


REPORTING
  explicit TinyTeX packages; pdflatex twice
  experiment -> data/chart -> paper, one dispatcher
  unequal PDF hashes = unequal bytes, not automatically unequal science
```

Reproducibility Engineering - Module 5 Exam Guide

1. The whole module in one page

Six rules to memorize

1. **Definition:** A build is reproducible when the same source code, declared build environment, and build instructions let any party recreate **bit-for-bit identical specified artifacts**. Identity is normally checked with a cryptographic hash.
2. **Build pipeline:** `source -> preprocess -> compile -> assemble -> object files -> link -> executable`.
3. **Make rule:** `target: prerequisites`, followed by a **literal tab** and the recipe. A target is rebuilt if it is missing or a prerequisite is newer.
4. **Determinism rule:** ensure stable inputs, stable outputs, and capture as little uncontrolled environment state as possible.
5. **Preprocessor warning:** `__DATE__` and `__TIME__` leak build time; `__FILE__` can leak a build path. These can make otherwise unchanged binaries differ.
6. **Assertion warning:** never put a required side effect inside `assert(...)`; `-DNDEBUG` removes the assertion **and does not evaluate its argument**.

Mnemonic:

```
SBI + SOS

Same Source + Build instructions + declared environment -> Identical bits
Stable inputs + Stable outputs + Small environmental capture
```

The C pipeline and the commands

```
file.c + included headers
          |
          | preprocessor: #include, #define, #if/#ifdef
          v
expanded translation unit
          |
          | compiler
          v
assembly
          |
          | assembler
          v
file.o object code ----+
other.o object code ---+-- linker + libraries --> executable
```

```
gcc -E file.c                  # preprocess only
gcc -S file.c                  # stop after producing assembly
gcc -c file.c -o file.o        # produce object code; do not link
gcc main.o helper.o -o tool    # link object files
gcc main.c helper.c -o tool    # let gcc drive every stage
./tool                         # run on Unix-like systems
```

The stage-control flags are documented by GCC's overall invocation options.

Strictly, GCC is a **compiler driver**: it coordinates preprocessing, compilation, assembly, and linking. In casual speech, the whole operation is often called compilation.

Make in a few lines

```
tool: main.o helper.o
  $(CC) -o $@ $^

main.o: main.c main.h helper.h
  $(CC) -c main.c

helper.o: helper.c helper.h
  $(CC) -c helper.c
```

- `tool`, `main.o`, and `helper.o` are targets. Because `tool` is first, it is the default goal for plain `make`.
- The names after `:` are prerequisites/dependencies.
- The indented shell line is a recipe; the indentation must be a real tab.
- `$@` = current target; `$^` = all normal prerequisites in order, with duplicates removed; `$<` = first normal prerequisite. Order-only prerequisites are available through `$|`.

The five most likely sources of different binaries

Variation	Typical leak	Core fix
Time	`__DATE__`, `__TIME__`, archive timestamps	Remove it or use a deterministic reference such as `SOURCE_DATE_EPOCH` when supported.
Input order	directory traversal, locale-sensitive wildcard sorting	List inputs explicitly or sort them in a locale-independent way.
Build path	`__FILE__`, debug info, generated paths	Use a fixed path or compiler path-remapping support.
Environment/toolchain	compiler/dependency versions, flags, locale, timezone	Declare, pin, minimize, and recreate the build environment.
Random/undefined state	random seeds, uninitialized memory, races	Remove randomness from outputs, seed it when legitimate, initialize values, and avoid race-dependent output.

2. Reproducible builds

Exact definition and necessary ingredients

The Reproducible Builds project defines a build as reproducible if, given the same:

- **source code**, normally an exact VCS revision or source archive;
- **build environment**, including relevant tools, versions, flags, dependencies, locale, and environment variables; and
- **build instructions**;

any party can recreate **bit-by-bit identical copies of every specified primary artifact**, such as an executable, package, or filesystem image. Build logs are usually ancillary rather than the artifact being compared.

This is stronger and more specific than merely saying "the program works again." The central claim is an independently verifiable path:

```
declared source + declared environment + declared build procedure
                                |
                                v
                  identical artifact bytes
                                |
                                v
                  identical cryptographic hash
```

Deterministic is necessary but not sufficient

- A **deterministic build system** gives the same output whenever its defined inputs and environment are the same.
- A **reproducible build** adds independent recreation: another party must be able to reconstruct a sufficiently matching environment, run the build, and verify identical artifacts.
- A build that is deterministic only on one undocumented laptop is not independently reproducible.
- Two identical hashes show byte identity of the compared artifacts; they do not, by themselves, prove that the source is safe or correct.

The official three-step model is:

1. Make the build process deterministic.
2. Record or define the tools and relevant environment.
3. Give others a way to recreate that environment, rebuild, and compare.

Why reproducible builds matter

The main security purpose is to detect a compromised build path: an independent clean build can reveal that a distributed binary contains changes absent from the published source. Other benefits include:

- finding race, timing, locale, encoding, and dependency problems;
- proving that a build-system change did not alter the produced binary;
- recreating matching debug symbols for production artifacts;
- making binary differences smaller and more meaningful;
- improving dependency awareness and software-supply-chain auditing;
- avoiding unnecessary downstream rebuilds when output is unchanged.

Exam nuance: reproducibility makes tampering **detectable through comparison**. It does not prevent every attack, guarantee the correctness of the source, or solve the "trusting trust" problem by itself.

The deterministic-build checklist

The shortest official summary is:

```
stable inputs + stable outputs + minimal environment capture
```

Use this expanded checklist in a design question:

Problem source	Why output can change	What to do
Current time/date	A different value is embedded on every build	Omit it; otherwise derive a reference time from source history and use `SOURCE_DATE_EPOCH` where supported.
Timezone	The same timestamp formats differently	Normalize timezone, commonly UTC.
Locale	Filename collation, messages, decimal formats, and generated text vary	Fix the locale or use a locale-independent operation.
Filesystem enumeration	Directory order is not a reliable API contract	Explicitly list or deterministically sort inputs.
Output order	Hash maps, parallel tasks, or traversal order change serialization	Sort/canonicalize output before writing it.
Randomness	Random identifiers, seeds, or layout change bytes	Eliminate it from the artifact or use a controlled deterministic seed.
Uninitialized memory	Arbitrary memory contents leak into output	Initialize every output-affecting value.
Absolute build path	Paths enter `__FILE__`, debug data, archives, or generated files	Use a fixed build root or path-prefix mapping.
User/host information	Username, hostname, home path, or environment enters output	Do not embed it; sanitize relevant variables.
Tool/dependency drift	Different compiler/library versions legitimately emit different bytes	Pin and make the toolchain/environment recreatable.
Network downloads	Mutable or unavailable resources change/disappear	Vendor, archive, checksum, or otherwise lock every dependency.
Archive metadata	Member order, timestamps, owner, group, or permissions differ	Normalize metadata and order.
Concurrency	Races change ordering or content	Make output independent of scheduling or serialize the affected step.

`SOURCE_DATE_EPOCH`

`SOURCE_DATE_EPOCH` is a distribution-independent convention for passing one deterministic timestamp through the build environment. Its value is an ASCII integer containing seconds since `1970-01-01 00:00:00 UTC`, usually derived from the latest source modification. Supporting tools use it instead of the current time for embedded timestamps.

```
export SOURCE_DATE_EPOCH=1710000000
make
```

Important limits:

- It works only when the involved tool honors it.
- It fixes time-dependent output, not locale, path, ordering, random, tool-version, or all archive-metadata problems.
- A fixed arbitrary timestamp can make output deterministic, but a source-derived value preserves useful provenance.

How to verify a reproducible build

A sound verification procedure is:

1. Select the exact source revision.
2. Recreate the declared build environment and dependencies.
3. Run the documented build without reusing undeclared local products.
4. Hash every specified artifact, for example with `sha256sum`.
5. If hashes differ, inspect the artifact difference and vary one environmental dimension at a time: time, timezone, locale, path, file order, user, toolchain, and parallelism.

```
sha256sum build-a/tool build-b/tool
cmp build-a/tool build-b/tool
```

`cmp` or hashes tell you **that** bytes differ. A structural comparison tool such as `diffoscope` can help explain **where and why** they differ.

---

3. C essentials used by this module

Anatomy of a small C program

```
#include <stdio.h>

int main()
{
    puts("C rocks!");
    return 0;
}
```

- A `.c` file contains human-readable source.
- `#include <stdio.h>` asks the preprocessor to include declarations for standard input/output functions.
- Execution begins at `main` in a hosted C program.
- `main` returns an `int`; `0` conventionally signals successful termination.
- `gcc source.c -o program` builds an executable; `./program` runs it on Unix-like systems.
- `gcc source.c -o program && ./program` runs only if the build succeeded.
- `echo $?` on a Unix-like shell displays the previous command's exit status.

Data types and conversions - lower priority

Type	Course-level meaning	Portability warning
`char`	stores a small integer/character code	Signedness is implementation-dependent unless written `signed char` or `unsigned char`.
`short`	integer type no wider than `int`	Do not assume an exact byte count.
`int`	usual whole-number type, at least 16 bits	Range varies by implementation.
`long`	integer type at least as wide as `int`, at least 32 bits	It is not necessarily wider than `int`.
`float`	basic floating-point type	Precision and representation are finite.
`double`	at least as precise as `float`	Floating-point calculations are not exact real arithmetic.

Use the implementation rather than guessing its sizes:

```
#include <limits.h>  /* INT_MIN, INT_MAX, ... */
#include <float.h>   /* FLT_MIN, FLT_MAX, ... */

printf("%zu\n", sizeof(int));
```

The standard conversion for a `sizeof` result (`size_t`) is `%zu`. Data-model differences are another reason to define the build environment rather than assuming that all targets have identical C types.

Key conversion rules:

```
int x = 7;
int y = 2;
float a = x / y;          /* integer division first: 3, then 3.0 */
float b = (float)x / y;   /* floating division: 3.5 */
```

- When both operands are integers, `/` performs integer division and discards the fractional part.
- Casting either operand to `float` makes the other participate in floating-point arithmetic.
- Converting `58.65f` to `int` truncates toward zero to `58`.
- Conversion to an unsigned integer is defined modulo one more than that type's maximum. An out-of-range conversion to a signed integer is implementation-defined or can raise an implementation-defined signal, so do not rely on the book's particular wraparound example.
- Useful warning flags include `-Wall -Wextra -Wconversion -pedantic`.

Declaration, prototype, and definition

```
float add_with_tax(float price);       /* declaration/prototype */

float add_with_tax(float price)        /* definition */
{
    return price * 1.06f;
}
```

- A **declaration** tells the compiler the name and type of a function before it is called.
- A **prototype** is a declaration that specifies parameter types, enabling argument checking.
- A **definition** supplies the body and creates the function.
- In modern C, do not call a function without a prior declaration. The legacy behavior described in the book assumed an undeclared function returned `int`; that behavior is invalid in C99 and later.
- Through C17, `float f();` is a declaration with an **unspecified** parameter list, not a prototype through C17. Write `float f(void);` for a no-argument prototype.

Headers and multiple source files

Use a header for the interface and a `.c` file for its implementation:

```
/* encrypt.h */
void encrypt(char *message);
```

```
/* encrypt.c */
#include "encrypt.h"

void encrypt(char *message)
{
    /* implementation */
}
```

```
/* main.c */
#include "encrypt.h"

int main(void)
{
    /* call encrypt(...) */
    return 0;
}
```

```
gcc main.c encrypt.c -o message_hider
```

Important distinctions:

- `#include <stdio.h>` normally searches the compiler's system include path.
- `#include "encrypt.h"` first denotes a local/project header according to the compiler's search rules.
- `#include` conceptually inserts the header contents during preprocessing; it does **not** separately compile the header.
- Put declarations in `.h`; put each ordinary definition in one `.c` file.

- Include the same header in the implementation and every caller so incompatible types are diagnosed.
- To share a variable deliberately, a header can contain an `extern` declaration while exactly one source file contains its definition. Prefer functions and limited state over global variables when possible.

Why object files and incremental builds exist

This command repeats preprocessing, compilation, and assembly for every source:

```
gcc main.c a.c b.c c.c -o tool
```

For a large program, preserve unchanged object files:

```
gcc -c main.c
gcc -c a.c
gcc -c b.c
gcc -c c.c
gcc main.o a.o b.o c.o -o tool
```

After only `b.c` changes:

```
gcc -c b.c
gcc main.o a.o b.o c.o -o tool
```

Only `b.o` must be regenerated, but the executable must be relinked. Make automates this dependency decision.

4. Makefiles and dependency reasoning

Vocabulary and syntax

```
target: prerequisite1 prerequisite2
  recipe command
```

Term	Meaning
Target	The file or named action the rule is meant to create/update.
Prerequisite/dependency	An input that must be current before the target.
Recipe	Shell command(s) used to create the target.
Rule	Target, prerequisites, and optional recipe together.

A recipe line starts with a **literal tab**, not spaces. This historical syntax is exam bait.

Make's update algorithm

When asked to build a target, Make reasons recursively:

1. Find the target's rule.
2. First bring every prerequisite target up to date.
3. Run the recipe if the target is missing or any prerequisite is newer.
4. If a prerequisite was rebuilt, its new timestamp can make every target above it stale.

Make usually reasons from modification timestamps, not file contents. Consequently:

- wrong or future-dated clocks can fool it;
- omitted header dependencies can leave stale object files;
- a target that initially looks current may need rebuilding after one of its prerequisites is regenerated;
- `make clean` followed by a full build can hide a defective dependency graph rather than fix it.

By course convention, rules live in `Makefile` or `makefile`, which Make discovers automatically. The dependency language is portable in principle, but recipes invoke underlying commands, so shell/tool differences can make a Makefile operating-system dependent.

A correct multi-file Makefile

```
CC = gcc
CFLAGS = -Wall -Wextra -pedantic

launch: launch.o thruster.o
  $(CC) launch.o thruster.o -o launch

launch.o: launch.c launch.h thruster.h
  $(CC) $(CFLAGS) -c launch.c

thruster.o: thruster.c thruster.h
  $(CC) $(CFLAGS) -c thruster.c
```

If `thruster.h` changes, **both** objects must rebuild because both source files use that interface. Then `launch` must be relinked. A dependency graph is only correct when it names every input that can affect the target.

Automatic variables and Make expressions

Expression	Meaning in the current rule
`$@`	target name
`$^`	all normal prerequisites, with duplicate names removed, in their resulting order; order-only prerequisites are excluded
`$<`	first prerequisite
`$(CC)`	value of Make variable `CC`
`$(wildcard *.c)`	matching `.c` filenames
`$(SRCS:.c=.o)`	suffix substitution from `.c` to `.o`
`$(sort LIST)`	sorted, duplicate-free list using Make's locale-independent ordering

These automatic-variable semantics are specified by the GNU Make manual.

Do not confuse Make's `$@` with shell positional parameters or `$(...)` with shell command substitution. They are expanded by Make in this context.

5. The C preprocessor

What it does

Preprocessing occurs before proper compilation. It transforms tokens and selects source text using directives where `#` is the first non-whitespace preprocessing token on the line:

```
#include <stdio.h>
#define DAYS 7

#ifdef SPANISH
char *greeting = "Hola";
#else
char *greeting = "Hello";
#endif
```

- `#include` makes declarations/text from a header available in the translation unit.
- `#define` creates an object-like or function-like macro.
- `#if`, `#ifdef`, `#ifndef`, `#else`, and `#endif` control conditional compilation.
- `#ifdef NAME` tests whether `NAME` is defined, not whether its replacement value is true; even `#define NAME 0` makes `#ifdef NAME` succeed.
- Macros are not runtime variables. They are token replacements performed before the compiler type-checks the result.
- The stages are normally connected internally rather than saving a permanent preprocessed file; `gcc -E` asks the driver to expose that intermediate result.

Inspect it with:

```
gcc -E test.c       # preprocessed output, normally with line markers
gcc -E -P test.c    # easier-to-read output without line markers
```

Function-like macro and parentheses trap

```
#define a(b) b + 1
int x = a(1) + 1;
```

Expansion:

```
int x = 1 + 1 + 1;
```

Therefore `x == 3`. But the macro is unsafe:

```
2 * a(3)       /* expands to 2 * 3 + 1, so 7 rather than 8 */
```

Safer form:

```
#define ADD_ONE(b) ((b) + 1)
```

Parenthesize every parameter occurrence and the complete replacement expression. Also avoid arguments with side effects when a macro may evaluate them more than once.

Standard predefined macros

Macro	Replacement	Example/format	Reproducibility risk
`__LINE__`	current presumed source line as an integer	`3`	Stable for unchanged line structure, but changes with edits.
`__FILE__`	current input filename/path as a string	`"testCPP.c"` or an absolute path	Can embed a checkout/build path.
`__DATE__`	translation date string	`"Jul 27 2026"`; one-digit day is space-padded	Direct time-dependent bytes.
`__TIME__`	translation time string	`"14:05:09"`	Direct time-dependent bytes.
`__STDC__`	normally `1` for standard-conforming preprocessing	`1`	Usually informational.
`__STDC_VERSION__`	selected C standard version	e.g. `201112L` for C11	Captures/channels language-mode differences.

The exact `__FILE__` string depends on how the file was passed/opened, and time/date are the translation values. `__LINE__` and `__FILE__` can also be influenced by `#line`.

`assert`, `NDEBUG`, and conditional removal

An assertion is intended to check a programmer invariant during development:

```
assert(index < length);
```

If `NDEBUG` is defined before `<assert.h>` is processed, assertion calls become disabled. The argument is not evaluated:

```
gcc program.c -o debug-style
gcc -DNDEBUG program.c -o release-style
```

`-DNDEBUG` is the command-line equivalent of defining `NDEBUG` for preprocessing.

Never do this:

```
assert(someinitialization() == 0);  /* required work hidden in a check */
```

Do this instead:

```
int result = someinitialization();  /* always runs */
assert(result == 0);                /* optional invariant check */
if (result != 0) {
    return 1;                       /* real production handling */
}
```

Assertions are not replacements for validation or recoverable error handling.

8. Exam traps

Frequent traps

Tempting but wrong answer	Correct reasoning
"Reproducible means it produces approximately the same behavior."	For reproducible **builds**, the specified artifacts must be bit-for-bit identical under the declared conditions.
"Deterministic and reproducible are synonyms."	Determinism is the output property; reproducibility additionally needs an independently recreatable environment and verification path.
"Matching hashes prove the program is secure."	They prove byte identity of the compared objects, not correctness or safety of the source.
"The wildcard result is random."	In the assigned example it is sorted according to the active locale; locale variation changes the order.
"Sorting is enough in every shell command."	The sort itself must have controlled collation, such as `LC_ALL=C sort`, unless the operation is specified as locale-independent.
"Make rebuilds a target whenever a dependency's contents differ."	Traditional Make primarily compares existence and modification times.
"Only the stale `.o` file changes."	After recompiling an object, every dependent final target must be reconsidered and normally relinked.
"A header does not need to be a prerequisite because it is not compiled."	A header changes the translation unit, so every affected object depends on it.
"`gcc -c` creates an executable."	It stops before linking and normally creates an object file.
"`#define` creates a constant variable."	It creates a preprocessing replacement; no typed runtime object is implied.
"The preprocessor evaluates `1024 + 1`."	It substitutes tokens; the compiler evaluates the later constant expression.
"`a(1) + 1` becomes `(1 + 1) + 1` because macros understand expressions."	It is literal token substitution; parentheses appear only if the macro definition contains them.
"`__FILE__` is always only the basename."	It is the input name/path by which preprocessing opened the file and can contain a relative or absolute path.
"`__LINE__` makes repeated builds nondeterministic."	It is stable for the same preprocessed source location; time and path macros are the direct concerns here.
"An assertion is ordinary production error handling."	It is an optional invariant check and may be disabled completely.
"The `-DNDEBUG` program is guaranteed to segfault."	Dereferencing address 5 is undefined behavior; a segmentation fault is the expected common outcome, not a language guarantee.

Tempting but wrong answer	Correct reasoning
"`git blame` finds the original author of every idea."	It attributes the current surviving lines to commits.
"Committer and author are always the same."	The author wrote the change; a different committer may have integrated it.
"Rebase moves the same commits."	Rebase reapplies changes on new parents and creates new commit identities.
"Resolving a conflict means deleting `<<<<<<<` markers."	The resulting code must preserve the intended semantics of both changes and must be tested.
"A diff exit status of 1 means the patch command failed."	For `diff`, 1 normally means the files differ successfully.

9. Final two-minute recall sheet

```
REPRODUCIBLE BUILD
same source + declared environment + instructions
-> any party
-> all specified artifacts identical bit for bit
-> verify with cryptographic hashes

DETERMINISM
stable inputs + stable outputs + minimal environment capture

C PIPELINE
.c -> preprocess -> compile -> assemble -> .o -> link -> executable
gcc -E | gcc -S | gcc -c | gcc objects -o program

HEADER
declarations/interface in .h; definitions in .c
"local.h" versus <system.h>
every affected .o must depend on every used header

MAKE
target: prerequisites
<TAB>recipe
missing target OR newer prerequisite -> rebuild
$@ target | $^ normal prerequisites (deduplicated) | $< first normal
prerequisite


WILDCARD BUG
locale -> wildcard order -> $^ link order -> different bytes
fix explicit SRCS or $(sort $(wildcard *.c))


PREDEFINED
__LINE__ integer line
__FILE__ file/path string
__TIME__ HH:MM:SS - volatile
__DATE__ Mmm dd yyyy - volatile



HISTORY
merge preserves topology; rebase/cherry-pick rewrites/replays linearly
diff exit 1 = files differ successfully
```

Reproducibility Engineering - Module 6 Exam Guide

2. Client/server RDBMS versus SQLite

Comparison table

Question	Client/server RDBMS	SQLite
Where is the engine?	Separate server process/system	Library embedded in the application
How does the application reach data?	Client library over a network/protocol	Library reads/writes the local file directly
Storage	Usually multiple server-managed files	Entire database packaged as one cross-platform file in the course model
Setup	Server installation, accounts, configuration, startup, network	Open or create a file; no server setup
Concurrency	Designed for many clients; may use row/table-level locking and parallelism	Multiple readers, but writes to one database are serialized by file locking
Security	DB authentication, roles, grants, and network controls	Mainly filesystem permissions; no native per-user DB authentication
Scaling/remote access	Appropriate for centralized access by many machines	Best for local storage, not a shared network filesystem
Replication	Commonly supported	No built-in real-time replication in the assigned excerpt
Reproducibility strength	Topology can be described with Compose; central state can be administered	Very easy to package, copy, archive, hash, and distribute
Reproducibility risk	More versions, configuration, services, network, credentials, timing, and state	Engine version/build flags, file state, pragmas, extensions, and host filesystem can still matter

3. The eight SQLite features

Feature	Course definition	Reproducibility relevance	Remaining caveat
Serverless	No separate server process; the library accesses storage directly.	Removes server installation, startup, network, and server/client compatibility steps.	The library version, compile options, filesystem, and DB state still matter.
Zero Configuration	Creating an instance is as easy as opening a file.	Fewer undocumented setup steps and lower barrier for an independent rerun.	Application pragmas, extensions, paths, and schema initialization must still be recorded.
Cross-Platform	The whole DB resides in a portable file format.	The same artifact can be transferred, archived, hashed, and opened on different platforms.	Cross-platform file format does not guarantee identical query timing or floating-point behavior.
Self-Contained	One library contains the complete DB system and integrates with the host application.	Bundling it with the application can freeze the engine version and remove an external runtime dependency.	A dynamically linked or silently upgraded library can reintroduce version drift.
Small Runtime Footprint	Small code and memory requirements.	Easier to package and run on constrained or clean systems; lowers reproduction cost.	Small is not the same as fully specified.
Transactional	Transactions are ACID-compliant and safe across processes/threads.	Atomic state changes reduce partially written or corrupted experimental state.	ACID safety does not imply high write concurrency or deterministic query order.
Full-Featured	Supports most SQL92/SQL2 query features.	Familiar declarative SQL makes workflows easier to inspect and transfer.	"Most" is not "all"; dialect, typing, functions, and query plans can differ across DBMSs.
Highly Reliable	SQLite is tested and verified aggressively.	Fewer defects and strong compatibility reduce tool-induced reproduction failures.	Reliability does not compensate for missing data, code, or provenance.

Mnemonic expansion:

```
S Z C S S T F H
Serverless, Zero configuration, Cross-platform, Self-contained,
Small footprint, Transactional, Full-featured, Highly reliable
```

ACID in one line each

- **Atomicity:** a transaction happens completely or not at all.
- **Consistency:** a transaction takes the database from one valid state to another, respecting declared constraints.
- **Isolation:** concurrent transactions do not observe unsafe intermediate effects, according to the isolation rules.
- **Durability:** committed changes survive failures covered by the system's guarantees.

ACID and concurrency are not synonyms. SQLite can preserve transactional correctness while serializing writes, which limits high write throughput.

Reliability details from the assigned excerpt

The textbook excerpt reports, for the version discussed there:

- more than 10 million unit and query tests;
- a pre-release soak test of more than 2.5 billion tests;
- 100% statement and branch coverage, including out-of-memory and out-of-storage cases;
- a strong history of backward-compatible file formats, SQL syntax, APIs, and behavior.

These numbers are recognition-level details, not a substitute for the eight feature names.

4. SQLite licensing, use cases, and limits

Public-domain source code

SQLite is in the public domain and may be used for any purpose.

Positive consequences for reproducibility:

- researchers may inspect and audit the implementation;
- the exact source or compiled library may be redistributed with an artifact package;
- the engine can be embedded without a license-negotiation barrier;
- an old version can be archived, built, modified, and ported for a future rerun;
- independent teams can examine implementation details instead of depending on a closed vendor.

Important limitation:

- legal availability is **not** reproducibility by itself;
- public-domain status does not identify which version, commit, compile flags, patches, or extensions were used;
- there is no obligation to publish private modifications, so a researcher must deliberately release the exact modified source or binary;
- data, queries, schema, configuration, application code, and execution instructions are still required.

Good use cases

SQLite is a complement to, not a universal replacement for, a server RDBMS:

- **application files:** configuration, state, caches, document containers;
- **application cache:** temporary or in-memory relational processing;
- **archives/data stores:** a portable, read-only or distributable multi-table file;
- **client/server stand-in:** demos, evaluation copies, testing, and bug reports;
- **teaching tool:** almost no setup and easy sharing;
- **generic SQL engine:** virtual tables expose logs or other data to SQL.

Other features worth recognizing:

- dynamic typing;
- attaching multiple database files to one connection;
- fully in-memory databases;
- virtual tables backed by code or external data.

The five cases where SQLite is not the best choice

1. High transaction rates

SQLite uses file locking. Multiple connections can read, but a write requires exclusive coordination and write transactions are serialized. A client/server DBMS is normally better for many concurrent writers or very high transaction throughput.

2. Extremely large datasets

Everything is in one file and one filesystem. Multi-gigabyte data may stress random access, backup, transfer, and filesystem behavior. The issue is practical performance and management, not that SQLite becomes non-relational.

3. Access Control

SQLite relies mainly on filesystem permissions: read/write, read-only, or no access. Anyone with direct write access to the file can bypass application-level authorization. Use a server DBMS for sensitive multi-user data requiring authentication, roles, and grants.

4. Client/Server

SQLite has no native database network server. Letting several computers modify one SQLite file on a network filesystem is dangerous because network file locking may be unreliable. A web server can still use SQLite safely when its processes are on the same machine and access local storage.

5. Replication

The assigned excerpt says SQLite has no internal transaction-safe replication or redundancy. Copying the file is only a simple snapshot strategy and must not race with modification. Use a more capable RDBMS when real-time replication/failover is required.

Exact-label cue: **HTR - ELD - AC - C/S - R** = High Transaction Rates; Extremely Large Datasets; Access Control; Client/Server; Replication. Do not turn "extremely large" into a fixed size threshold.

Snapshot caveat

5. Docker Compose with PostgreSQL

Volume and state trap

An ordinary:

```
docker compose down
```

removes service containers and the Compose network but preserves the named volume by default. This helps durability, but it can hurt experimental repeatability: the next run reuses the PostgreSQL cluster, catalogs, persistent configuration, and non-pgbench objects. In this exact setup, `pgbench -i` recreates the standard benchmark tables, so it does not retain accumulated rows in those four tables. PostgreSQL shared buffers die with the container; a separately surviving host OS page cache is not stored in the named volume.

Therefore a reproducible database experiment must state whether it starts from:

- a fresh empty volume;
- a versioned database dump;
- a filesystem snapshot;
- a named checkpoint/state;
- or the previous run's persistent state.

Do not casually use `docker compose down -v`: it removes the declared named volume and its data.

`depends_on` is not readiness

The intended dependency, when expressed with valid short syntax, records startup order. It does **not** prove that PostgreSQL has completed initialization and is accepting connections. The benchmark may race with server startup.

A stronger real experiment would use a PostgreSQL health check such as `pg_isready`, a `service_healthy` dependency, or explicit retry logic.

Compose helps, but does not finish reproducibility

Also preserve or control:

- the exact image digest, not only a mutable tag;
- schema and initial data;
- PostgreSQL configuration and extensions;
- benchmark scale, clients, threads, duration, and random behavior;
- CPU, memory, storage, operating system/kernel, and competing load;
- clean versus warm caches;
- logs and exact output-processing scripts;
- secrets securely, without publishing real credentials.

6. Deterministic database experiments and SQL

The database is **stateful**. Reproducing the container image alone does not reproduce the database contents. Record the schema, exact data/snapshot, transaction state, initialization procedure, engine/configuration, query, parameters, and relevant external sources.

SQL row order is not implicit

Without an `ORDER BY`, SQL does not promise a row order. A sequential scan, index scan, different query plan, parallel execution, changed statistics, or engine version may return the same set of rows in a different sequence.

```
-- Nondeterministic order
SELECT run_id, score
FROM measurements;

-- Still not a total order if scores tie
SELECT run_id, score
FROM measurements
ORDER BY score DESC;

-- Deterministic total order if run_id is a non-null unique key,
-- for example a PRIMARY KEY
SELECT run_id, score
FROM measurements
ORDER BY score DESC, run_id ASC;
```

Core rule: **order by enough columns to break every tie**.

`LIMIT` and `OFFSET`

```
-- Unpredictable subset
SELECT * FROM measurements LIMIT 10;

-- Predictable only if the order is unique/total
SELECT *
FROM measurements
ORDER BY recorded_at, run_id
LIMIT 10;
```

`LIMIT 10` without a total order means "some ten rows," not a reproducible first ten. Different limits can even induce different query plans.

`OFFSET n` has the same ordering requirement: without a total order, which `n` rows are skipped is unspecified. Even with a total order, inserts/deletes between separate page requests can shift rows unless both requests use the same frozen snapshot; keyset/cursor pagination can avoid that drift. A large offset can also be inefficient because the server still computes the skipped rows.

Other common nondeterminism

- `random()` or sampling without a recorded seed;
- current date/time, sequence values, generated IDs, or volatile functions;
- a live table changing during the experiment;
- order-sensitive aggregates such as concatenation without an internal order;
- floating-point aggregation in varying parallel orders;
- locale, collation, time zone, encoding, and null-order differences;
- `SELECT *` when the schema later changes;
- reading a mutable external file, service, or foreign server.

Reproducible SQL checklist

1. Freeze or identify a database snapshot and schema version.
2. Preserve the query text, parameters, transaction/isolation context, and DBMS version.
3. Use a unique total `ORDER BY` when output sequence or `LIMIT` matters.
4. Seed controllable randomness and avoid or record volatile inputs.
5. Fix time zone, locale/collation, and encoding when they affect semantics.
6. Hash/archive external files and identify remote data versions.
7. Compare results as sets/multisets when order is irrelevant; use tolerances for justified floating-point differences.

Exam trap: deterministic **row order** and deterministic **row membership/value** are different requirements.

7. PostgreSQL foreign data wrappers

Mental model

A foreign data wrapper (FDW) lets PostgreSQL present an external source as a relational **foreign table**. The table definition is local metadata; the underlying data can remain in a file, another DBMS, or a service.

```
SQL query -> PostgreSQL planner/executor -> FDW -> external source
                        |
                        +-> join with local PostgreSQL tables
```

FDWs provide a SQL/MED-style mechanism for querying external sources.

Historical web-service example

The scanned book then uses the historical `www_fdw` wrapper and a Twitter JSON endpoint to illustrate a non-file source. The details are obsolete, but the architecture lesson remains:

```
CREATE EXTENSION wrapper
  -> CREATE SERVER with source-specific options/URI
  -> CREATE USER MAPPING for external identity/context
  -> CREATE FOREIGN TABLE with request and response fields
  -> GRANT SELECT
  -> query through ordinary SQL
```

Each wrapper defines its own options, formats, pushdown behavior, and read/write capabilities. A **user mapping does not itself grant SQL privileges**; the excerpt separately issues `GRANT SELECT`. Do not memorize the defunct Twitter endpoint or assume the old wrapper's limitations describe every current FDW.

Reproducibility implications

FDWs improve integration and make external data queryable with SQL, but they add dependencies:

- source path/URL and availability;
- source contents and version at query time;
- wrapper and PostgreSQL versions;
- schema/type mapping and parsing options;
- permissions, credentials, network, and remote service behavior;
- which predicates/aggregations are pushed to the remote source;
- query plan, statistics, and performance.

A foreign-table definition is **not a snapshot**. Archive/hash the external input or record an immutable source version if a later rerun must see the same rows.

8. PostgreSQL replication excerpt

The supplied PostgreSQL book pages include the end of an older replication walkthrough before the FDW section. The transferable concepts are:

- a primary (called **master** in the older text) produces write-ahead log (WAL);
- a standby (called **slave** there) replays WAL from the primary/archive;
- the standby can normally be queried but is not independently writable;
- a trigger/failover action promotes the standby after it replays the WAL available to it;
- the excerpt warns that **unlogged tables do not participate in replication**.

Reproducibility lesson:

- replication improves availability and preserves a changing copy, but a live replica is not an immutable experimental snapshot;
- identify a consistent point in the WAL/history, plus schema, configuration, and external dependencies;
- lag or promotion timing can otherwise expose a different state; with asynchronous replication, the newest primary commits can be absent when promotion occurs;
- data omitted from replication must be captured separately.

The exact `recovery.conf` commands in this older excerpt are version-specific and obsolete in modern PostgreSQL. Learn the WAL/standby/failover concept unless the examiner explicitly asks for the assigned historical syntax.

9. Functional equivalence versus bitwise identity

Definitions

- **Functionally equivalent:** programs satisfy the same relevant behavioral specification for the tested inputs/environment.
- **Bitwise identical:** the complete output files contain exactly the same bytes.

Bitwise identity is stronger. Two binaries may print the same output while differing in machine instructions, layout, build IDs, debug metadata, paths, timestamps, padding, or symbol tables.

Useful checks:

```
./hello-gcc
./hello-clang

cmp -s hello-gcc hello-clang
sha256sum hello-gcc hello-clang
```

- Equal normal SHA-256 checksums are practical evidence of byte identity.
- Different checksums prove the binaries differ, but do not explain where; use a structural comparison tool such as `diffoscope` for diagnosis.

10. Preprocessor macros and reproducible builds

Macro	Expansion	Reproducibility risk
`__FILE__`	Source-file path/name as seen by the preprocessor	Different invocation/build paths can embed different strings.
`__LINE__`	Current source line number	Stable if the source layout is unchanged; not dependent on day or build directory.
`__TIME__`	Compilation time as a string	Different build times can change output and binary bytes.
`__DATE__`	Compilation date as a string	Different build dates can change bytes.

Common causes and mitigations

Cause	Typical mitigation
Embedded time/date	Use a documented fixed build epoch such as `SOURCE_DATE_EPOCH`; avoid volatile macros.
Absolute build/source path	Use stable paths or compiler prefix-map options such as `-ffile-prefix-map` / `-fdebug-prefix-map`.
Compiler/linker drift	Pin and archive toolchain versions and flags.
Dependency drift	Lock/archive dependencies and base images.
Locale/time zone/hostname/user	Normalize or explicitly set them.
Unstable file ordering	Sort inputs and archive members deterministically.
Randomness	Set and record seeds; make generation order stable.
Parallel race/order	Fix the build rule or ordering; do not merely hope a single-threaded build hides it.
Build IDs/metadata/permissions	Configure deterministic values and normalize metadata/umask.

11. ReproTest

What it does

ReproTest builds an artifact twice under deliberately different simulated environments and compares the outputs. It tests whether the build is robust to irrelevant environmental variation.

Default variations can include:

- environment variables and executable path;
- build path and home directory;
- time and time zone;
- locale;
- user/group and umask;
- hostname/domain;
- file ordering;
- kernel, CPU count, and address-space behavior.

ReproTest result is scoped

Passing means "reproducible under the variations and artifacts tested." It does not prove source correctness, security, semantic correctness on every platform, or reproducibility under an untested compiler/hardware change.

12. C memory vocabulary

A simplified process-memory model helps locate where build differences appear.

```
Highest addresses
+------------------+
| Stack            | local variables; calls add frames; usually grows downward
+------------------+
| Heap             | dynamically allocated runtime data
+------------------+
| Globals          | long-lived mutable global/static variables
+------------------+
| Constants        | read-only literals/data
+------------------+
| Code             | read-only machine instructions
+------------------+
Lowest addresses
```

Do not say that all bytes in an executable map neatly to one of these runtime regions. Object/executable files also contain symbol tables, relocation data, debug sections, headers, and other metadata that may not be loaded as ordinary program memory.

13. Building Python from source and out-of-tree builds

Why out-of-tree is good practice

- generated files do not pollute or overwrite the source tree;
- cleaning is easy: remove one build directory while preserving source;
- multiple configurations/toolchains can be built side by side from one source tree;
- it is easier to see which files are original inputs and which are derived outputs;
- stale outputs are less likely to be accidentally committed or packaged;
- clean rebuilds and comparisons become easier to automate.

Out-of-tree layout improves hygiene, but exact source, archive checksum, configure options, compiler/toolchain, dependencies, environment, and install steps must still be preserved.

14. Make and dependency graphs

Core vocabulary

```
target: prerequisite1 prerequisite2
  recipe command
```

- **target:** file/action to produce.
- **prerequisites:** files/targets used to produce it.
- **recipe:** shell command that updates it; traditional Make requires a tab at the start.
- A target is out of date when it is missing or older than a normal prerequisite.
- Make walks dependencies from the requested goal and runs only necessary recipes.

Why Make matters for reproducibility

A Makefile records **prospective provenance** as an executable dependency graph. It reduces forgotten manual steps and recomputes derived artifacts after relevant input changes. It does not automatically pin software, capture data, remove nondeterminism, or prove that the declared graph is complete.

`.PHONY`

```
.PHONY: all clean
```

`all` and `clean` are actions, not meaningful files. Marking them phony ensures the recipes/dependencies are considered even if files named `all` or `clean` exist.

Make limitations/traps

- Make normally uses modification times, not content hashes.
- Clock skew or preserved timestamps can confuse decisions.
- An omitted prerequisite makes the graph wrong and may leave stale output.
- A phony prerequisite forces rebuild behavior; use it deliberately.
- Successful automation can still produce a nondeterministic artifact.
- `make -n` previews recipes and is useful for checking your reasoning.

16. Common exam traps

1. **Serverless does not mean database-less.** The SQLite engine is embedded as a library and still manages a relational database file.
2. **"SQLite client" does not imply a client/server architecture.** It can mean an application directly using the SQLite library.
3. **ACID does not mean unlimited concurrency.** Correct serialized writes can still have low throughput.
4. **One cross-platform file does not capture the full experiment.** Preserve code, schema, queries, parameters, version, and state too.
5. **Public domain does not mean reproducible.** It removes a legal barrier; it does not identify the exact artifact.
6. **`5433:5432` is host:container.** Service-to-service traffic uses `db:5432`, not `localhost:5433`.
7. **`depends_on` is startup order, not readiness.** Add health/retry logic in a real experiment.
8. **A persistent volume can preserve unwanted history.** State must be reset or versioned deliberately.
9. **Changing Postgres initialization variables does not necessarily change an already initialized volume.** Initialization concerns the first empty-data-directory start.
10. **Foreign-table creation is not source validation.** Errors can wait until query time.
11. **A selective `WHERE` does not create an index on a CSV file.** It may still scan every row.
12. **A foreign table is not a snapshot.** The external source can change between runs.
13. **Same output does not imply same binary.** Compiler and metadata differences can remain.
14. **`__LINE__` is source-dependent but not time/path-dependent.**
15. **Debug data is not the runtime stack.** Paths usually differ in binary metadata/read-only data.
16. **Out-of-tree builds organize outputs; they do not pin inputs.**
17. **Make rebuilds downstream, not upstream.** Follow dependency arrows from changed prerequisite to dependant target.
18. **Without a total `ORDER BY`, row order and `LIMIT` subsets are not reproducible.** A non-unique sort key may still leave ties.
19. **Replication is not an immutable snapshot.** Identify the exact database state/WAL point.
20. **Passing ReproTest is evidence under tested variations, not proof of program correctness.**

20. Final 2-minute recall sheet

```
ARCHITECTURES
  client/server = app + DB client library --network--> RDBMS server -> DB files
  SQLite        = app + SQLite library ----------------> SQLite file

SQLITE 8
  Serverless, Zero Configuration, Cross-Platform, Self-Contained,
  Small Runtime Footprint, Transactional, Full-Featured, Highly Reliable

PUBLIC DOMAIN
  lets you inspect/modify/archive/redistribute exact code
  does not identify the version or guarantee reproducibility

SQLITE NOT BEST FOR
  high transactions, huge data, fine access control, client/server, replication

COMPOSE
  down keeps named volume; depends_on does not mean ready

FDW
  CREATE defines metadata; SELECT validates/reads/parses
  creation proves none of exists/readable/parseable
  no raw-file index; SELECT *, WHERE selective, COUNT(*) can all scan CSV

DETERMINISTIC SQL
  no ORDER BY = no order guarantee
  ORDER BY nonunique = ties still unspecified
  LIMIT needs a total order + frozen input state

BINARY BUILDS
  same behavior != same bytes
  compiler/version/flags/path/time/environment matter
  risky: __FILE__, __TIME__
  -g may embed build paths

REPROTEST
  rebuild twice under varied environment, compare artifacts

MAKE
  target: prerequisites
  newer prerequisite -> rebuild target -> rebuild downstream dependants
  .PHONY all clean = action names, not real output files

OUT-OF-TREE
  source stays clean; builds/configurations separated; easy clean rebuild
```

Reproducibility Engineering - Module 7 Exam Guide

1. The whole module in one page

Tidy data: the mapping to memorize

Meaning	Physical layout
Each **variable**	one **column**
Each **observation**	one **row**
Each type of **observational unit**	one **table**
Each **value/entry**	one **cell**

Fast mnemonic: **Var -> Col; Obs -> Row; Type -> Table; Value -> Cell.** The first three are Wickham's tidy-data rules; value-to-cell is a useful extension.

The article's motivating number is **80%**: it is often said that 80% of data-analysis effort is spent cleaning and preparing data. It is a quoted motivation, not a measurement performed by Wickham.

The five messy forms

Messy structure	Repair	Assigned example
Headers contain values	**Melt/stack**: columns -> rows	income bands, `wk1...wk75`, `y2000...`
One column contains several variables	**Split** it	`m014` -> sex `m`, age `0–14`
Variables occur in rows and columns	**Melt, then cast/unstack**	weather `d1...d31` and `tmin/tmax`
Several observational-unit types share a table	**Split/normalize**	song facts versus weekly ranks
One observational-unit type is spread across files/tables	Add the source variable, then **append/union**	one file per year

Direction rules and terminology:

```
Wickham: melt = columns -> rows; cast = rows -> columns
Modern tools often say: unpivot = columns -> rows; pivot = rows -> columns
Badia uses "pivoting" more broadly for restructuring in either direction
```

Probability language

For a property with `yes` and `no` counts:

```
fraction/probability = yes / (yes + no)
odds                 = yes / no
odds ratio           = odds in group 1 / odds in group 2
```

SQL reshape formulas

```
wide -> long: VALUES + CROSS JOIN + CASE
long -> wide: SUM(CASE ...) + GROUP BY
```

Workflow formula

Two **independent** questions:

1. Was the old representation preserved? **destructive / non-destructive**
2. Can the transformation itself be inverted? **reversible / non-reversible**

For every action record:

```
TARGET – ACTION – PARAMETERS – WHEN – WHY – WHO
```

2. What tidy data means

Structure is not semantics

Most statistical data is physically rectangular, but rows and columns alone do not reveal its meaning. The same underlying values can be transposed or arranged differently. Tidy data links the **semantics** of the data to a standard **structure**.

- A **value** is one datum, normally numeric or textual.
- A **variable** contains values measuring the same attribute across units, such as height, temperature, or duration.
- An **observation** contains the values measured on the same unit across attributes, such as one person, day, or race.
- An **observational-unit type** is the entity or grain being observed, such as a person, a person-day, or a calendar day.

Before judging a table, finish this sentence:

> **One row represents ...**

If that sentence changes from row to row, or a row packs several separate trial observations into cells without explicit aggregation, the table is not tidy. A row may legitimately summarize many trials when the summary statistic/count and its grain are explicit.

Variable definitions depend on the analysis

The distinction is semantic, not purely mechanical:

- `height` and `weight` naturally look like two variables.
- `height` and `width` might instead be values of a `dimension` variable.
- `home_phone` and `work_phone` may be two variables, or the useful variables may be `phone_number` and `phone_type` in fraud analysis.

A useful diagnostic from Wickham is that functional relationships are easier to express between variables, while comparisons are easier between groups of observations.

Multiple observational-unit types

One study can need multiple tidy tables. An allergy study might have:

- `person(person_id, age, sex, race)` - one row per person;
- `symptom(person_id, date, sneezes, eye_redness)` - one row per person-day;
- `weather(date, temperature, pollen)` - one row per day.

Combining all three grains in one table repeats facts and creates update anomalies. Separate tables can be linked with keys.

Missing observations versus structural missing values

- If a measurement **should have been made but was not**, retain an explicit missing value.
- If the measurement is **logically impossible** and its absence can be reconstructed, it is structurally missing and may be omitted.

Example: an absent treatment result that should exist is real missingness; a pregnancy measurement for an impossible subject category can be structural missingness. Do not casually delete all `NULL`/missing values. Experimental design determines their meaning.

Fixed and measured variables

- **Fixed variables** describe the experimental design and are known in advance; database terminology often calls them dimensions.
- **Measured variables** are the quantities actually observed.

Ordering rows or columns does not change tidiness, but a readable table normally puts fixed variables before measured variables, keeps related columns together, and orders rows by the fixed variables.

Why tidy data helps

- Every tool can find a variable in the same place: a column.
- Aggregation and vectorized operations become straightforward.
- Values from the same observation stay paired in one row.
- Tool output needs less ad-hoc conversion before becoming another tool's input.
- A small set of reshape operations handles many apparently different datasets.

Important: **tidy is not a synonym for clean, correct, complete, normalized for every purpose, or error-free**. Tidiness describes structure. A tidy table can still contain impossible ages, duplicates, bias, or missing data.

Wickham relates tidy data to Codd's third normal form, but frames the constraints in statistical language and focuses on a dataset rather than an entire relational database.

3. The five forms of messy data in detail

3.1 Column headers are values

Presentation tables often turn values of a variable into headings.

```
religion | <$10k | $10–20k | $20–30k | ...
```

The real variables are:

```
religion | income_band | frequency
```

Melt/stack the income columns:

1. Keep columns that already represent variables; Wickham calls these `colvars`.
2. Move every other former header into one new variable.
3. Move the corresponding cells into one value variable.
4. Rename generic `column` and `value` fields to their meanings, here `income_band` and `frequency`.

The same diagnosis applies to Billboard columns `wk1` through `wk75`: week numbers are values of `week`; ranks are values of `rank`. After melting, derive the calendar date from entry date and week if needed.

A wide layout may still be convenient for data entry, compact display, or matrix computation. **Messy for analysis does not mean useless for every purpose.**

3.2 Multiple variables in one column

The tuberculosis header `m014` combines two variables:

```
m       -> sex = male
014     -> age = 0–14
```

Repair:

1. Melt the demographic columns into a temporary column/value pair.
2. Split the compound heading into `sex` and `age` using a delimiter, position/regular expression, or lookup table.
3. Rename the measured value `cases`.

Target shape:

```
country | year | sex | age | cases
```

This shape also makes it easy to join population data and calculate rates. Preserve the difference between count `0` and missing: it can reflect how the data was collected.

3.3 Variables in both rows and columns

The weather example contains:

- ordinary columns `id`, `year`, and `month`;
- day values hidden in headers `d1...d31`;
- variable names `tmin` and `tmax` stored as row values in `element`.

Repair:

1. Melt `d1...d31` so day becomes a value.
2. combine year, month, and day into `date`;
3. omit only reconstructible structural dates, such as February 30;
4. cast/unstack `element` so `tmin` and `tmax` become columns.

Final grain: one row per weather station-day.

3.4 Multiple observational-unit types in one table

Billboard data contains facts about two units:

```
song(song_id, artist, track, duration, ...)
ranking(song_id, date_or_week, rank)
```

If song metadata is repeated on every weekly rank row, the same fact is stored many times and can become inconsistent. Split the table and link it with `song_id`. This is normalization: express each fact once.

Many analysis tools expect one rectangular table, so analysis may later join the normalized tables. The tradeoff is:

```
normalized storage -> less redundancy and inconsistency
denormalized analysis table -> easier input for many analytical tools
```

3.5 One observational-unit type across files/tables

If each file contains the same type of record and has the same schema:

1. read each file;
2. add a source column, since the filename may encode year, location, or person;
3. append all rows.

If schemas, names, file formats, or missing-value conventions changed, first tidy files individually or in compatible groups, then combine them.

4. Restructuring data with SQL

Three restructuring cases

Situation	Main SQL operation
Related objects/multivalued attributes connected by identifiers	`JOIN` on primary/foreign keys
Same kind of data split among compatible tables	set operation, usually `UNION ALL`
Values of an implicit variable are encoded in rows or column names	pivot/unpivot with `CASE`, or a DBMS-specific operator

In spreadsheets and files, connecting identifiers may exist without declared key constraints. Finding the true key is then part of restructuring.

Dummy variables / one-hot encoding

The reading uses several names: dummy, indicator, design, one-hot, Boolean, binary, or qualitative variables. They convert categorical values into numeric inputs.

For categories `A`, `B`, and `C`:

```
SELECT
    name,
    SUM(CASE WHEN category = 'A' THEN 1 ELSE 0 END) AS A,
    SUM(CASE WHEN category = 'B' THEN 1 ELSE 0 END) AS B,
    SUM(CASE WHEN category = 'C' THEN 1 ELSE 0 END) AS C
FROM Data
GROUP BY name;
```

Mathematically, `n − 1` indicators can encode `n` categories because the omitted category is inferred when all others are zero. The reading notes that all `n` columns are nevertheless customary for many ML/data-mining inputs.

6. Workflows, destructive changes, and metadata

Why workflows matter

Exploring, cleaning, and preparing data is **iterative**. EDA reveals problems, transformations reveal more structure, and later evidence can invalidate earlier assumptions. A **workflow** is the ordered sequence of actions applied to data or results of earlier actions.

The two independent axes

Axis	First class	Second class
Is the old representation retained?	**Destructive:** overwritten/lost	**Non-destructive:** old version remains
Can the transformation itself be inverted?	**Reversible:** original reconstructible	**Non-reversible:** information discarded

The four combinations are possible:

Mode	Reversible transformation	Non-reversible transformation
Destructive	in-place `x := x + 1`, assuming no overflow/rounding	in-place `TRIM(name)` or `DROP COLUMN`
Non-destructive	new column/table containing `x + 1`	new table containing trimmed names while raw table remains

Key nuance: preserving a raw table lets the **workflow** return to it, but it does not make `TRIM` mathematically invertible from the trimmed result alone.

A destructive operation can be reversible. Concatenating two strings with a recorded separator that never occurs in either input can be undone by splitting. A non-reversible operation can be implemented non-destructively so the original is still available. Non-destructive work costs additional storage.

SQL implementation patterns

Destructive in-place update:

```
UPDATE Data
SET attribute = function(attribute);
```

Non-destructive new attribute:

```
ALTER TABLE Data ADD COLUMN transformed datatype;

UPDATE Data
SET transformed = function(attribute);
```

Non-destructive new table/checkpoint:

```
CREATE TABLE NewData AS
SELECT
    ...,
    function(attribute) AS transformed
FROM Data;
```

Copying the entire dataset after every tiny action duplicates unchanged data and creates too many tables. A checkpoint is more sensible after a major transformation or a coherent batch.

Views as executable workflow steps

```
CREATE VIEW CleanData AS
SELECT ...
FROM RawData
WHERE ...;
```

A normal view:

- stores its **query definition**, not an independent copy of rows;
- reruns that query when read;
- changes when its source tables change;
- consumes no extra table-data storage;
- can be defined on another view, forming a multi-step workflow;
- has its definition recorded by the catalog.

Therefore, a view preserves structural provenance, but it is **not a frozen snapshot**. The DBMS records **how** the view is defined, not the analyst's human **why**.

Descriptive metadata, provenance, and lineage

- If metadata arrives with a dataset, EDA should verify that reality matches it.
- If it does not arrive, use EDA to create a description before serious analysis.
- Record every modified attribute, new attribute, derived dataset, and transformation.
- The chain of changes leading to a dataset is its **provenance/lineage**.

This supports repeatability, transparency, alternative analyses, sharing, publication, and auditing. Bad assumptions can be traced to the downstream products they affected.

What the DBMS catalog records

Typical system/catalog metadata includes:

- table and view names;
- schemas, column names, and data types;
- primary, unique, and foreign keys;
- users/roles and privileges;
- view definitions.

It does not automatically provide enough scientific meaning, quality assessment, or decision rationale.

PostgreSQL commands to recognize:

Command	Meaning
`\l`	list databases
`\d`	list tables/views in the current database
`\dt`	tables only
`\dv`	views only
`\dp`	objects and access privileges
`\d name`	columns, types, and details for an object
`\dg` or `\du`	roles/users

MySQL commands to recognize:

```
SHOW DATABASES;
SHOW SCHEMAS;
SHOW TABLES FROM database_name;
SHOW TABLES FROM database_name LIKE 'pattern';
SHOW CREATE TABLE table_name;
SHOW COLUMNS FROM table_name;
SHOW CREATE VIEW view_name;
```

Attribute-level metadata

The reading proposes one row per attribute, with fields such as:

```
attribute_name, representation, domain, provenance,
accuracy, completeness, consistency, currency,
precision, certainty
```

- `representation` should match the storage type/format.
- `domain` should explain real-world meaning and valid, invalid, and typical values in plain language.
- Quality fields describe fitness and reliability; not every field applies to every attribute, so some metadata `NULL`s are legitimate.

Field	Meaning
accuracy	closeness to the true value
completeness	whether necessary values/scope are present
consistency	absence of contradictions
currency	how up to date the data is
precision	measurement resolution/granularity
certainty	confidence in the value/source

The action log: six items

Record for every transformation:

1. target attribute(s);
2. action/function;
3. parameter values;
4. when it happened;
5. why it happened;
6. who performed it.

Mnemonic: **TARGET - ACTION - PARAMETERS - WHEN - WHY - WHO**.

The SQL statement can often capture the first three. A timestamp reconstructs order and dependent changes. `why` is especially important and often omitted: later discoveries can overturn the assumption behind a cleaning choice. `who` supports legal, regulatory, and access audits. Views automatically retain their query definition, but still need an external record of `why`.

8. Database architectures and benchmarking

Do not define an architecture merely by container count. A MariaDB server and its client can share a container and still use a client-server architecture.

What makes the comparison credible

- same BenchBase source revision;
- equivalent YCSB workload parameters;
- same host hardware and comparable current load;
- fresh database state for each workload;
- separately named result directories;
- explicit DBMS/container versions;
- recorded throughput unit and raw JSON;
- preferably repeated runs, warm-up policy, and summary statistics.

Image size is not benchmark throughput

The self-built SQLite image intentionally contains both build-time and runtime material. It is expected to be larger than a prebuilt runtime-only image because it can retain:

- full JDK and Git;
- source and `.git` history copied into the build;
- Maven wrapper/downloads and dependency cache;
- compiler/build outputs;
- the compressed distribution and extracted runtime.

A runtime image can use a multi-stage build and copy only the runnable distribution into a smaller final base. Image size measures deployment/storage footprint, not speed, memory use, database volume size, or scientific reproducibility by itself.

11. Highest-yield exam traps

Tidy-data traps

1. **Rectangular does not imply tidy.** First identify the row grain and semantic variables.
2. **Tidy does not imply clean.** Structural tidiness says nothing about correctness, duplicates, bias, or validity.
3. **One observational-unit type gets one table.** It is not one table per individual observation.
4. A name such as `y2000`, `wk1`, `Male`, or `Rozz` in a header may be a **value disguised as a column name**.
5. A cell such as `A,B,B` is not atomic when those elements are separate trial observations.
6. Repeating student or song metadata usually signals mixed grains and normalization need.
7. **Melt** moves columns to rows; **cast/pivot** moves rows to columns.
8. Do not drop all missing values. Distinguish an expected-but-missing measurement from reconstructible structural missingness.
9. A wide layout can be useful for human entry, compact display, or matrix computation even when it is untidy for analysis.

SQL and numerical traps

1. On a tidy table of aggregated counts, totals use `SUM(n)`, not `COUNT(*)`.

2. Force non-integer division with `1.0 *`, `CAST`, or the DBMS equivalent.
3. Fraction/probability is `yes / total`; odds is `yes / no`.
4. `(9/52)/(4/48) = 2.0769` is a **risk ratio**, not the requested odds ratio.
5. The earthquake cross join creates all magnitude-year combinations; `CASE` selects the matching old column.
6. `3` earthquake rows times `3` generated years gives `9` rows.
7. `GROUP BY` determines output grain. Long-to-wide needs one output row per `magnitude`.
8. `SUM(CASE...)` and `MAX(CASE...)` agree only under suitable uniqueness assumptions.
9. No `ORDER BY` means no guaranteed order. `LIMIT` does not fix that.
10. An `ORDER BY` on a non-unique field still needs a unique tie-breaker for deterministic top-N selection.

Workflow and metadata traps

1. **Destructive/non-destructive** and **reversible/non-reversible** are independent axes.
2. `TRIM` stays intrinsically non-reversible even if `CREATE TABLE AS` preserves the raw source.
3. Re-adding a dropped column does not recover the lost values.
4. A reversible update needs its assumptions: no overflow, rounding, concurrent mutation, or unknown parameters.
5. A normal view is a live query definition, not a frozen data snapshot.
6. The catalog records technical structure and view SQL, not the analyst's human rationale.
7. Provenance/lineage is the history of sources and transformations, not merely a schema description.
8. For every action, remember **target, action, parameters, when, why, who**.

Benchmark traps

1. Embedded versus client-server describes process/library architecture, not simply the number of containers.
2. One run on one machine does not establish a universal DBMS ranking.
3. Docker image bytes are not throughput, runtime memory, database volume size, or total disk usage.
4. `file_fdw` exposes external data; it does not import and index it as a local table.

14. Two-minute final recall sheet

```
TIDY
  variable -> column
  observation -> row
  observational-unit type -> table
  value -> cell

FIVE MESSES
  headings are values       -> melt
  several vars in one col   -> split
  vars in rows and columns  -> melt, then cast
  several unit types        -> normalize/split tables
  one type across files     -> source column + append

SQL RESHAPE
  wide -> long = VALUES + CROSS JOIN + CASE
  long -> wide = SUM(CASE...) + GROUP BY


WORKFLOW
  axis 1: destructive / non-destructive
  axis 2: reversible / non-reversible
  log: TARGET ACTION PARAMETERS WHEN WHY WHO
  view = live query definition, not snapshot
```

Reproducibility Engineering - Module 8 Exam Guide

1. The entire module in two pages

Five rules that unlock most questions

1. **Relational data is tabular; XML and JSON are hierarchical; HDF5 is a binary hierarchy optimized for large numerical arrays.**
2. **JSON objects are unordered mappings, but JSON arrays are ordered sequences.** Do not call all JSON unordered.
3. **`properties` does not make a property mandatory.** Only `required` does that, and extra properties remain allowed unless the schema explicitly forbids them.
4. **JSON Schema composition is Boolean logic:** `allOf` = AND, `anyOf` = inclusive OR, `oneOf` = exactly one/XOR, `not` = negation.
5. **HDF5 = GDA: G**roups organize, **D**atasets store array data, **A**ttributes store small metadata.

Format comparison to memorize

Feature	Relational	XML	JSON	HDF5
Main structure	Flat tables of rows and columns	Ordered element tree with text and attributes	Tree of objects, arrays, and scalar values	File-system-like hierarchy of groups and array datasets
Typical schema	Required and fixed in advance (lecture contrast)	Flexible/self-describing; optional DTD or XML Schema	Flexible/self-describing; optional JSON Schema	Dataset type/shape are stored; overall application layout is usually a convention
Main query/access style	SQL	XPath/XQuery/XSLT	Lecture-era answer: no mature widely used query language; usually manipulated in a program	Path lookup and array slicing through an HDF5 API such as `h5py`
Ordering	A relation is unordered; query output needs `ORDER BY`	Child/document order matters; attributes are unordered	Object property order does not matter; array order does	Array positions are ordered/indexed; groups act like named containers
Typical implementation	Mature native relational DBMS	Lecture-era answer: often an XML layer over a relational implementation	Closely coupled to program objects/parsers and document/NoSQL exchange	Binary open format plus HDF5 library and language bindings

JSON Schema validator data flow

```
JSON instance + JSON Schema -> validator -> valid / invalid + validation errors
```

- The **instance** is the data being checked.
- The **schema** is another JSON document containing rules.
- The **validator** evaluates the instance against those rules.
- The result is a Boolean validity decision, normally accompanied by useful error information.

Product-schema rules

```
root                must be an object
productId           integer, required
productName         string, required
price               number, required, strictly greater than 0
tags                optional array
tags items          strings
tags length         at least 1
tags duplicates     forbidden
unknown properties  allowed by default
```

Composition truth table to memorize

For these two subschemas:

```
A = { "type": "string" }
B = { "maxLength": 5 }
```

Instance	A	B	allOf	anyOf	oneOf
`"foo"`	T	T	T	T	**F**
`"a"`	T	T	T	T	**F**
`"1234567890"`	T	F	F	T	**T**
`42`	F	T	F	T	**T**

The surprising cell is `B(42) = true`: `maxLength` applies to strings and does not reject a number. A type-specific keyword does **not** silently add a type check. Add `"type": "string"` when the type matters.

HDF5 mental picture

```
weather.hdf5
/
+-- 15                          group
|   +-- temperature             dataset [18.2, 18.4, ...]
|   |   +-- attrs: delta, start_time, unit
|   +-- wind                    dataset [3.1, 3.3, ...]
|       +-- attrs: delta, start_time, unit
+-- 20                          group
    +-- temperature             dataset [64.0, 65.0, ...]
        +-- attrs: delta, start_time, unit

root attrs: description
```

Visitor in one sentence

Use **Visitor** when operations change more often than the composite object structure: a Traverser walks the Composite, the Visitor collects exposed state and centralizes new operations. This makes operations easy to add but weakens encapsulation and makes structural changes harder.

Tidy data in one sentence

```
one variable per column + one observation per row + one value per cell
```

Fast reshaping rule:

```
variable names stored in rows       -> PIVOT wider
variable values embedded in headers -> UNPIVOT longer
multiple values inside one cell      -> split
one variable split across columns    -> concatenate
```

2. Relational, XML, JSON, and equivalence

Relational data

- Data is represented by relations/tables.
- Columns represent attributes/variables and rows represent tuples/observations.
- The schema defines names, types, keys, and constraints before or alongside loading the data.
- SQL provides standardized declarative querying, joins, grouping, aggregation, and constraints.
- A relation has no inherent row order. `ORDER BY` is required when result order matters.
- Hierarchies normally require several tables connected through keys and joins.

Best fit: strongly structured data, integrity constraints, relationships, transactions, joins, and ad hoc queries.

XML

- XML represents a rooted, hierarchical document tree.
- Elements can contain text, attributes, and child elements.
- Child/document order is meaningful; XML attributes themselves are unordered.
- A document can be used without a schema, or validated with a DTD/XML Schema.
- XPath navigates paths; XQuery queries and constructs XML; XSLT transforms it.
- XML is verbose but supports document-oriented features such as mixed text and markup.

Best fit: marked-up documents, mixed content, namespaces, established XML ecosystems, and data where document order matters.

XML validation

- **Well-formed XML** obeys XML syntax: one root element, properly nested and matching tags, quoted attribute values, and case-sensitive names.
- **Valid XML** is well-formed **and** conforms to a declared schema such as a DTD or XML Schema/XSD.
- A **DTD** declares allowed elements, attributes, content structure, and occurrence. Common markers are:

```
?  zero or one
*  zero or more
+  one or more
```

- DTD attribute type `ID` gives a unique identifier; `IDREF` refers to one declared ID; `IDREFS` contains several references.
- **XML Schema/XSD** is itself written in XML and supports richer primitive types, complex types, namespaces, numerical/string restrictions, and cardinality rules such as `minOccurs`/`maxOccurs`.
- A well-formed document can still be invalid against its DTD/XSD, just as syntactically valid JSON can fail a JSON Schema.

JSON

JSON values are:

```
object | array | string | number | boolean | null
```

- An **object** maps string property names to values: `{ "name": "Ada" }`.
- An **array** is an ordered sequence: `["home", "green"]`.
- JSON is lighter than XML and maps naturally into dictionaries/objects, lists/arrays, and scalar values in programming languages.
- JSON itself does not require a schema. JSON Schema adds optional validation rules.
- Strict JSON has no comments and no trailing commas; property names and strings use double quotes.
- Property names are case-sensitive: `productName` and `ProductName` are different.

JSON was designed as a compact serialization/interchange form for program data and maps directly to common dictionaries/objects and lists/arrays. Compared with XML, it is generally less verbose and has a smaller conceptual model. XML and JSON are both useful for flexible or semi-structured data, but flexibility trades away some guarantees that a schema can restore.

Best fit: APIs, configuration, lightweight data interchange, nested application data, and human-readable structured data.

The ordering rule examiners like

```
Relational relation: unordered
XML child nodes:     ordered
JSON object members: unordered
JSON array items:    ordered
```

Consequences:

- Reordering object properties does not change the JSON value.
- Reordering array items normally does change the JSON value.
- A textual `diff` detects serialization differences; it does not decide structural or semantic equivalence.
- SQL can return rows in any order unless `ORDER BY` is present.

Three different kinds of equality

Kind	Question	Example
Syntactic/textual equality	Are the serializations character-for-character or byte-for-byte equal?	Whitespace or property reordering breaks it.
Structural equality	Do they represent the same data structure/value?	Object member order is ignored; array order is preserved.
Semantic equivalence of schemas	Do both schemas accept exactly the same set of instances?	Different schema syntax can still impose the same constraints.

Example:

```
{"x": 1, "tags": ["a", "b"]}
{"tags": ["a", "b"], "x": 1}
```

These are textually different but structurally equivalent. Replacing the second array with `["b", "a"]` makes them structurally different.

---

3. JSON Schema - the scoring core

What JSON Schema provides

Validating an instance against a schema can:

- reject wrong top-level or property types;
- enforce required properties and numerical/string/array constraints;
- catch errors at a system boundary instead of much later in an analysis;
- give producers and consumers an explicit data contract;
- document expected structure and meaning;
- support editor hints, generated forms/code, tests, and automated validation;
- improve interoperability and consistency across tools and experiments.

A schema only checks rules it actually expresses. A schema cannot prove that a scientifically wrong value is correct merely because the value has the expected type and range.

Keep two validity levels separate:

- **Syntactic validity:** the text obeys JSON grammar.
- **Schema validity:** the parsed JSON instance satisfies a particular JSON Schema.

A document can be valid JSON syntax and still be invalid against the product schema.

Product-schema anatomy

```
{
  "title": "Product",
  "description": "A product from Acme's catalog",
  "type": "object",
  "properties": {
    "productId": {
      "description": "The unique identifier for a product",
      "type": "integer"
    },
    "productName": {
      "description": "Name of the product",
      "type": "string"
    },
    "price": {
      "description": "The price of the product",
      "type": "number",
      "exclusiveMinimum": 0
    },
    "tags": {
      "description": "Tags for the product",
      "type": "array",
      "items": { "type": "string" },
      "minItems": 1,
      "uniqueItems": true
    }
  },
  "required": ["productId", "productName", "price"]
}
```

Keyword map:

Keyword	Meaning in this schema
`title`, `description`	Annotations for humans/tools; they do not reject an instance.
`type: object`	The root instance must be an object.
`properties`	If a named property exists, validate its value using the nested schema.
`required`	The listed property names must exist.
`type: integer`	Value must be an integer.
`type: number`	Value may be an integer or non-integer number.
`exclusiveMinimum: 0`	Value must be strictly greater than zero.
`type: array`	Value must be an array.
`items`	Every array item must satisfy the given schema.
`minItems: 1`	The array cannot be empty.
`uniqueItems: true`	No two array items may be equal.

The two defaults students confuse

Defined does not mean required

This schema is valid for `{}`:

```
{
  "type": "object",
  "properties": { "name": { "type": "string" } }
}
```

`properties` says what to do **if** `name` appears. Add `"required": ["name"]` to demand its presence.

Unlisted does not mean forbidden

Unknown properties are accepted by default. To close an object schema, add:

```
"additionalProperties": false
```

The product schema does not contain that keyword, so a valid product may also have `"discount": 0.1`.

Progressive validation

When asked whether an instance is valid "up to line N," apply only the constraints introduced by that prefix:

1. `title` and `description` are annotations, so all JSON instances still pass.
2. `type: object` rejects strings, numbers, arrays, booleans, and `null` at the root.
3. `properties` constrains only matching, present properties.
4. `required` finally makes selected names mandatory.

Composition operators

Operator	Logic	Valid exactly when...
`allOf`	AND	every subschema validates
`anyOf`	inclusive OR	at least one subschema validates
`oneOf`	exactly-one/XOR	exactly one subschema validates
`not`	NOT	the nested subschema does not validate

Do not read `oneOf` as ordinary English "one or more." If two alternatives both match, `oneOf` fails.

Type-specific keyword trap

This schema by itself does not require a string:

```
{ "maxLength": 5 }
```

It constrains string instances, while a number such as `42` is not rejected by `maxLength`. The safe explicit form is:

```
{ "type": "string", "maxLength": 5 }
```

Structural versus semantic schema equivalence

```
{ "oneOf": [{"type": "string"}, {"type": "integer"}] }
```

and

```
{ "anyOf": [{"type": "integer"}, {"type": "string"}] }
```

are not structurally equivalent: the operator and array order differ. They are semantically equivalent because no instance can be both a JSON string and a JSON integer. For disjoint alternatives, "at least one" and "exactly one" accept the same set.

Counterexample showing that this is not generally true:

```
{
  "anyOf": [
    {"type": "number"},
    {"type": "integer"}
  ]
}
```

An integer matches both branches, so it passes `anyOf` but would fail the corresponding `oneOf`.

5. HDF5 and `h5py` in depth

Why the format exists

The reading begins with an endianness failure: raw floating-point bytes written on a little-endian machine were interpreted on a big-endian machine as values around 40 orders of magnitude too small. The lesson is broader than byte order:

> A storage-format decision is also a communication and reproducibility decision.

A standard self-describing format records enough structure and type information for different programs and machines to interpret the same data consistently. HDF5 handles cross-platform representation details such as endianness.

The three public data-model objects

Object	Analogy	Purpose
Group	Directory/folder	Contains named groups and datasets; creates hierarchy.
Dataset	Typed NumPy-like array on disk	Stores homogeneous numerical or other array data with a shape and dtype.
Attribute	Small key-value annotation	Stores metadata attached directly to a group or dataset.

Rule of thumb: put large or sliceable data in a dataset; put small descriptive facts such as units, timestamps, and sampling intervals in attributes.

HDF5 is three related things

1. An open file specification and data model.
2. A maintained library, originally in C, with language bindings.
3. An ecosystem of clients and scientific platforms such as Python, MATLAB, IDL, Java, and others.

An organization can define an **application format within HDF5**: for example, one group per station, named datasets for measurements, and agreed attribute names. HDF5 supplies the container semantics; the community supplies the domain convention.

Architecture to recognize:

```
user code
  -> h5py or PyTables
  -> HDF5 C API and public groups/datasets/attributes
  -> internal structures and a 1-D logical address space
  -> file driver
  -> operating-system storage
```

Group members are internally indexed with B-trees for efficient lookup even when a group contains many objects. `h5py` closely exposes native HDF5 concepts; PyTables builds additional scientific-database features such as indexing on top of HDF5.

Why hierarchy beats a flat naming convention

A flat archive might need names such as:

```
temperature_15
delta_temperature_15
wind_15
delta_wind_15
temperature_20
...
```

HDF5 expresses the relationship directly:

```
/15/temperature
/15/wind
/20/temperature
```

Metadata sits on the relevant object instead of in a distant lookup table. This reduces naming hacks and makes a file explorable.

Partial I/O

```
dataset = f["/15/temperature"]
first_ten = dataset[0:10]
every_other = dataset[0:10:2]
```

`dataset` is a proxy for data on disk. Slicing asks HDF5 to read only the selected region into memory. This is crucial when the complete dataset is hundreds of gigabytes or larger.

Contrast with NumPy slicing:

- A NumPy slice is commonly a **view** into the same in-memory array; changing it can change the original.
- A slice read from an HDF5 dataset returns an in-memory **copy** of the disk data.

Allocation and compression

The reading emphasizes that a declared dataset can be much larger than currently written data, with storage allocated as needed. HDF5 also supports transparent dataset-level compression:

```
compressed = f.create_dataset(
    "comp",
    shape=(1024,),
    dtype="int32",
    compression="gzip",
)
compressed[:] = np.arange(1024)
```

Compression is transparent to later reads. Chunk layout and access patterns affect performance, so write/read in sensible blocks rather than one scalar at a time.

Common creation and access patterns

```
with h5py.File("weather.hdf5", "w") as f:
    f["/15/temperature"] = np.array([18.2, 18.4])
    f["/15/temperature"].attrs["unit"] = "degree Celsius"
```

The context manager closes the file even if an exception occurs.

File modes:

Mode	Meaning
`"r"`	Read-only; file must exist.
`"r+"`	Read and write; file must exist.
`"w"`	Create a new file, overwriting an existing file.
`"w-"` or `"x"`	Create, but fail if the file already exists.
`"a"`	Read/write; create if absent.

For reproducible code, state the mode explicitly. Do not depend on the old excerpt's historical default behavior.

The excerpt predates Python 3's dominance. Translate its old syntax when using current Python:

```
print x or print "%s" % x  -> print(x) or print(f"{x}")
mapping.iteritems()         -> mapping.items()
implicit h5py file mode     -> pass "r", "w", "a", etc. explicitly
```

NumPy and HDF5 types

- A NumPy array has a fixed `dtype`, shape, and memory representation.
- An HDF5 dataset likewise has a fixed type and shape.
- `h5py` maps HDF5 types to NumPy dtypes, enabling efficient interchange.
- Appropriate types matter for correctness, file size, and performance.

Where HDF5 is the wrong tool

HDF5 is not a replacement for every database or text format:

- Use a relational DBMS when relationships, joins, constraints, transactions, or multi-user query/update behavior dominate.
- Use CSV for tiny, simple 1D/tabular datasets that must open everywhere without a special library.
- Use JSON for lightweight nested API/configuration data.
- Use XML for ordered/mixed-content documents and XML-specific standards.
- Binary HDF5 is not pleasant to inspect in a text editor, review with a normal line diff, or merge in Git.

Tools to recognize

- **HDFView:** graphical HDF5 browser; displays hierarchy, data, and attributes.
- **ViTables:** graphical viewer oriented toward PyTables but able to read generic HDF5.
- **`h5ls`:** lists groups/datasets; `h5ls -vlr file.hdf5` gives verbose recursive metadata.
- **`h5dump`:** prints metadata and actual stored values in a verbose textual representation.

Independent inspection is a reproducibility practice: it can reveal wrong dtypes, shapes, paths, or attributes before archival/sharing.

File drivers - recognition level

Driver	Purpose
`core`	Keep the HDF5 image in memory for fast access; with `backing_store=True`, load an existing image on open and save it on close.
`family`	Split one logical HDF5 file into fixed-size file members.
`mpio`	Parallel access by multiple processes through MPI.

Drivers change storage mechanics underneath the same high-level group/dataset/attribute interface.

Recognition-only feature: an HDF5 **user block** is arbitrary prefix data before the HDF5 payload. Its size is a power of two and at least 512 bytes. Do not modify that prefix while the file is open through HDF5.

Visitor pattern and hierarchical traversal

The assigned design-pattern excerpt presents these roles:

```
Composite object hierarchy <- walked by Traverser <- guides Visitor
Visitor collects exposed state -> Client asks for operations/results
```

Why use it:

- operations can be added without placing each new method in every composite class;
- operation code is centralized in the Visitor;
- the composite structure itself need not be rewritten for each new calculation.

Costs:

- composite classes must expose state, weakening encapsulation;
- the traversal logic and Visitor depend on the structure;
- adding/changing composite element types is harder than adding a new operation.

HDF5 is a natural use case: files contain arbitrarily deep groups and datasets. A library-supplied visitor/traversal facility can systematically reach every object, avoiding fragile hand-written nested loops. The HDF5 reading recommends using its Visitor feature before manual recursion.

---

7. Common exam traps

1. **JSON is not wholly unordered.** Objects are unordered; arrays are ordered.
2. **`diff` tests text, not structure.** Whitespace and object-property order can change without changing the JSON value.
3. **`title` and `description` do not validate.** They are annotations.
4. **`properties` does not imply `required`.** A missing defined property is allowed unless listed in `required`.
5. **Unknown properties are allowed by default.** Use `additionalProperties: false` to reject them.
6. **Property names are case-sensitive.** `ProductName` does not satisfy required `productName`.
7. **`exclusiveMinimum: 0` means `> 0`, not `>= 0`.**
8. **`uniqueItems` checks duplicates; `minItems` checks length.** They solve different problems.
9. **A type-specific keyword does not add the type.** `maxLength` alone does not reject `42`.
10. **`anyOf` is inclusive OR.** One or several branches may match.
11. **`oneOf` is exactly one.** Two matches make the instance invalid.
12. **Different schema syntax may be semantically equivalent.** Compare accepted instance sets.
13. **Attributes are metadata, not the main large array.** Store large/sliceable values as datasets.
14. **HDF5 is not a relational DBMS.** It does not replace joins, relational constraints, or transaction-oriented querying.
15. **HDF5 slicing reads selected data from disk.** It does not first load the full file.
16. **NumPy slice and HDF5 slice differ.** NumPy often gives a view; HDF5 reads give a copy.
17. **`"w"` overwrites.** Use `"w-"`/`"x"` when accidental replacement must fail.
18. **Visitor trades encapsulation for extensibility of operations.** It is not free abstraction.
19. **Tidy direction matters.** Variable names in rows go wider; variable values in headers go longer.
20. **Use `UNION ALL` for reshaping.** `UNION` can delete legitimate duplicates.
21. **Preserve leading zeros.** Keep `00` and `014` as text until they no longer encode categories.
22. **`NULL` is not zero.** Missing case count and observed zero cases have different meanings.
23. **SQL result order is not guaranteed.** Add `ORDER BY` when output order is part of the answer.

---

9. Final two-minute recall sheet

```
FORMATS
  relational = flat tables, explicit schema, SQL, unordered relations
  XML        = ordered element tree, optional DTD/XSD, XPath/XQuery
  JSON       = unordered objects + ordered arrays, optional JSON Schema
  HDF5       = binary groups + datasets + attributes, partial numerical I/O

JSON SCHEMA
  properties != required
  unknown properties allowed unless additionalProperties: false
  exclusiveMinimum: 0 means > 0
  minItems checks count; uniqueItems checks duplicates
  title/description annotate; type/properties/required validate

BOOLEAN COMPOSITION
  allOf = AND
  anyOf = one or more
```

```
  oneOf = exactly one
  not   = NOT
  maxLength alone does not require a string

HDF5 = GDA
  Group     = hierarchy/container
  Dataset   = typed array on disk
  Attribute = small attached metadata
  slicing   = partial disk I/O
  w overwrites; w-/x fails if present; r reads; r+ updates; a creates/updates

VISITOR
  operation changes easy; structure changes hard
  centralized operations; weakened encapsulation

TIDY
  variable -> column
  observation -> row
  value -> cell
  names in rows -> PIVOT
  values in headers -> UNPIVOT
  several values in cell -> split
  preserve 00 and 014 as text
```

Reproducibility Engineering - Module 9 Exam Guide

1. The whole module in one page

Five memory blocks

```
LOCAL vs REMOTE
  Local model = large, hardware-hungry, more self-contained and controllable
  Remote API  = small package, easy local hardware, but provider/network/cost
dependency

REPEATABLE LLM CALL
  exact request + pinned model/version + temperature 0 + fixed seed
  + code/dependencies + raw response/metadata
  -> best effort, never a promise of bitwise identity

PICFAT-D PROMPT
  Persona, Instruction, Context, Format, Audience, Tone, Data

OUTPUT CONTROL: ESGV
  Examples -> Scaffolding -> Grammar/constrained decoding -> Validate/retry

JSON TRUST LADDER
  parses as JSON < conforms to schema < satisfies semantics < is factually
correct
```

The most important distinction

Level	What it guarantees	What it does not guarantee
Prompt says "return JSON"	Nothing formally; it merely guides the model	Valid JSON, schema adherence, truth
JSON mode / JSON grammar	Parseable JSON syntax, subject to documented edge cases	A particular schema, truth
Strict Structured Outputs	Conformance to the **supported subset** of the supplied schema	Truth, absence of hallucinations, cross-field meaning not encoded by the schema
Application validation	Whatever syntactic and semantic checks the program explicitly performs	Unchecked facts or assumptions

Exam sentence: Constrained decoding controls which tokens may be emitted; it does not make the selected values true.

Provider table to memorize

This is the course-relevant comparison for strict/constrained JSON generation. "Partial" means the qualification is part of the answer.

JSON Schema feature	OpenAI	Anthropic	llama.cpp converter
`additionalProperties`	Must be `false` for objects in strict mode	Must be `false`	Supports `false`, `true`, and a schema for extra values
Truly optional properties	No; every declared property must be required. Simulate an optional value with a nullable union	Yes, within complexity limits	Yes; properties absent from `required` are optional
`minimum`, `maximum`	Yes for numeric properties; not supported by fine-tuned models	No; SDKs may strip them and validate afterward	Partial: supported for **integers**, not general numbers
`pattern`, `format`	Yes on current base models (not fine-tuned models); `format` supports a documented set	Partial: regex subset and a documented format set	Partial: regex subset; formats include date, time, date-time, and UUID variants
`anyOf`	Yes, but the root must still be an object rather than `anyOf`	Yes	Yes
`oneOf`	No	No	Accepted, but converted like `anyOf`; exclusive-one semantics are not enforced

Fast comparison: **OpenAI requires all keys; Anthropic permits omitted keys; llama.cpp is broad but sometimes approximates the keyword's semantics.**

2. Reproducibility architecture: local model or remote API?

Assume that a "local model inside the container" includes the exact weights and inference software. If weights are downloaded at run time from a mutable URL, it is no longer fully self-contained.

Criterion	Local open-weight model in/with container	Remote provider API
Self-contained?	Largely yes if weights, tokenizer, runtime, configuration, and code are archived	No; depends on network, authentication, provider infrastructure, and a remotely hosted model
Artifact size	Very large - often gigabytes for weights plus runtime	Small - client code, prompts, schemas, and dependency metadata
Local hardware	High CPU/RAM and often GPU/VRAM requirements	Low; ordinary client hardware is enough
Long-term availability	Better under the experimenter's control if artifacts and licenses permit archiving	Provider may rename, update, restrict, or retire the model/API
Cost to rerun	Hardware, electricity, storage, and setup time; low marginal fee if hardware already exists	Per-token/request fees, quota, account, and possibly changing prices
Transparency	Weights and inference stack can be inspected and pinned; "open weights" still need not expose training data or training code	Usually a black box; backend implementation and updates are controlled by provider
Exact environment control	Stronger: runtime, model file, quantization, tokenizer, and decoding implementation can be pinned	Weaker: the caller controls request fields but not the serving stack
Convenience/portability	Heavy artifacts and hardware can make reproduction difficult	Easy to call from many environments while the service exists

The trade-off

A local package improves **control, inspectability, and long-term independence**, but costs storage and compute and may fail on unavailable hardware. A remote API produces a light, easy-to-run package, but externalizes critical experimental state to a changing service. Neither architecture is automatically reproducible.

Making a remote LLM experiment as reproducible as possible

Archive all state you control:

1. The exact ordered request: every developer/system/user/assistant message and every input file after preprocessing.
2. Exact model identifier and, where offered, a dated snapshot rather than a moving alias.
3. Every parameter: temperature, `top_p`, seed, reasoning effort, token limits, stop rules, tools, schemas, and response format.
4. Provider, endpoint/API version, SDK and dependency versions, request date/time, and region/service tier if relevant.
5. Complete program, lockfile/container, preprocessing, postprocessing, validation, and evaluation code.
6. Raw response before parsing plus response ID, finish/stop reason, usage, and backend fingerprint/version metadata when supplied.
7. The exact outputs used in the paper. This preserves the historical evidence even if a future rerun differs or becomes impossible.
8. Expected tolerances or evaluation criteria. Do not define success as byte identity if the scientific claim only needs semantic/statistical agreement.
9. An availability plan: budget/quota instructions, a clear error if the model is retired, and - if scientifically acceptable - a documented fallback. A fallback is a changed experiment and must be reported as such.

Do **not** archive or publish an API key. The reproducer supplies a separate credential and pays for their own calls.

3. Sampling, randomness, and repeatability

How text generation works

At every step, the model assigns probabilities to candidate next tokens. A decoder then chooses one, appends it, and repeats. For a prefix such as `I am driving a`, likely continuations such as `car` usually outrank an implausible one such as `elephant`.

Greedy decoding and the three sampling controls

Control	Mechanism	Lower setting	Higher setting
`do_sample`	Switches between greedy choice and sampling in the book's Transformers example	`False`: choose highest-probability token	`True`: sample from candidates
`temperature`	Reshapes probability distribution	Sharper, more predictable	Flatter, more diverse/random
`top_p`	Keeps the smallest token set whose cumulative probability reaches `p`	Smaller variable-size nucleus	Larger candidate pool; `1` permits all
`top_k`	Keeps exactly the `k` most probable candidates	Fewer candidates	More candidates

Key distinctions:

- In the book's local pipeline, `do_sample=False` is the executable greedy setting.
- `temperature=0` is the course/API shorthand for the most deterministic available decoding.
- `top_p` selects a **variable-size probability mass**; `top_k` selects a **fixed number**.
- Prefer changing temperature or `top_p`, rather than casually changing both, because their effects interact.

Use-case combinations from the assigned chapter

Use case	Temperature	`top_p`	Intended behavior
Brainstorming	High	High	Very diverse and unexpected
Email generation	Low	Low	Focused and conservative
Creative writing	High	Low	Creative within a smaller coherent pool
Translation	Low	High	Stable answer with broader vocabulary

Why a fixed seed and temperature 0 still do not guarantee identical bytes

- `seed` is documented as a **best effort**, not a determinism contract.
- The provider may update model weights, tokenizer, safety system, routing, quantization, kernels, batching, or inference code.
- Floating-point parallelism and accelerator kernels can be nondeterministic.
- A tiny numeric difference can change a token choice and then the whole remaining sequence.
- Load balancing can route calls to different hardware or backend revisions.
- Ties and near-ties can be resolved differently.
- Some reasoning models do not support temperature/seed controls in the same way and instead use reasoning-specific controls.
- A changed backend fingerprint is evidence that the serving configuration changed; an unchanged fingerprint still is not proof of identical output.

Therefore, the package must contain both the **recipe for rerunning** and the **raw result originally observed**.

Reproducibility target

Choose the right claim:

- **Bitwise repeatability:** exact bytes match. Often unrealistic for a remote LLM.
- **Structural repeatability:** output always satisfies the required schema.
- **Semantic repeatability:** answers convey the same meaning or classification.
- **Statistical repeatability:** aggregate performance over repeated calls stays within a stated tolerance.

A rigorous experiment states which level it tests and how agreement is measured.

4. API keys and secret handling

Never do this

- Hard-code a key in Python, a notebook, a Dockerfile, or a command committed to Git.
- use `ARG`/`ENV` in a Dockerfile to bake the key into an image layer.
- `COPY` a credential file into the image or reproduction archive.
- print the key in logs, error messages, screenshots, or CI output.
- publish your personal key so others can reproduce the work.

Environment variables reduce accidental inclusion in artifacts, but any process with suitable access may still inspect them. For deployed systems, use a proper secret manager or container secret mechanism.

---

5. Prompt engineering fundamentals

Prompt engineering is the iterative design of model input to elicit useful output. It includes instructions, context, examples, output cues, evaluation, and safeguards. There is no universal perfect prompt: behavior depends on model, data, ordering, and use case.

From completion to instruction

A bare prompt such as `The sky is` merely invites continuation. A task prompt normally has at least:

1. **Instruction:** what operation to perform.
2. **Data:** what the operation applies to.

An output indicator such as `Sentiment:` can encourage a short label rather than an essay, but it remains soft guidance.

Seven prompt components: PICFAT-D

Component	Question answered	Example
Persona	Who should the model act as?	"You are an expert research software engineer."
Instruction	What must it do?	"Extract the software, datasets, and hardware..."
Context	Why/what should it focus on?	"Focus on the points needed to reproduce..."
Format	What shape should the answer have?	"Answer with a JSON object..."
Audience	Who will consume it?	"The output is for reviewers..."
Tone	What style/voice?	"Keep the JSON values concise and factual."
Data	What source material is processed?	`Paper: <text ...>`

Three high-value prompt rules

1. **Be specific:** state task, scope, length, allowed labels, and format.
2. **Handle uncertainty:** permit an explicit `unknown`/`I don't know` outcome. This reduces pressure to invent an answer but cannot guarantee honesty.
3. **Place crucial instructions near an edge:** models often weight the beginning (**primacy**) and end (**recency**) more than information "lost in the middle."

Prompt iteration is an experiment

Build prompts modularly:

1. Start with instruction and data.
2. Add only components that have a purpose.
3. Change one component/order at a time.
4. Evaluate on representative cases and record the model/version/parameters.
5. Keep the measured improvement; remove needless text.

This is also a reproducibility issue: the exact prompt is an experimental artifact and must be versioned.

Chat roles and templates

Messages such as `user` and `assistant` are converted into the model-specific training template. The Phi-3 example becomes:

```
<s><|user|>
Create a funny joke about chickens.<|end|>
<|assistant|>
```

- Roles distinguish instructions/examples from expected answers.
- Special tokens mark sequence, speaker, and stopping boundaries.
- A template appropriate for one model need not be correct for another.
- In OpenAI's API terminology, a `developer` message provides application rules and outranks a `user` message; generated messages have the `assistant` role.

---

6. In-context learning, chaining, and reasoning patterns

Zero-shot, one-shot, and few-shot

In-context learning supplies task demonstrations in the current prompt without updating model weights. In the assigned chapter:

- zero-shot = no examples;
- one-shot = one example;
- few-shot = two or more examples.

Examples can demonstrate content, labels, tone, or output structure. They increase the chance of compliance but do **not** constrain the token stream; the model can still ignore them.

Modular prompting versus chain prompting

- **Modular prompt:** persona, instruction, context, format, audience, tone, and data are pieces of one call.
- **Chain prompting:** one call's output becomes a later call's input.

Product example: features -> product name -> slogan -> sales pitch. Chaining gives each step a narrower task and its own parameters/token budget. Costs are extra calls, latency, and **error propagation** from an early stage.

Other chain patterns:

- generate -> validate;
- several parallel answers -> merge;
- outline -> characters -> story beats -> dialogue.

System 1/System 2 analogy

- **System 1:** fast, automatic, intuitive; analogous to immediate token generation.
- **System 2:** slow, deliberate, logical/self-reflective; reasoning prompts try to mimic this process.

The chapter carefully says LLM behavior **resembles** reasoning; the analogy does not prove human-like understanding.

Compare the four multi-step techniques

Technique	Core mechanism	Evaluation	Main cost/trap
Chain prompting	Output of stage A becomes input to B	Per workflow design	Upstream error propagates
Chain-of-thought (CoT)	Generate intermediate reasoning before answer	Usually one path	More tokens; reasoning can still be wrong
Self-consistency	Sample several diverse complete reasoning paths	Majority vote over final answers	About `n` calls for `n` samples
Tree-of-thought (ToT)	Generate, rate, keep, and prune intermediate branches	At intermediate steps	Many generation/evaluation calls

Chain-of-thought

- Standard CoT shows a worked reasoning example.
- Zero-shot CoT gives a cue such as `Let's think step-by-step` without a worked example.
- In the book's apple example: `23 - 20 = 3`, then `3 + 6 = 9`.

Self-consistency

1. ask the same question multiple times;
2. allow diverse sampling paths;
3. optionally use CoT for each;
4. extract final answers;
5. return the majority answer.

Repeating deterministic greedy decoding is not useful self-consistency; diversity is the point. A majority can still be systematically wrong.

Tree-of-thought

At each step, generate several candidate thoughts, rate/vote, prune weak branches, and expand promising branches. A simpler prompt can imitate a discussion among several experts, but one model role-playing experts is not the same as truly independent evidence.

---

7. Output verification and control

Four different reasons to validate

Goal	Example	Appropriate check
Structure	Must parse as JSON	JSON parser / constrained grammar
Allowed content	Label must be one of three values	`enum`, set membership
Ethics/safety	No PII, profanity, or prohibited content	Dedicated policies/classifiers/review
Accuracy	Claim must be factual and supported	Grounding, tools, source checks, domain validation

Passing one row does not imply passing the others.

Four practical techniques

Technique	How it works	Strength	Limitation
Examples / few-shot formatting	Show correctly formatted answers	Easy; helps both content and layout	Model may ignore them
Scaffolding / slot filling	Program emits known braces/keys; model emits only unknown values	Fixed scaffolding cannot be malformed by model	Values can still be invalid or false
Grammar/schema constrained decoding	Mask every next token that would violate the supported grammar	Valid structure by construction	Provider supports only a subset; semantics/truth remain
Validate, retry, or repair	Parse/validate after generation; feed error back or use repair tool	Works with ordinary models and rich validators	More latency/cost; retries need a cap and may never succeed

The book also names **fine-tuning** as a third general output-control family alongside examples and grammar, but does not cover it in this extract.

Prompting with a schema versus constrained decoding

	Schema pasted into prompt	Proper constrained decoding
Enforcement time	Model sees it as text	Decoder checks before every token
Illegal token	Still possible	Masked/unavailable
Guarantee	Best-effort instruction following	Schema adherence within supported subset
Failure handling	Parser + validator + retry/repair	Schema can be rejected; refusals/truncation still need handling
Cost	Schema consumes context; retries may add calls	Grammar compilation/first-call latency and restricted decoding
Truth	Not guaranteed	Not guaranteed

Constrained sampling algorithm

1. Compile the supported schema/grammar.
2. At the current prefix, compute tokens that can still lead to a valid completion.
3. Mask all invalid tokens.
4. Apply temperature/`top_p` among the remaining legal candidates.
5. Sample/choose a token and repeat.

Grammar controls **admissibility**. Sampling parameters still control **which admissible value** is selected.

8. JSON essentials and `jq`

JSON mental model

JSON values are:

- object: key/value mappings in `{}`;
- array: ordered values in `[]`;
- string;
- number;
- boolean: `true` or `false`;
- `null`.

Important semantics:

- Whitespace outside strings is insignificant.
- Object key order is not semantically significant.
- Array element order **is** significant.
- `95` is a number; `"95"` is a string.
- JSON syntax alone says nothing about which keys should exist or what values mean.

9. JSON Schema

JSON Schema describes permitted document structure. A JSON instance is valid for a schema if it violates none of the applicable constraints.

Keywords to recognize

Keyword	Meaning
`type`	Required JSON type, e.g. `object`, `array`, `integer`
`properties`	Schemas for named object properties
`required`	Keys that must appear
`additionalProperties: false`	Reject undeclared object keys
`items`	Schema for array elements
`minimum`, `maximum`	Inclusive numeric bounds
`enum`	Value must be one listed choice
`pattern`	String must match a regular expression
`format`	Named string format when supported/enforced
`anyOf`	Valid against at least one listed subschema
`oneOf`	Valid against **exactly one** listed subschema

Do not confuse `properties` with `required`: declaring a property schema does not automatically require the key in ordinary JSON Schema.

`anyOf` versus `oneOf`

For subschemas A and B:

Instance matches	`anyOf`	`oneOf`
neither	invalid	invalid
only A	valid	valid
only B	valid	valid
both A and B	valid	**invalid**

11. Bowtie and reproducible validation

Different JSON Schema validators can disagree because they have different versions, draft support, bugs, or subsets. Therefore, "the file is valid" is incomplete provenance. State **which validator, which version/image, and which schema draft** produced the verdict.

What Bowtie is

Bowtie is a **meta-validator/orchestrator**. It does not supply just one validation algorithm; it runs selected validator implementations in their own container images and collects/compares the results.

Why pinned images help:

- freeze validator implementation and dependencies together;
- avoid accidental dependence on host language/package versions;
- identify exactly which implementation produced each verdict;
- make cross-language disagreements visible;
- improve later reruns if the exact image remains archived.

Limits:

- A container tag can be mutable; prefer immutable digests for strong identity.
- Container availability, host architecture, and Docker itself remain dependencies.
- Agreement among validators does not prove the data's scientific meaning is correct.

12. Structured Outputs in depth

JSON mode is not Structured Outputs

Mode	Valid JSON?	Matches specific schema?
Prompt only	Not guaranteed	No
JSON mode	Yes, barring documented edge cases/truncation	No
Structured Outputs with strict supported schema	Yes	Yes, for supported structural constraints

OpenAI strict-schema rules

- Set `strict: true`.
- Use response-format Structured Outputs when the model should answer the user in a schema; use strict function/tool schemas when the model must call application code with validated arguments.
- Every object must use `additionalProperties: false`.
- Every property must be named in `required`.
- To model a conceptually optional value, require the key but allow `null`, for example `"type": ["string", "null"]`.
- The root must be an object, not a root-level `anyOf`.
- Nested `anyOf` is supported; `oneOf` is not.
- Keywords such as `allOf`, `not`, conditional `if`/`then`/`else`, and dependency keywords are outside the documented strict subset.
- Current general models support numeric `minimum`/`maximum`, string `pattern`/selected `format`, and array bounds; fine-tuned models have narrower constraint support.

Anthropic qualifications

- Objects require `additionalProperties: false`.
- Properties not named in `required` can truly be omitted.
- `anyOf` is supported; `oneOf` is not listed as supported.
- Numerical bounds such as `minimum`/`maximum` are not enforced by constrained decoding.
- Simple regex patterns and selected string formats are supported.
- Some SDK helpers remove unsupported constraints, put the intent into descriptions, and validate the final result against the original client-side schema. That can make the request's effective schema different from the one written in application code.
- Refusal, token-limit truncation, and documented enum-casing behavior require explicit handling.

llama.cpp qualifications

The official converter turns a JSON Schema subset into a GBNF grammar:

- optional properties and additional properties are supported;
- if `additionalProperties` is omitted, the examined converter effectively emits only declared keys, which differs from ordinary JSON Schema's default of allowing extras;
- `minimum`/`maximum` are implemented for integers, not arbitrary `number` values;
- regex is supported only by the converter's subset;
- supported formats include date, time, date-time, and UUID variants, while formats such as email/URI are not fully implemented in the examined converter;
- both `oneOf` and `anyOf` enter the same union-generation branch. This admits alternatives but loses `oneOf`'s "not both" semantics when branches overlap.

15. Additional model details

Model choice and loading

- Earlier representation-focused models such as BERT are associated with classification/understanding tasks; generative pretrained transformers produce new text from prompts.
- A foundation model is pretrained on large text corpora; many task-specific models are fine-tuned from it.
- The chapter contrasts proprietary models (generally stronger, in its wording) with open models (more flexible and free to use).
- It recommends learning with a smaller model first. Phi-3-mini has 3.8 billion parameters and is described as suitable for devices with up to 8 GB VRAM.

Figure 6-1 shows model families and sizes at recognition level: Llama (7B/13B/33B/70B), StableLM (3B/7B), Falcon (7B/40B/180B), Llama 2 (7B/13B/70B), and Mistral (7B). The transferable point is that foundation-model families are released at several parameter scales.

```
model = AutoModelForCausalLM.from_pretrained(
    "microsoft/Phi-3-mini-4k-instruct",
    device_map="cuda",
    torch_dtype="auto",
    trust_remote_code=True,
)
tokenizer = AutoTokenizer.from_pretrained(
    "microsoft/Phi-3-mini-4k-instruct"
)
pipe = pipeline(
    "text-generation",
    model=model,
    tokenizer=tokenizer,
    return_full_text=False,
    max_new_tokens=500,
    do_sample=False,
)
```

Know the meanings:

- `device_map="cuda"`: place model on GPU.
- `torch_dtype="auto"`: choose appropriate model numeric type.
- `trust_remote_code=True`: allow repository-defined code; this is also a security/reproducibility dependency.
- `return_full_text=False`: omit the prompt from returned generated text.
- `max_new_tokens=500`: cap generated tokens.
- `do_sample=False`: greedy decoding.

GGUF and llama-cpp-python

- GGUF is used by llama.cpp and commonly stores compressed/quantized models.
- `n_gpu_layers=-1` places all layers on GPU.
- `n_ctx=2048` selects context size.
- Clearing an old model with `del`, `gc.collect()`, and `torch.cuda.empty_cache()` releases Python/GPU memory before loading another.

Instruction-based applications shown

Recognize supervised classification, search, summarization, code generation, and named-entity recognition. An NER prompt can name allowed entity categories and explicitly exclude verbs/adjectives.

Also recognize the output-control library names **Guidance**, **Guardrails**, and **LMQL**. The chapter presents them as tools that can constrain or validate generation; knowing a name does not imply that every tool provides the same guarantee.

16. Common exam traps

1. **Temperature 0 is not a mathematical guarantee of identical output.** It is the most deterministic supported setting.
2. **A seed is best effort.** Save the backend fingerprint and raw output too.
3. **Do not put an API key in the reproduction package.** Reproducers provide their own.
4. **A remote API container is not self-contained.** The important model state remains outside it.
5. **Open weights are not automatically reproducible.** You still need exact weight hash, tokenizer, inference engine, quantization, prompt template, hardware, and parameters.
6. **Few-shot examples guide; they do not enforce.**
7. **Scaffolding fixes only the scaffolding.** Generated slot values can still be wrong or malformed for their intended meaning.
8. **JSON mode is not schema mode.**
9. **Schema-valid is not factually correct.**
10. **`properties` does not mean `required` in ordinary JSON Schema.**
11. **`number` accepts values such as `60.5`; `integer` does not.**
12. **Minimum/maximum are inclusive.**
13. **`oneOf` means exactly one, not at least one.** Matching both branches is invalid.
14. **Extra keys are allowed by ordinary JSON Schema unless restricted.**
15. **Object key order and whitespace do not change JSON data; array order does.**
16. **Plain `diff` compares representation, not JSON meaning.** Normalize/sort first.
17. **Bowtie is an orchestrator/meta-validator, not a single new validator algorithm.**
18. **Pinned container images improve provenance but do not prove scientific correctness.**

19. **Provider feature tables need qualifications.** For example, llama.cpp supports integer bounds but not all numeric bounds.
20. **An SDK may silently transform a schema.** Record the effective request and validate application semantics afterward.
21. **A refusal or token-limit stop may fall outside the requested schema.** Handle status/stop fields before parsing.
22. **Self-consistency is not independence.** Several samples from one model can share the same bias.
23. **ToT is not merely a long CoT.** It evaluates/prunes branching intermediate alternatives.
24. **Prompt chaining is not prompt components.** Chaining spans multiple calls.

17. Final two-minute recall sheet

```
ARCHITECTURE
  Local  = big + hardware + controllable + more self-contained
  Remote = small client + cheap local hardware + API/network/provider/cost
dependency

REPEATABILITY
  temperature=0 + fixed seed + exact request/model/versions
  + response metadata + raw output
  still NOT guaranteed bitwise identical

SECRET
  os.environ["OPENAI_API_KEY"]
  docker run -e OPENAI_API_KEY ...
  never Dockerfile/Git/image/log

PROMPT = PICFAT-D
  Persona Instruction Context Format Audience Tone Data

SHOTS
  zero = 0 examples; one = 1; few = 2+

REASONING
  CoT = one stepwise path
  Self-consistency = many complete paths -> majority answer
  ToT = branch -> rate -> prune -> expand

OUTPUT CONTROL
  examples = guidance
  scaffolding = fixed wrapper
  grammar = legal tokens only
  validate/retry = check after generation

TRUST LADDER
  JSON syntax < schema < semantics < truth

JSON SCHEMA
  properties declare; required requires
  additionalProperties:false rejects extras
  anyOf >= 1; oneOf exactly 1


BOWTIE
  host-side meta-validator -> pinned validator containers -> comparable
verdicts
```

Reproducibility Engineering - Module 10 Exam Guide

1. High-yield module map

1.1 Remote experiments: overview

```
source repos + binaries + patch stack -> [Docker build environment]
                                         (it integrates these inputs)
reproducible build recipe            => [Docker build environment]  (1)
[Docker build environment]           => [experiment execution package]  (2)
[experiment execution package]       -- deploy/copy --> cloud or local HW  (3)
cloud or local HW                    => raw measured results  (4)
raw results                          => charts, tables, and paper  (5)
```

1.2 The package-content mnemonic

Memorize **B-D-D-E**:

- **Binaries** compiled for the target;
- **Data**, or a deterministic data generator;
- **Dispatcher** that runs all measurements consistently;
- **Evaluation** and visualization scripts.

A strong package also records exact versions, parameters, checksums, licenses, raw outputs, and target hardware/runtime configuration.

1.3 Secrets: the comparison to memorize

Method	Where the value is inside the container	Does `docker inspect` expose the value?	What it improves
`docker run -e OPENAI_API_KEY`	Environment variable	**Yes, plaintext**	Runtime injection; avoids baking it into the image
Compose `env_file: .env`	The **same environment variable**	**Yes, plaintext**	Keeps the literal out of the typed shell command and authored Compose YAML; centralizes local configuration
Read-only bind-mounted file	File such as `/run/secrets/openai_api_key`	**Normally no value**; it shows path/mount metadata	Keeps the value out of container environment metadata

`.env` is **not encryption**, a mounted file is not magic, and both host files must be protected. Use **both** `.gitignore` and `.dockerignore`: the first prevents commits; the second excludes files from the Docker build context.

1.5 Structured output: the guarantee ladder

Method	What is guaranteed?
Schema written only in the prompt	Nothing formal; the model may ignore it or add prose
Constrained decoding	A successful, complete output conforms to the generated grammar
Grammar from a JSON Schema	Conformity only to the schema features correctly supported by that converter/tool
Independent full-schema validation	If the validator accepts the instance, it satisfies that validator's implemented schema semantics

None of these alone proves that the answer is factually correct or fulfills a semantic request such as "return the smallest valid number." **Valid structure is not valid meaning.**

2. Remote experiments and reproduction packages

2.1 Why the ordinary container story is insufficient

Some experiments must run on a remote cloud instance, an HPC node, an ARM board, a GPU server, a measurement appliance, or other special hardware. That target may not allow Docker at all. Even when Docker runs there, measuring inside a container can change the conditions being measured.

The solution is to separate three concerns:

1. **Build in a controlled environment.** Use a recorded recipe to compile target binaries and assemble all required files.
2. **Measure on the real target.** Copy a neutral, self-contained execution package and run it with minimal target assumptions.
3. **Analyze in a controlled environment.** Copy raw results back and regenerate charts, tables, and the paper reproducibly.

The container therefore controls the **build and analysis**, while the execution package crosses the boundary to the measurement target. "Just run the same container everywhere" is not the answer to this module.

Terminology distinction:

- The **reproduction package** is the overall published/archived setup: build recipe and sources, controlled environment, deployable payload, raw results, analysis, documentation, and provenance.
- The **experiment execution package** is the self-contained deployable subset produced for the measurement target.

2.4 What must be reproducible at each stage

Stage	Preserve/control	Typical failure
Build	Source revision, compiler, libraries, build flags, patches, locale, build recipe	A mutable `HEAD` or package repository now yields different binaries
Package	Target binaries, input/generator, scripts, configuration, expected layout, checksums	A dependency is silently assumed to exist on the target
Deploy	Transfer procedure, target identity, permissions, architecture	Package works only on the author's machine
Run	Dispatcher, order, repetitions, warm-up, seeds, environment and hardware state	Manual steps or different target tunables change measurements
Analyze	Raw results, parser, statistical method, plotting and paper build	Only final plots are archived, so the derivation cannot be checked

For performance experiments, "Linux version X, machine Y, 24 GiB RAM" is not a complete environment description. Kernel extensions, frequency governors, schedulers, CPU isolation, caches, firmware, background load, compiler flags, database configuration, and many other tunables can change measurements substantially.

2.5 Terminology and the article's two pillars

The assigned article uses the course's current ACM convention:

- **Repeatable:** the **same team**, using the **same experimental setup**, reliably obtains the same result in later trials.
- **Reproducible:** a **different team**, using the **same experimental setup**, obtains the same result.

Terminology is not globally standardized, and ACM changed these definitions on **24 August 2020**. In an exam, state the convention and classify from team plus setup rather than relying on an unexplained label.

The assigned *Nullius in Verba* article advocates two broad pillars:

1. A **multi-stage reproduction package**: deterministically build executable artifacts - ideally bitwise-identical where feasible - bundle artifacts/data/instrumentation into a self-contained collection, then execute and validate measurements on the required hardware.
2. **Long-term availability**: use mature, community-supported open tools, archived formats, and conventions that have a realistic chance of remaining usable for decades.

`Nullius in verba` is the Royal Society motto commonly rendered as **"take nobody's word for it."** The reproducibility lesson is that a result should be independently inspectable and rerunnable, not merely asserted.

2.6 Long-term reproducibility

A Dockerfile or repository that builds today is not automatically a durable archive. External state changes:

- base-image tags are replaced or removed;
- package repositories and package versions change;
- runtime configuration changes beyond the distribution/kernel label;
- Git repositories disappear or move between hosts;
- projects become unmaintained;
- download links and signing keys expire;
- proprietary services, licenses, or hardware vanish.

The walkthrough's goal is a **complete, self-contained environment** that can still be built or run "on an island" without Internet access, or decades after the original repositories disappear.

A durable release should therefore archive the exact repository state, source dependencies, patches, pre-built image or reproducible image inputs, experiment package, measured data, checksums, documentation, and license information under a persistent identifier. A DOI helps locate an object; it does not prove that the object is complete or executable. The SQPolite project explicitly distinguishes its DOI-archived, external-resource-free release from its Internet-dependent from-scratch build, which is **not** expected to work forever.

The scratch Dockerfile itself makes the lesson concrete: it starts from a tag, contacts live Ubuntu/R package repositories, installs packages without complete version pins, and clones some live Git repositories. It is a useful readable recipe for rebuilding now, but external services can make the same instructions fail or produce different content later. The archived image, sources, measured data, and checksums are the durable path.

3. SQPolite blueprint

3.1 What the fictional experiment does

SQPolite modifies SQLite for an intentionally playful research question: does treating the database politely affect behavior and latency?

- An ordinary query begins with `SELECT`.

- A polite query begins with `PLEASE SELECT`.
- The modified parser accepts the new `PLEASE` keyword.
- A separate `pplease(text)` extension represents output-side politeness by randomly adding `, please.` at sentence endings about half the time.
- Impolite queries may receive a randomized delay of roughly 250-499 ms on about one quarter of eligible `SELECT` operations, creating a measurable latency difference.
- The experiment runs TPC-H queries in polite and impolite variants, records timings, plots measurements, and incorporates plots into a paper.

The purpose is not the scientific claim. The project is a blueprint showing how modified third-party software, data generation, measurements, analysis, and paper generation fit into one traceable package.

The polite and impolite workloads are paired: their substantive SQL is the same except that the polite files replace relevant `SELECT` tokens, including nested queries, with `PLEASE SELECT`. This design enables result validation while comparing latency behavior. However, the supplied `latency.c` driver passes a null row callback to `sqlite3_exec`; it aborts on SQL errors but discards returned tuples and does **not** compare them with expected results. Output validation is an architectural requirement depicted in the walkthrough, not a feature actually completed by this demo harness.

3.2 Component map

Component	Role
`lfd/sqlite` / `sqpolite`	Fork of SQLite containing the research changes
`pplease` / `pplease.so`	Custom SQLite function/extension that makes returned text more polite
`TPCH-sqlite`	SQLite-compatible TPC-H setup
`tpch-dbgen`	Deterministic TPC-H data generator
`TPCH-sqlite.diff`	Small local patch that makes the setup invoke `sqpolite` instead of `sqlite3`
`queries.impolite/`	Standard `SELECT` query workload
`queries.polite/`	Matching workload with every relevant `SELECT` changed to `PLEASE SELECT`
`prepare_data.sh`	Builds the generator/database and creates scale-factor datasets
`dispatch.sh`	Runs selected queries repeatedly and records results plus target configuration
`doall.sh`	Runs the standard polite and impolite measurement scenarios
`Dockerfile`	Constructs the build/analysis environment and the deliverable tarball
paper repository	R/knitr/LaTeX sources that turn measurements into plots and a PDF

3.3 Fork versus patch

Technique	Advantage	Caveat
Fork the upstream project	Retains upstream commit history and attribution; research commits can be reviewed and later rebased	The remote host can disappear, so archive/pin the fork
Distribute an explicit patch	Clearly exposes the exact local change; simple, stable, automatable, and not inherently tied to Git	The patch applies only to the correct pinned base revision
Copy only the final modified tree	Easy snapshot	Hides which lines came from upstream, why they changed, and who is responsible

The walkthrough uses a fork for the substantive SQLite research changes and a small patch for adapting `TPCH-sqlite`. Both strategies preserve the relationship to third-party code more clearly than an unexplained copied tree.

3.4 Clean history is communication

Research usually develops chronologically through partial commits, false starts, fixups, and transient debugging. The final reproduction package should preserve important design decisions but present them as a **logical series of orthogonal commits**: the smallest useful, reviewable increments with clear explanations and attribution.

The SQPolite fork demonstrates this contrast:

- `master` presents four clean conceptual commits: add politeness output, require/track polite input, add the impoliteness penalty, and add latency measurement.
- `devel_process` retains thirteen development commits, including fixups, renames, intermediate clocks, and later licensing cleanup.

Commit trailers such as `Signed-off-by`, `Reviewed-by`, and `Tested-by` create a trail of authorship, review, testing, and responsibility. Rewriting the presentation history is useful only before publication/shared reliance and must not erase credit or falsify what happened.

4. Building and evaluating a remote-experiment package

4.2 Result validation

Performance alone is insufficient. A fast system that returns the wrong answer has not won the benchmark. Record and check:

- exit status and error logs;
- expected versus actual query results;
- row counts, ordering rules, tolerances, or checksums as appropriate;
- whether every requested trial completed;
- raw timing values before aggregation;
- the association between a result and its exact configuration.

Choose the right equality notion: exact byte identity for deterministic artifacts, tolerance/permutation-aware equivalence where ordering or floating point permits it, and distributional/statistical comparison for stochastic or physical measurements.

4.3 Reproducibility limits

The package reduces uncontrolled variation but cannot make unlike hardware identical. CPU microarchitecture, storage, network, GPU, firmware, and thermal conditions may legitimately change performance. A good reproduction states which aspects should be identical - such as query answers and scripts - and which are expected to differ - such as absolute latency - then defines an appropriate comparison and tolerance.

6. Reproducible LLM output - temperature and seed

6.2 Temperature

For logits `z_i` and temperature `T > 0`, a common sampling rule is:

```
p_i(T) = exp(z_i / T) / sum_j exp(z_j / T)
```

- Lower `T` sharpens the distribution toward high-logit tokens.
- `T -> 0` is implemented as greedy choice of the highest-scoring token; do not literally divide by zero.
- Higher `T` flattens the distribution and increases diversity/risk.

Temperature changes **which distribution is sampled**. It does not provide a random sequence by itself.

6.3 Seed

A seed initializes the pseudorandom number generator used during sampling. With the same prompt, messages, model/weights, sampler parameters, server implementation, seed, and execution conditions, it replays the same sequence of random choices.

A seed does **not**:

- repair a changed temperature or model;
- pin dependencies, prompt templates, or model files;
- remove hardware/framework nondeterminism;
- guarantee that a remote provider routes requests through an identical backend.

Mnemonic: **temperature shapes; seed repeats**.

6.5 What must be pinned for a reproducible LLM call

Record or archive:

- exact model and weight-file checksum;
- quantization format;
- prompt, system prompt, chat template, message order, and encoding;
- sampler/server version and all generation parameters;
- seed and request order;
- context size and truncation behavior;
- client/API version;
- hardware, backend, thread/parallelism, driver, and relevant flags.

The teaching Compose file is convenient, but mutable tags such as `ghcr.io/ggml-org/llama.cpp:server`, a model URL under `main`, and `openai>=1.0` are not long-term pins. For an archival experiment, pin an image digest, model checksum/revision, and exact client version.

8. Common exam traps

1. **Container is not execution package.** In the remote workflow, the container builds/analyzes; the tarball contains the files that can run on a target without Docker. The tutorial can run measurements inside the container as a demonstration, but its README says that is usually not recommended for research measurements.
2. **Deployment is not execution.** Step 3 copies/deploys; step 4 runs and produces results.
3. **Raw result is not final paper.** Step 4 produces data; step 5 evaluates and visualizes it.
4. **Docker does not erase hardware.** It controls much of the software environment but shares the host kernel and cannot make different CPUs/storage/GPUs identical.
5. **GitHub is not an archive.** A mutable repository or live clone dependency is weaker than an immutable, checksummed archival release.
6. **DOI is not executability.** A persistent link can point to an incomplete or broken package.
7. **Fork preserves provenance, not eternity.** Archive exact fork revisions and upstream bases.
8. **Patch needs a base.** A timeless-looking diff is useful only with the exact source revision to which it applies.
9. **`doall.sh` is not automatically full Gold.** The whole final analysis, including figures and paper outputs, must be one-command automated.
10. **`.env` does not hide a container environment variable.** It improves source/command hygiene, but `docker inspect` still reveals the injected value.
11. **`.gitignore` is not `.dockerignore`.** One controls Git tracking; the other controls the image build context.
12. **Read-only is not unreadable.** `:ro` blocks writes through the mount; authorized readers still see the key.
13. **A fixed seed is not universal determinism.** Freeze the rest of the model, software, prompt, sampler, call order, and execution stack.
14. **The same seed under a different temperature does not mean the same text.** The draws address a changed distribution.
15. **Prompted JSON is not constrained JSON.** A request to obey a schema remains probabilistic.
16. **Grammar validity is not full-schema validity.** The converter may omit semantics.
17. **Schema validity is not semantic truth.** A legal value may be wrong or fail "smallest."
18. **`anyOf` is not exclusive OR.** It permits one or several matches; `oneOf` requires exactly one.
19. **An empty grammar diff can reveal a bug/limitation.** Repeatable translation is not necessarily faithful translation.

10. Final two-minute recall sheet

```
REMOTE: B-P-D-R-V
  Build controlled artifacts
  Package self-contained experiment
  Deploy/copy to cloud or local target
  Run -> raw results
  Visualize/evaluate back in controlled environment

PACKAGE: B-D-D-E
  Binaries
  Data or deterministic generator
  Dispatcher
  Evaluation scripts

ARROWS
  A -> B  means B integrates A
  A => B  means B is produced by A
  dashed   means temporal flow

LONG TERM
  pin + vendor/archive + checksum + document + license
  GitHub != archive; DOI != proof it works; Docker != hardware identity

SECRETS: E-E-F
  -e ENV: inspect shows value
  .env: still ENV; inspect shows value
  mounted File: inspect shows path/mount, not file value
```

```
  .gitignore != .dockerignore

LLM
  Temperature shapes the probability distribution
  Seed repeats pseudorandom draws
  T=0 local CPU (stated exclusions): deterministic; seed not required
  same seed + changed T/model/setup: not necessarily same text

STRUCTURE
  prompt-only schema: no formal guarantee
  constrained decoding: grammar-valid output
  only supported/transformed schema features are guaranteed
  shape != truth; valid != smallest
  anyOf >= 1 matching branch
  oneOf == 1 matching branch
```

Reproducibility Engineering - Module 11 Exam Guide

1. The module in one page

1.1 FAIR trigger-word table

Category	Trigger words	Principle IDs
Findable	persistent unique ID; rich discovery metadata; metadata names the data ID; searchable index	F1-F4
Accessible	retrieval by ID; standardized open protocol; authentication/authorization; metadata survives	A1, A1.1, A1.2, A2
Interoperable	formal shared knowledge language; FAIR vocabulary; qualified links	I1-I3
Reusable	rich relevant attributes; clear license; provenance; domain standards	R1, R1.1-R1.3

Fast traps:

- A license is **Reusable**, not Accessible.
- A persistent identifier is **Findable**, not Accessible.
- Metadata surviving after deletion is **Accessible**.
- A formal representation language is **Interoperable**.
- Domain-relevant community standards are **Reusable**, while commonly adopted exchange standards/formats are usually tested as **Interoperable**.

1.2 Legal scenario decision table

Clue	Main framework	Test
Identifiable living person	**GDPR/data protection**	Processing needs consent or another lawful basis.
Original figure, code, schema, view, GUI, API implementation	**Copyright**	Sufficient original intellectual creation/expression.
Customer list or confidential know-how	**Trade secret**	Not generally known/readily accessible, commercially valuable because secret, and protected by reasonable measures.
Database instance funded through substantial collection/checking/presentation effort	**EU sui generis database right**	Substantial qualitative or quantitative investment in obtaining, verifying, or presenting contents.
User accepted click-through, NDA, README terms, or license	**Contract/license**	Binds the assenting parties according to its terms.
Plain public facts or measurements alone	**Normally none of the first three**	But the collection, schema, contract, or database instance may still be protected separately.

Always identify the **layer** before naming the law:

```
record/content -> selection/schema/view -> code/interface -> database instance
-> access contract
```

Several layers can be protected at once and by different people.

1.3 Sui generis test in six questions

1. Is there a database: systematically/methodically arranged and individually accessible items?
2. Is the maker eligible, usually established in the European Economic Area (EEA)?
3. Was there substantial investment?
4. Was that investment in **obtaining, verifying, or presenting** contents, rather than simply creating the information?
5. Was all or a substantial part extracted/re-utilized, or were small parts taken repeatedly and systematically until equivalent?
6. Did the relevant act occur in the EEA, and is there a license or narrow exception?

Case mnemonic:

```
BHB created race information       -> not protected
Toll Collect obtained sensor facts -> protected
```

2. FAIR data stewardship

2.1 What FAIR applies to

FAIR concerns **scholarly digital research objects**, not only the final table. This includes:

- research data;
- algorithms used to process/analyze it;
- software tools;
- workflows and analytical pipelines.

Physical laboratory equipment is outside that digital-object scope, although metadata describing equipment can itself be FAIR.

The stakeholders include original researchers, researchers who want to reuse work, professional data publishers/repositories, funders, tool builders, and **computational agents**. The last group is central: modern data volume, variety, and speed make human-only discovery and integration inadequate.

2.2 Machine-readable versus machine-actionable

Machine-readable means software can parse the syntax. **Machine-actionable** is stronger: an agent can infer enough explicit meaning, permissions, relationships, and processing information to decide what to do with a previously unseen object with little or no human help.

Ask two separate questions:

1. Metadata: **What is this object, and may I use it?**
2. Content: **How should I process or integrate it?**

A CSV is machine-readable, but columns called `x`, `y`, and `z` with no units, provenance, license, or vocabulary are not very machine-actionable.

2.3 The 15 principles to memorize

Memory chain:

```
F: PID - Rich - Link - Index
A: Retrieve - Open - Auth - Metadata survives
I: Language - Vocabulary - Qualified links
R: Rich - License - Provenance - Standards
```

Findable

ID	Exam-ready wording	Why it exists
F1	(Meta)data have a **globally unique, persistent identifier**.	Stable, unambiguous reference.
F2	Data have **rich metadata**.	Discovery requires description beyond a filename.
F3	Metadata clearly include the **identifier of the data** described.	Keeps description and object linked when separated/indexed.
F4	(Meta)data are **registered or indexed in a searchable resource**.	An identifier is useless if nobody can discover it.

Accessible

ID	Exam-ready wording	Why it exists
A1	(Meta)data are retrievable by identifier through a **standardized communications protocol**.	Predictable automated retrieval.
A1.1	The protocol is **open, free, and universally implementable**.	No proprietary client gate.
A1.2	The protocol supports **authentication and authorization when needed**.	FAIR accommodates controlled data.
A2	**Metadata remain accessible** even after the data disappear.	Preserves citation, provenance, and knowledge that the object existed.

Interoperable

ID	Exam-ready wording	Why it exists
I1	(Meta)data use a **formal, accessible, shared, broadly applicable knowledge-representation language**.	Machines need common syntax and semantics.
I2	(Meta)data use vocabularies that are **themselves FAIR**.	A disappearing/opaque vocabulary breaks interpretation.
I3	(Meta)data contain **qualified references** to other (meta)data.	The link states the relationship, not merely a URL.

Reusable

ID	Exam-ready wording	Why it exists
R1	(Meta)data have a plurality of **accurate, relevant descriptive attributes**.	A potential user can judge fitness for purpose.
R1.1	(Meta)data have a **clear, accessible usage license**.	Machines and people can know permitted use.
R1.2	(Meta)data have **detailed provenance**.	Users can trace source, transformations, and responsibility.
R1.3	(Meta)data follow **domain-relevant community standards**.	Reuse requires the conventions of the intended field.

2.4 FAIR does not mean open

This is the most important conceptual trap:

- A1.1 requires an **open protocol**, not publicly downloadable data.
- A1.2 explicitly permits authentication and authorization.
- R1.1 requires a **clear** license, not necessarily a permissive license.
- Sensitive data can have public metadata and a documented application process while the data remain restricted.

Therefore, a well-described controlled health dataset can be more FAIR than an openly downloadable but undocumented file.

Related traps:

- FAIR is not automatically **free**.
- FAIR is not automatically **ethical** or legally reusable; the license and lawful basis still matter.
- FAIR is not a binary badge. Objects can satisfy different principles to different degrees.
- A DOI helps F1 but does not make the object fully FAIR.

2.5 Principles, not an implementation standard

FAIR is a set of high-level, implementation-neutral guideposts. It does not mandate DOI, HTTP, RDF, JSON, XML, HDF5, or a specific repository. Those are possible implementation choices.

Prefer **general-purpose, open interoperability standards**. A bespoke parser per type does not scale across repositories, languages, future tools, or previously unknown data.

If a domain vocabulary lacks a term, extend a suitable vocabulary or publish a new one with persistent documentation so the vocabulary itself follows FAIR (I2).

2.6 F2 versus R1, and other near-neighbors

Pair	Difference
F2 rich metadata vs **R1 rich attributes**	F2 emphasizes enough description to **find** the object; R1 emphasizes enough accurate context to **judge and reuse** it. They reinforce each other.
A1.1 open protocol vs **open data**	Protocol implementation is open; the data may be access-controlled under A1.2.
I1 formal language vs **R1.3 domain standard**	I1 is a machine-interpretable representation language; R1.3 is the community convention needed for valid domain reuse.
I3 qualified link vs bare URL	A qualified reference expresses the relationship, such as derived-from or measured-by.
F1 ID vs **F4 index**	An object can have a persistent ID yet remain undiscoverable if no searchable resource indexes it.

2.7 Why the principles were proposed

The 2014 Leiden workshop **Jointly Designing a Data Fairport** brought academic and private stakeholders together. A FORCE11 group refined the resulting minimal community principles, formally published by Wilkinson et al. in 2016.

The paper's main argument:

- scientific data grows faster than humans can manually curate and integrate;
- specialist repositories provide strong conventions but cannot hold every new data type;
- general repositories accept variety but may not harmonize it;
- machines therefore need persistent identifiers, explicit semantics, standardized access, licenses, and provenance;
- good stewardship is not the final scientific goal, but a precondition for discovery and innovation.

3. Legal protection: identify the layer first

3.1 Why data rarely has one owner

The word **ownership** is misleading. Pure information/facts generally do not belong to somebody in the same way as a physical object. Instead, different

rights govern different aspects and coexist:

```
personal person-linked content           -> data protection
original record/figure/text/image        -> copyright in content
confidential commercially valuable fact  -> trade-secret protection
original code/schema/view/interface      -> copyright
database instance with qualifying obtaining/verifying/presenting investment ->
sui generis database right
agreed access/use conditions              -> contract or license
```

One database can contain all of these at the same time, held by different actors. Obtaining permission from one holder does not erase the others.

3.2 Research data and openness

The Open Data Directive's research-data concept covers digital documents other than scientific publications that are created/collected during research and used as evidence or accepted as needed to validate findings.

The rule is **as open as possible, as closed as necessary**, not publish everything. The paper describes a narrow obligation for publicly funded data that have already been made public through a repository, subject to commercial interests, knowledge transfer, privacy, secrecy, and earlier IP rights.

Public visibility still does not answer who may append, modify, or delete.

3.3 The three content frameworks

GDPR/data protection

Applies to data linked to an identifiable **living person**. Consent is one possible lawful basis, not the only one. GDPR regulates processing and duties; it should not be described as ordinary property ownership.

Examples: names with identifiers, patient records, identifiable survey responses. Pseudonymized data may remain personal data, and combining datasets can make previously de-identified records identifiable again.

Copyright

Protects original expression, not raw facts. It can apply to an original figure, text, photograph, code, creative selection/arrangement, schema, or interface. A copyrighted Bart Simpson image remains copyrighted when stored as a BLOB; the database location does not change the content right.

Trade secrets

The course test requires all three:

1. not generally known or readily accessible;
2. commercial value because it is secret;
3. reasonable protective measures, such as NDAs and technical access controls.

A confidential customer list is the canonical example. A vague request to keep something quiet is weaker than an NDA plus authentication, logging, and intrusion protection.

3.4 Protection by database-application component

The 2023 article maps the classic Model-View-Controller system plus database layers as follows.

Component	Typical protection	Exam nuance
View/GUI	General copyright if original	Visual/interface expression can be protected.
Controller/application/DBMS	Software copyright	Includes sufficiently original stored procedures and UDFs.
Model	Software and/or general copyright	Depends on whether code or original expression/structure is at issue.
Schema/views/selection	Copyright as an original database work	Original selection/arrangement; trivial functional structure fails.
Database instance	Sui generis database right, if there is substantial qualifying investment	Obtaining/verifying/presenting investment, not creativity or mere data creation.
Individual record	Normally no database right by itself	May independently contain personal, copyrighted, or secret content.

Interface nuance: the 2023 article says the underlying **idea/principle or bare interface specification** is not automatically software-copyrighted; an original implementation, documentation, or expressive API design may be.

3.5 Schema originality

The test is original selection/arrangement, not complexity for its own sake.

Copyright normally begins with the natural person(s) who created the protected expression; an institution may receive rights through employment rules or contract. This differs from the database producer right, which arises for the investor and may belong directly to an organization.

3.6 Rights in one research collaboration

The 2024 article's example contains:

- researchers A/B/C organizing data;
- University Y and funder I financing the work;
- manufacturer M providing internal data under NDA;
- students S/T writing code and designing the UI.

Possible result:

- A/B/C hold copyright if their selection/structure is original;
- Y or I may hold the sui generis right as substantial investor;
- S/T may hold copyright in code/UI unless assigned;
- M retains trade-secret protection;
- personal data would add GDPR duties.

Publication therefore needs all relevant permissions, or exclusion of the protected component. There is no automatic rule that "the professor" or "the university" owns everything.

---

4. The EU sui generis database right

4.1 What it protects

The right protects a **database instance as a collection**, independently of copyright in content, schema, code, or interface. It rewards a qualitatively or quantitatively substantial investment in **obtaining, verifying, or presenting** the contents. It can block extraction or re-utilization of all or a substantial part.

Core comparison:

Copyright/database work	Sui generis database right
Protects sufficiently original expression, selection, or arrangement	Protects qualifying investment in the database instance
Creativity/originality matters	Creativity does not matter
Often begins with a human author	Often held by institution/funder/company that bore the cost
Can protect schema, code, GUI, content	Protects the accumulated instance

The ordinary term is **sui generis database right**; the German article calls it the **Datenbankherstellerrecht** (database producer/maker right).

4.2 What counts as a database

The legal definition is broad: independent items arranged systematically or methodically and individually accessible.

- A sensible sort order is not required.
- IDs, line numbers, paths, or full-text search can make items individually accessible.
- Digital form is not essential; a paper card index can qualify.
- A truly random heap whose items cannot be stably referenced is less likely to qualify.

4.3 Scope of prohibited extraction

- Taking all records: can infringe.
- Taking a qualitatively or quantitatively substantial part: can infringe.
- Taking one insignificant record once: normally not under this right.
- Repeatedly and systematically taking small pieces until the collection is effectively copied: can infringe; the rule prevents circumvention.

An individual record may still have its own GDPR, copyright, or secrecy protection.

4.4 Obtain versus create - the decisive distinction

Investment counts when directed to finding/collecting existing information, checking it, or presenting it. Cost spent creating the underlying events/information does not automatically count.

British Horseracing Board v William Hill

BHB spent heavily organizing races and generating lists of runners, riders, and dates. The European court treated the decisive expense as creating the underlying race information rather than obtaining independent existing content. **Database not protected on that investment theory.**

Toll Collect

Toll Collect installed terminals/devices that captured externally existing facts about truck use. The German court treated that effort as obtaining database contents. The calculated toll amount itself is generated, but the measured inputs are collected. **Database protected.**

Mnemonic:

```
create the event/schedule -> BHB -> no
capture external facts    -> Toll -> yes
```

The distinction is controversial for sensors, scraped data, derived metadata, and AI because real systems mix acquisition and generation.

4.5 Holder and duration

The right belongs to the party bearing the qualifying investment.

The 2023 article states a 15-year term. A new substantial investment can restart protection for the updated instance, potentially extending protection repeatedly.

4.6 Territoriality

The course model has two distinct territorial checks:

1. **Maker eligibility:** generally an EEA maker/institution.
2. **Place of conduct:** the database right prohibits relevant acts in the EEA.

4.7 Contracts invert the practical default inside and outside the EEA regime

A contract/license binds people who accepted it; it normally does not bind a stranger who obtained the data from another person and never assented. Statutory copyright, trade-secret, or database rights can reach outsiders independently.

The 2023 article's exam-worthy inversion:

- Outside the EEA, in a jurisdiction with no equivalent producer right, the maker needs a valid pre-access contract to create restrictions. Without assent, unprotected facts may be reused.
- Within the EEA regime, silence preserves the qualifying maker's statutory right. A license is what gives the **user** safe permission.

Thus: **outside the regime, the license protects the maker; inside the EEA regime, it often protects the user.** This wording includes Norway, Iceland, and Liechtenstein, which are outside the EU but inside the EEA.

4.8 Licenses and the research exception

Database rights need a license that actually covers them. The 2023 source highlights CC 4.0-era licenses and choices such as:

- **CC0:** waiver/dedication with no attribution condition;
- **CC BY:** attribution;
- **CC BY-SA:** derivatives under the same license;
- **CC BY-ND:** redistribution only unchanged;
- **NC:** non-commercial restriction, whose boundary can be uncertain;
- database-focused alternatives: ODbL, ODC-By, PDDL, CDLA.

The license must come from the relevant holder(s), and rights in content/code/schema/instance may require several compatible permissions.

The non-commercial scientific-research exception described by the source is narrow: since 2019 it can permit copying a substantial part to the extent required for non-commercial scientific research, but **not a complete copy**, distribution, or making that copy publicly available. It also supplies no corresponding exception for computer programs such as the DBMS or stored procedures, and the source excludes third-party-funded research for private companies.

5. Legal perspectives on research-data storage

5.1 The paper's central thesis

Artifact sharing improved, but long-term storage still fails when people graduate, teams split, affiliations change, sensitive material enters a dataset,

repository terms change, or funding ends. The result is a dynamic **nexus of contracts**, not one permanent owner/data relationship.

The proposed solution is **legal-by-design research-data infrastructure**:

1. identify actors, rights, foreseeable changes, and exit rules early;
2. create reusable but project-adaptable contract templates;
3. translate those rules into permissions, views, logging, security, portability, and retention;
4. avoid a gap between what is legally allowed and what the system technically permits.

5.2 Stakeholders and their interests

Stakeholder	Role and likely interest
Data providers	Supply content that may be personal, copyrighted, or secret; attach consent/license/NDA conditions.
Individual researchers	Collect and organize data; need academic freedom, collaboration, attribution, and continued access.
Research institutions/funders	Finance staff/infrastructure; may hold producer rights, impose policies, and contract with repositories.
Support personnel	Build schemas, software, and UIs; may initially own copyright unless assigned/licensed.
Data repositories/infrastructure providers	Store/process data and enforce technical access; storage alone gives no ownership, but provider terms matter.
Interested third parties/public	Reviewers, collaborators, reusers, commercial users; need varying read/use rights and restrictions.

Researchers are not a single stable legal entity. People join and leave, may belong to different institutions, and may acquire new rights in modified/forked databases without eliminating upstream rights.

5.3 Controller versus processor

For personal data, the research institution is usually the **controller** in the paper's ordinary scenario; an external repository is usually a **processor** acting on its behalf. GDPR Article 28 requires a written processing arrangement defining responsibilities and safeguards.

Outsourcing does **not** outsource responsibility. The controller must require and supervise suitable technical and organizational measures. Similar contractual needs can arise for trade secrets.

5.5 The institution as the contractual spider

The institution usually employs researchers/support staff and contracts with infrastructure providers, so it sits at the web's center. But no arrangement is perfect:

- institution control can subordinate research to cost, bandwidth, or license policies;
- researcher control can lack legal advice and actual system-control capability;
- multi-institution projects need a leading party and governance rules;
- large cloud providers often offer take-it-or-leave-it conditions;
- academic freedom prevents treating university researchers exactly like ordinary private employees.

A robust approach is to create a standard institutional framework, disclose its core terms to researchers, allow project-specific changes, and anticipate multi-institution/third-party cases.

5.6 When a researcher changes affiliation

Under the German default described by the paper, an individual researcher may retain copyright in their work while the old institution has a non-exclusive use right. But copyright is not physical or technical access:

- uncopyrighted data may remain on the old system;
- schema/code may belong to another contributor;
- support-personnel rights may have been assigned to the institution;
- repository and data-provider contracts do not automatically follow the researcher;
- a huge or sensitive database cannot always be copied onto a drive;
- duplicated forks create synchronization and integrity problems.

Plan in advance for export, read/write access, cost, contract assignment/joinder, ongoing NDA/license permission, membership voting, and departure/exclusion.

5.7 Datasets change their legal character

During active research:

- new columns can introduce personal data;
- joining sources can re-identify pseudonymized people;
- copyrighted or confidential content can contaminate a previously open dataset;
- volume and compute requirements can outgrow the original provider contract.

Therefore security/classification cannot be a one-time upload check. Use periodic reviews, scalable access control, logging, and adaptable contracts. Maximum restriction may reduce legal risk but can also make science impossible.

5.8 Reviewer and third-party access

Before publication, the later public license may not apply. Reviewers normally need read-only, limited access; the system may need filtering, pseudonymization, quotas, logging, and watermarking. A code of honor is insufficient for GDPR or trade secrets.

Double-anonymous review creates a design conflict:

- institutional hosting/domain/logs may reveal author affiliation;
- third-party hosting protects anonymity but weakens direct supervision;
- a neutral independent or multi-institution platform should conceal identity while preserving evidence for a proven violation.

5.9 Integrity, security, and availability

Protect against unauthorized access, device failure, transmission errors, inconsistent replicas, and lost providers. Sensitivity should drive stronger measures. The paper distinguishes legal confidentiality duties from broader research-ethical expectations for integrity and redundancy.

For reproducibility, security is not enough: a perfectly confidential file that disappears after project funding still fails future validation.

5.10 License choice and repository power

Repositories often offer a fixed license menu, but several holders may need unanimity. Decide early. A repository cannot mine customer data merely because it stores it; a storage contract is not a use license, and a copyright text/data-mining exception does not legalize personal-data processing.

License shorthand:

License element	Meaning
CC0	Waive/dedicate claims; no legal attribution condition. Scientific ethics may still require citation.
BY	Attribution/source/license.
SA	Adapted/derivative material remains under the same license.
ND	Redistribution only without adaptation.
NC	No commercial use; boundary can be uncertain.

5.11 Storage termination

When funding ends, storage and compliance still cost money. A provider may terminate, fail, enter bankruptcy, merge, or use foreign subcontractors. Contracts need retention and exit rules, but a verified local backup remains advisable.

5.12 Legal-paper traps

1. Publicly funded does not automatically mean public.
2. Publicly accessible does not mean licensed for reuse.
3. GDPR protects people; it is not ordinary ownership of facts.
4. Repository storage does not grant the repository data ownership.
5. A contract binds assenters, not automatically every later recipient.
6. One license cannot authorize rights held by somebody else.
7. Researcher copyright does not guarantee access to the instance.
8. Research exceptions are not universal immunity.
9. Pseudonymization is not necessarily anonymization.
10. Outsourcing to a processor does not remove controller responsibility.
11. A public-release license may not govern prepublication review.
12. Design permissions and termination before the conflict, not afterward.

---

6. Multi-stage Docker builds

6.1 Why use two stages

Compiling requires GCC, headers, linker, libraries, shell, and source. Running a statically linked program needs almost none of them.

```
single-stage gcc:14 image
  compiler + headers + shell + package tools + source + binary

multi-stage final image
  /mentos only
```

A second `FROM` starts a new filesystem. `COPY --from=builder` selects an artifact from the earlier stage; it does not merge the whole builder. Builder layers may remain in local build cache, but they are not in the final runtime image.

Benefits:

- far smaller transfer/storage footprint;
- smaller attack surface;
- clean build/runtime dependency separation;
- fewer accidental runtime files.

It does not automatically make compilation deterministic or portable to a different CPU architecture/kernel ABI.

7. Remote execution and provenance

7.1 Why not run the builder image remotely

A real target may offer only SSH, lack Docker, or need direct access to special hardware. The controlled environment builds a self-contained package; the target performs the measurement; results return to a controlled analysis environment.

Mnemonic:

```
B-P-S-R-C-A
Build -> Package -> Ship -> Run -> Collect -> Analyze
```

The target still matters. Containers do not erase CPU, GPU, RAM, kernel, driver, storage, or resource-limit differences.

8. HDF5 storage and inspection

8.1 The data model

Mnemonic: **F-G-D-A**.

```
File
└── Group       directory-like named container
    └── Dataset homogeneous n-dimensional typed array
        └── Attribute small metadata attached to an object
```

HDF5 is a binary, hierarchical, self-describing format suited to large arrays and partial reads. "Self-describing" means names, hierarchy, shapes, datatypes, and attributes travel with the values; it does not mean the file automatically documents every scientific assumption.

Rule of thumb:

- group = organize;
- dataset = bulk/sliceable data;
- attribute = small descriptive fact.

8.3 Cheatsheet functions and modes

```
h5py.File(filename, "w")
```

Creates/truncates a file for writing. **`w` overwrites an existing file.**

```
h5py.File(filename, "r")   # read-only, must exist
h5py.File(filename, "r+")  # read/write, must exist
```

Core operations:

```
f.create_group(name)
g.create_dataset(name, data=data)
f[path]
x.attrs[attr] = value
```

Use a context manager so data is flushed and the file closes even after an error:

```
with h5py.File("measurements.h5", "r") as h5_file:
    ...
```

`f[path]` returns an HDF5 object/proxy. To read values use `dataset[:]`, `dataset[...]`, or `dataset[()]`. Printing the proxy alone does not print its array.

8.9 HDF5 versus JSON

Property	HDF5	JSON
Encoding	Typed binary	Text
Hierarchy	Groups/datasets	Objects/arrays
Shape and datatype	Native dataset metadata	Application convention
Arbitrary numeric slices	Direct dataset selections	No intrinsic file index
Large numerical size	Usually compact	Text digits, punctuation, and repeated keys add overhead
Human-readable/diffable	Poor	Good
Images	Native numerical datasets/image convention	No native image/binary type; indirect encoding needed

Nuances:

- Tiny HDF5 files can be larger than tiny CSV/JSON because metadata overhead dominates.
- HDF5 is not automatically compressed; compression/chunking must be configured.
- Efficient arbitrary slicing at scale depends on a sensible chunk layout/access pattern.

9. Cross-topic connections likely to earn explanation marks

FAIR and law are compatible, not opposites

FAIR A1.2 permits controlled access; legal rules may require it. A restricted dataset can still expose searchable metadata, a persistent ID, a standardized authenticated access procedure, a clear license/application rule, provenance, and domain standards.

```
FAIR asks: can an authorized agent find, understand, and request/reuse it?
Law asks: who is authorized, under what basis, for what action?
```

FAIR metadata should accompany remote results

The remote workflow already captures provenance. To improve FAIRness:

- assign a persistent release identifier;
- index the package/results;
- describe target hardware/software and inputs richly;
- use standard formats/protocols/vocabularies;
- state licenses for code, data, and instance rights;
- link raw results to source commit, artifact checksum, analysis, and paper with qualified relationships.

HDF5 helps but does not guarantee FAIRness

HDF5 supplies typed shapes, hierarchy, and attached attributes, supporting interpretation and partial access. A file called `result.h5` on a laptop with no ID, index, license, provenance, community vocabulary, or access protocol is still not FAIR.

Small image versus reproducible experiment

A tiny `scratch` image improves distribution and attack surface, but not every reproducibility dimension. Pin source/base digests, preserve compiler identity, checksum the binary, capture target hardware, return raw data, automate analysis, and license the artifacts.

Legal permission is part of reproducibility

An executable workflow that nobody is allowed to access/use is not practically reproducible. Conversely, a permissive license cannot reconstruct missing dependencies or provenance. Technical and legal availability must align.

---

10. Common exam traps

1. **FAIR is not open data.** Open protocol can carry restricted data.
2. **Machine-readable is not machine-actionable.** Parsing is weaker than autonomous interpretation/use.
3. **FAIR includes more than data.** Algorithms, tools, and workflows are digital objects too.
4. **FAIR is not a technical standard.** It precedes implementation choices.
5. **DOI is not full FAIRness.** It mainly supports F1.
6. **License belongs under Reusable.** Not Accessible.
7. **Authentication is allowed.** A1.2 explicitly says so.
8. **Facts are not automatically owned.** Separate protections may still apply.
9. **GDPR is not copyright.** It protects people in processing contexts.
10. **Schema is not instance.** Original schema -> copyright; qualifying-investment instance -> sui generis.
11. **Copyright is not investment protection.** Originality versus qualifying cost.
12. **Create is not obtain.** BHB loses; Toll Collect wins in the course cases.
13. **One record is not automatically a substantial part.** Repeated systematic extraction can aggregate.
14. **Public website is not a license.** Open access does not equal legal permission.
15. **US maker and US conduct are two different territorial reasons.** Check both maker and act.
16. **Research is not a blanket exception.** Scope and redistribution are limited.
17. **Repository is not owner merely by storage.** But its contract/technical framework matters.
18. **Contract binds assenters.** It does not automatically bind an innocent outsider.
19. **Outsourcing does not transfer controller responsibility.** Supervision remains.
20. **Second `FROM` starts fresh.** Builder files do not leak unless copied.
21. **`scratch` has no dynamic loader or shell.** Use a static binary and exec-form entrypoint.
22. **Static is not cross-architecture.** CPU/kernel compatibility still matters.
23. **`tmux` is persistence across disconnect, not reboot and not determinism.**
24. **HDF5 dataset is not a group.** Dataset stores the typed array.
25. **Attribute is metadata, not a new array row.**
26. **Shape does not reveal NaNs/type/decimal places.** `{120,2}` only gives dimensions.
27. **`h5dump -H` hides values, not metadata.**
28. **Printing an h5py proxy does not load data.** Slice it.
29. **`w` truncates.** Never use it for readback.
30. **HDF5 is not automatically compressed or optimally chunked.**
31. **JSON has no native image type, but can store encoded image data.** Distinguish the literal ability to store pixels/Base64 from native binary/image datatype support.

---

12. Final two-minute recall sheet

```
FAIR
  F = PID, rich metadata, metadata->data ID, searchable index
  A = retrieve by ID, open protocol, auth allowed, metadata survives
  I = formal language, FAIR vocabulary, qualified links
  R = rich attributes, license, provenance, community standards
  accessible != public; FAIR != implementation standard
  machine-readable < machine-actionable

LAW: identify the layer
  person-linked content -> GDPR
  original expression   -> copyright
  protected valuable secret -> trade secret
  qualifying-investment DB instance -> sui generis
  accepted terms        -> contract/license

SUI GENERIS
  instance + substantial investment
  obtain / verify / present, not merely create
  all/substantial extraction OR systematic small extraction
  holder = investor; usually institution
  BHB no; Toll Collect yes
  maker eligibility + place of act


REMOTE: B-P-S-R-C-A
  Build Package Ship Run Collect Analyze

HDF5: F-G-D-A
  h5ls -r = layout
  h5dump -d = one dataset details/data
  h5dump -H = all metadata, no values
```